

尚硅谷大数据技术之 Hudi

(Hudi & 尚硅谷 联合出品)

版本: V1.0

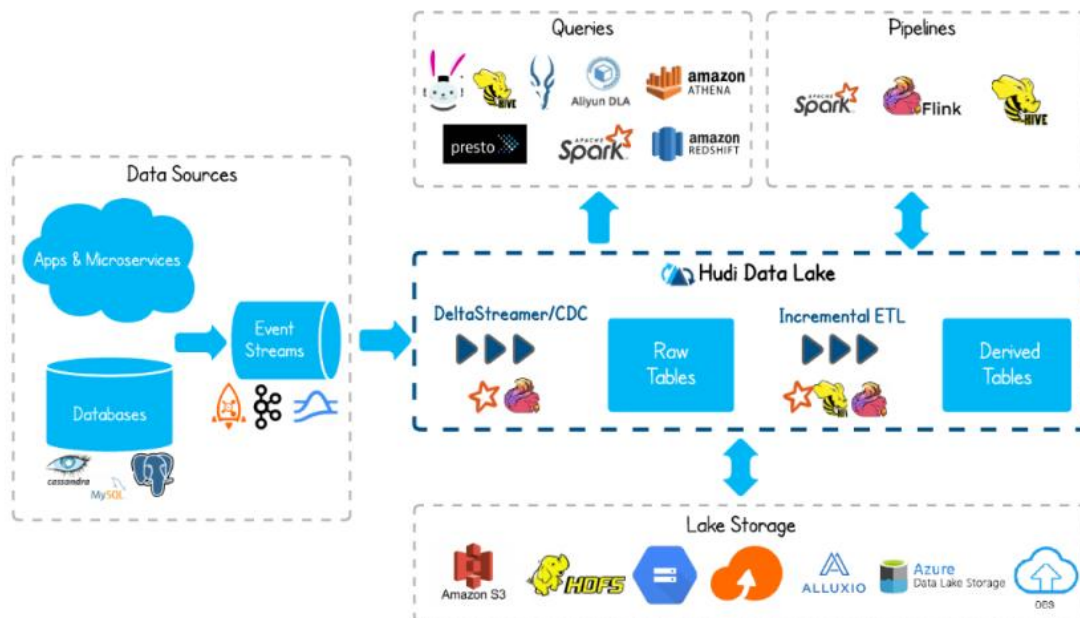
第 1 章 Hudi 概述

1.1 Hudi 简介

Apache Hudi (Hadoop Upserts Delete and Incremental) 是下一代流数据湖平台。Apache Hudi 将核心仓库和数据库功能直接引入数据湖。Hudi 提供了表、事务、高效的 upserts/delete、高级索引、流摄取服务、数据集群/压缩优化和并发，同时保持数据的开源文件格式。

Apache Hudi 不仅非常适合于流工作负载，而且还允许创建高效的增量批处理管道。

Apache Hudi 可以轻松地在任何云存储平台上使用。Hudi 的高级性能优化，使分析工作负载更快的任何流行的查询引擎，包括 Apache Spark、Flink、Presto、Trino、Hive 等。





1.2 发展历史

- 2015 年：发表了增量处理的核心思想/原则（O'reilly 文章）。
- 2016 年：由 Uber 创建并为所有数据库/关键业务提供支持。
- 2017 年：由 Uber 开源，并支撑 100PB 数据湖。
- 2018 年：吸引大量使用者，并因云计算普及。
- 2019 年：成为 ASF 孵化项目，并增加更多平台组件。
- 2020 年：毕业成为 Apache 顶级项目，社区、下载量、采用率增长超过 10 倍。
- 2021 年：支持 Uber 500PB 数据湖，SQL DML、Flink 集成、索引、元服务器、缓存。

1.3 Hudi 特性

- 可插拔索引机制支持快速 Upsert/Delete。
- 支持增量拉取表变更以进行处理。
- 支持事务提交及回滚，并发控制。
- 支持 Spark、Presto、Trino、Hive、Flink 等引擎的 SQL 读写。
- 自动管理小文件，数据聚簇，压缩，清理。
- 流式摄入，内置 CDC 源和工具。
- 内置可扩展存储访问的元数据跟踪。

- 向后兼容的方式实现表结构变更的支持。

1.4 使用场景

1) 近实时写入

- 减少碎片化工具的使用。
- CDC 增量导入 RDBMS 数据。
- 限制小文件的大小和数量。

2) 近实时分析

- 相对于秒级存储 (Druid, OpenTSDB)，节省资源。
- 提供分钟级别时效性，支撑更高效的查询。
- Hudi 作为 lib，非常轻量。

3) 增量 pipeline

- 区分 arrivetime 和 event time 处理延迟数据。
- 更短的调度 interval 减少端到端延迟 (小时 -> 分钟) => Incremental Processing。

4) 增量导出

- 替代部分 Kafka 的场景，数据导出到在线服务存储 e.g. ES。

第 2 章 编译安装

2.1 编译环境准备

本教程的相关组件版本如下：

Hadoop	3.1.3
Hive	3.1.2
Flink	1.13.6, scala-2.12
Spark	3.2.2, scala-2.12

1) 安装 Maven

(1) 上传 apache-maven-3.6.1-bin.tar.gz 到 /opt/software 目录，并解压更名

```
tar -zxvf apache-maven-3.6.1-bin.tar.gz -C /opt/module/
mv apache-maven-3.6.1 maven-3.6.1
```

(2) 添加环境变量到 /etc/profile 中

```
sudo vim /etc/profile
#MAVEN_HOME
export MAVEN_HOME=/opt/module/maven-3.6.1
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
export PATH=$PATH:$MAVEN_HOME/bin
```

(3) 测试安装结果

```
source /etc/profile
mvn -v
```

2) 修改为阿里镜像

(1) 修改 setting.xml, 指定为阿里仓库地址

```
vim /opt/module/maven-3.6.1/conf/settings.xml
```

```
<!-- 添加阿里云镜像-->
```

```
<mirror>
```

```
    <id>nexus-aliyun</id>
```

```
    <mirrorOf>central</mirrorOf>
```

```
    <name>Nexus aliyun</name>
```

```
<url>http://maven.aliyun.com/nexus/content/groups/public</url>
```

```
</mirror>
```

2.2 编译 Hudi

2.2.1 上传源码包

将 hudi-0.12.0.src.tgz 上传到/opt/software, 并解压

```
tar -zxvf /opt/software/hudi-0.12.0.src.tgz -C /opt/software
```

也可以从 github 下载: <https://github.com/apache/hudi/>

2.2.2 修改 pom 文件

```
vim /opt/software/hudi-0.12.0/pom.xml
```

1) 新增 repository 加速依赖下载

```
<repository>
```

```
    <id>nexus-aliyun</id>
```

```
    <name>nexus-aliyun</name>
```

```
<url>http://maven.aliyun.com/nexus/content/groups/public</url>
```

```
    <releases>
```

```
        <enabled>true</enabled>
```

```
    </releases>
```

```
    <snapshots>
```

```
        <enabled>>false</enabled>
```

```
    </snapshots>
```

```
</repository>
```

2) 修改依赖的组件版本

```
<hadoop.version>3.1.3</hadoop.version>
```

```
<hive.version>3.1.2</hive.version>
```

```

107 <log4j.version>1.2.17</log4j.version>
108 <log4j.test.version>2.17.0</log4j.test.version>
109 <slf4j.version>1.7.30</slf4j.version>
110 <joda.version>2.9.9</joda.version>
111 <!--<hadoop.version>2.10.1</hadoop.version>-->
112 <hadoop.version>3.1.3</hadoop.version>
113 <hive.groupid>org.apache.hive</hive.groupid>
114 <!--<hive.version>2.3.1</hive.version>-->
115 <hive.version>3.1.2</hive.version>
116 <hive.exec.classifier>core</hive.exec.classifier>
117 <metrics.version>4.1.1</metrics.version>
118 <orc.version>1.6.0</orc.version>

```

2.2.3 修改源码兼容 hadoop3

Hudi 默认依赖的 hadoop2，要兼容 hadoop3，除了修改版本，还需要修改如下代码：

```

vim
/opt/software/hudi-0.12.0/hudi-common/src/main/java/org/apache/hu
di/common/table/log/block/HoodieParquetDataBlock.java

```

修改第 110 行，原先只有一个参数，添加第二个参数 null：

```

106 Double.parseDouble(String.valueOf(0.1)); //HoodieStorageConfig.PARQUET_COMPRESSION
107 _RATIO.getDefaultValue());
108 ByteArrayOutputStream baos = new ByteArrayOutputStream();
109 try (FSDataOutputStream outputStream = new FSDataOutputStream(baos, null)) {
110     try (HoodieParquetStreamWriter<IndexedRecord> parquetWriter = new HoodieParquetStreamWri
111         ter<>(outputStream, avroParquetConfig)) {
112         for (IndexedRecord record : records) {
113             String recordKey = getRecordKey(record).orElse(null);
114             parquetWriter.writeAvro(recordKey, record);
115         }
116         outputStream.flush();
117     }
118 }
119 return baos.toByteArray();
120 }
121 @

```

否则会因为 hadoop2.x 和 3.x 版本兼容问题，报错如下：

```

[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.8.0:compile (default-compile) on project hudi-common: Compilation failure
[ERROR] /opt/software/hudi-0.12.0/hudi-common/src/main/java/org/apache/hudi/common/table/log/block/HoodieParquetDataBlock.java:[110,44] 对于FSDataOutputStream(java.io.ByteArrayOutputStream)，找不到合适的构造器
[ERROR] 构造器 org.apache.hadoop.fs.FSDataOutputStream.FSDataOutputStream(java.io.OutputStream,org.apache.hadoop.fs.FileSystem.Statistics)不适用
[ERROR] (实际参数列表和形式参数列表长度不同)
[ERROR] 构造器 org.apache.hadoop.fs.FSDataOutputStream.FSDataOutputStream(java.io.OutputStream,org.apache.hadoop.fs.FileSystem.Statistics,long)不适用
[ERROR] (实际参数列表和形式参数列表长度不同)

```

2.2.4 手动安装 Kafka 依赖

有几个 kafka 的依赖需要手动安装，否则编译报错如下：

```

[ERROR] Failed to execute goal on project hudi-utilities_2.12: Could not resolve dependencies for project org.apache.hudi:hudi-utilities_2.12:jar:0.12.0: The following artifacts could not be resolved: io.confluent:kafka-avro-serializer:jar:5.3.4, io.confluent:common-config:jar:5.3.4, io.confluent:common-utils:jar:5.3.4, io.confluent:kafka-schema-registry-client:jar:5.3.4: Could not find artifact io.confluent:kafka-avro-serializer:jar:5.3.4 in aliyunmaven (https://maven.aliyun.com/repository/public) -> [Help 1]

```

1) 下载 jar 包

通过网址下载：<http://packages.confluent.io/archive/5.3/confluent-5.3.4-2.12.zip>

解压后找到以下 jar 包，上传服务器 hadoop102

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

- common-config-5.3.4.jar
- common-utils-5.3.4.jar
- kafka-avro-serializer-5.3.4.jar
- kafka-schema-registry-client-5.3.4.jar

2) install 到 maven 本地仓库

```
mvn install:install-file -DgroupId=io.confluent
-DartifactId=common-config -Dversion=5.3.4 -Dpackaging=jar
-Dfile=./common-config-5.3.4.jar
mvn install:install-file -DgroupId=io.confluent
-DartifactId=common-utils -Dversion=5.3.4 -Dpackaging=jar
-Dfile=./common-utils-5.3.4.jar
mvn install:install-file -DgroupId=io.confluent
-DartifactId=kafka-avro-serializer -Dversion=5.3.4 -Dpackaging=jar
-Dfile=./kafka-avro-serializer-5.3.4.jar
mvn install:install-file -DgroupId=io.confluent
-DartifactId=kafka-schema-registry-client -Dversion=5.3.4
-Dpackaging=jar -Dfile=./kafka-schema-registry-client-5.3.4.jar
```

2.2.5 解决 spark 模块依赖冲突

修改了 Hive 版本为 3.1.2，其携带的 jetty 是 0.9.3，hudi 本身用的 0.9.4，存在依赖冲突。

- 1) 修改 hudi-spark-bundle 的 pom 文件，排除低版本 jetty，添加 hudi 指定版本的 jetty:

```
vim /opt/software/hudi-0.12.0/packaging/hudi-spark-bundle/pom.xml
```

在 382 行的位置，修改如下（红色部分）：

```
<!-- Hive -->
<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-service</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <artifactId>guava</artifactId>
      <groupId>com.google.guava</groupId>
    </exclusion>
    <exclusion>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.pentaho</groupId>
      <artifactId>*</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-service-rpc</artifactId>
```

```
<version>${hive.version}</version>
<scope>${spark.bundle.hive.scope}</scope>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-jdbc</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <groupId>javax.servlet</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>*</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-metastore</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <groupId>javax.servlet</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.datanucleus</groupId>
      <artifactId>datanucleus-core</artifactId>
    </exclusion>
    <exclusion>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <artifactId>guava</artifactId>
      <groupId>com.google.guava</groupId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-common</artifactId>
  <version>${hive.version}</version>
  <scope>${spark.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
```

```
<groupId>org.eclipse.jetty.orbit</groupId>
<artifactId>javax.servlet</artifactId>
</exclusion>
<exclusion>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>*</artifactId>
</exclusion>
</exclusions>
</dependency>

<!-- 增加 hudi 配置版本的 jetty -->
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-server</artifactId>
  <version>${jetty.version}</version>
</dependency>
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-util</artifactId>
  <version>${jetty.version}</version>
</dependency>
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-webapp</artifactId>
  <version>${jetty.version}</version>
</dependency>
<dependency>
  <groupId>org.eclipse.jetty</groupId>
  <artifactId>jetty-http</artifactId>
  <version>${jetty.version}</version>
</dependency>
```

否则在使用 spark 向 hudi 表插入数据时，会报错如下：

```
java.lang.NoSuchMethodError:
org.apache.hudi.org.apache.jetty.server.session.SessionHandler.setHttpOnly(Z)V
```

```
java.lang.NoSuchMethodError: org.apache.hudi.org.apache.jetty.server.session.SessionHandler.setHttpOnly(Z)V
at io.javalin.core.util.JettyServerUtil.defaultSessionHandler(JettyServerUtil.kt:50)
at io.javalin.Javalin.<init>(Javalin.java:94)
at io.javalin.Javalin.create(Javalin.java:107)
at org.apache.hudi.timeline.service.TimelineService.startService(TimelineService.java:279)
at org.apache.hudi.client.embedded.EmbeddedTimelineService.startServer(EmbeddedTimelineService.java:95)
at org.apache.hudi.client.embedded.EmbeddedTimelineServerHelper.startTimelineService(EmbeddedTimelineServerHelper.java:71)
at org.apache.hudi.client.embedded.EmbeddedTimelineServerHelper.createEmbeddedTimelineService(EmbeddedTimelineServerHelper.java:58)
at org.apache.hudi.client.BaseHoodieClient.startEmbeddedServerView(BaseHoodieClient.java:109)
at org.apache.hudi.client.BaseHoodieClient.<init>(BaseHoodieClient.java:77)
at org.apache.hudi.client.BaseHoodieWriteClient.<init>(BaseHoodieWriteClient.java:175)
at org.apache.hudi.client.sparkRDDWriteClient.<init>(SparkRDDWriteClient.java:95)
at org.apache.hudi.client.sparkRDDWriteClient.<init>(SparkRDDWriteClient.java:79)
at org.apache.hudi.DataSourceUtils.createHoodieClient(DataSourceUtils.java:200)
at org.apache.hudi.HoodieSparkSqlWriter$.anonfun$write$11(HoodieSparkSqlWriter.scala:286)
at scala.Option.getOrElse(Option.scala:189)
at org.apache.hudi.HoodieSparkSqlWriter$.write(HoodieSparkSqlWriter.scala:285)
at org.apache.hudi.DefaultSource.createRelation(DefaultSource.scala:163)
at org.apache.spark.sql.execution.datasources.SaveIntoDataSourceCommand.run(SaveIntoDataSourceCommand.scala:45)
at org.apache.spark.sql.execution.command.ExecutedCommandExec.sideEffectResult$lzycompute(commands.scala:75)
at org.apache.spark.sql.execution.command.ExecutedCommandExec.sideEffectResult(commands.scala:73)
at org.apache.spark.sql.execution.command.ExecutedCommandExec.executeCollect(commands.scala:84)
at org.apache.spark.sql.execution.QueryExecution$.anonfun$eagerlyExecuteCommands$1$.anonfun$applyOrElse$1(QueryExecution.scala:97)
at org.apache.spark.sql.execution.SQLExecution$.anonfun$withNewExecutionId$5(SQLExecution.scala:103)
```

2) 修改 hudi-utilities-bundle 的 pom 文件，排除低版本 jetty，添加 hudi 指定版本的 jetty:

```
vim
/opt/software/hudi-0.12.0/packaging/hudi-utilities-bundle/pom.xml
```

在 345 行的位置，修改如下（红色部分）：

```
<!-- Hoodie -->
<dependency>
  <groupId>org.apache.hudi</groupId>
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
<artifactId>hudi-common</artifactId>
<version>${project.version}</version>
<exclusions>
  <exclusion>
    <groupId>org.eclipse.jetty</groupId>
    <artifactId>*</artifactId>
  </exclusion>
</exclusions>
</dependency>
<dependency>
  <groupId>org.apache.hudi</groupId>
  <artifactId>hudi-client-common</artifactId>
  <version>${project.version}</version>
  <exclusions>
    <exclusion>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>*</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<!-- Hive -->
<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-service</artifactId>
  <version>${hive.version}</version>
  <scope>${utilities.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <artifactId>servlet-api</artifactId>
      <groupId>javax.servlet</groupId>
    </exclusion>
    <exclusion>
      <artifactId>guava</artifactId>
      <groupId>com.google.guava</groupId>
    </exclusion>
    <exclusion>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.pentaho</groupId>
      <artifactId>*</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-service-rpc</artifactId>
  <version>${hive.version}</version>
  <scope>${utilities.bundle.hive.scope}</scope>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
```

```
<artifactId>hive-jdbc</artifactId>
<version>${hive.version}</version>
<scope>${utilities.bundle.hive.scope}</scope>
<exclusions>
  <exclusion>
    <groupId>javax.servlet</groupId>
    <artifactId>*</artifactId>
  </exclusion>
  <exclusion>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>*</artifactId>
  </exclusion>
  <exclusion>
    <groupId>org.eclipse.jetty</groupId>
    <artifactId>*</artifactId>
  </exclusion>
</exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-metastore</artifactId>
  <version>${hive.version}</version>
  <scope>${utilities.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <groupId>javax.servlet</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.datanucleus</groupId>
      <artifactId>datanucleus-core</artifactId>
    </exclusion>
    <exclusion>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>*</artifactId>
    </exclusion>
    <exclusion>
      <artifactId>guava</artifactId>
      <groupId>com.google.guava</groupId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>${hive.groupid}</groupId>
  <artifactId>hive-common</artifactId>
  <version>${hive.version}</version>
  <scope>${utilities.bundle.hive.scope}</scope>
  <exclusions>
    <exclusion>
      <groupId>org.eclipse.jetty.orbit</groupId>
      <artifactId>javax.servlet</artifactId>
    </exclusion>
    <exclusion>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>*</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

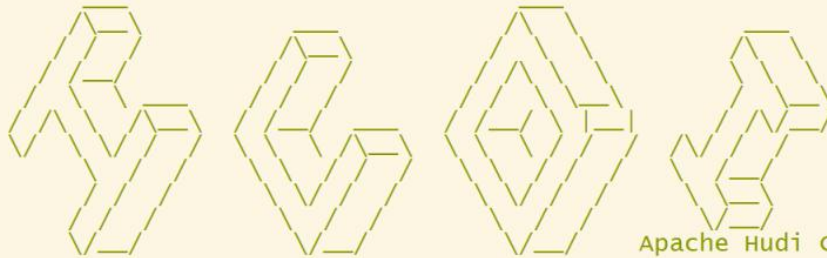
否则在使用 DeltaStreamer 工具向 hudi 表插入数据时，也会报 Jetty 的错误。

```
mvn clean package -DskipTests -Dspark3.2 -Dflink1.13 -Dscala-2.12
-Dhadoop.version=3.1.3 -Dflink-bundle-shade-hive3
```

编译成功后，进入 hudi-cli 说明成功：

```
Table command getting loaded
HoodiesplashScreen loaded
```

```
*
*
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
```



Apache Hudi CLI

```
welcome to Apache Hudi CLI. Please type help if you are looking for help.
hudi->
```

编译完成后，相关的包在 `packaging` 目录的各个模块中：

```
[atguigu@hadoop1 packaging]$ pwd
/opt/software/hudi-0.12.0/packaging
[atguigu@hadoop1 packaging]$ ls
hudi-aws-bundle          hudi-hive-sync-bundle    hudi-timeline-server-bundle
hudi-datahub-sync-bundle hudi-integ-test-bundle   hudi-trino-bundle
hudi-flink-bundle        hudi-kafka-connect-bundle hudi-utilities-bundle
hudi-gcp-bundle          hudi-presto-bundle       hudi-utilities-slim-bundle
hudi-hadoop-mr-bundle    hudi-spark-bundle        README.md
```

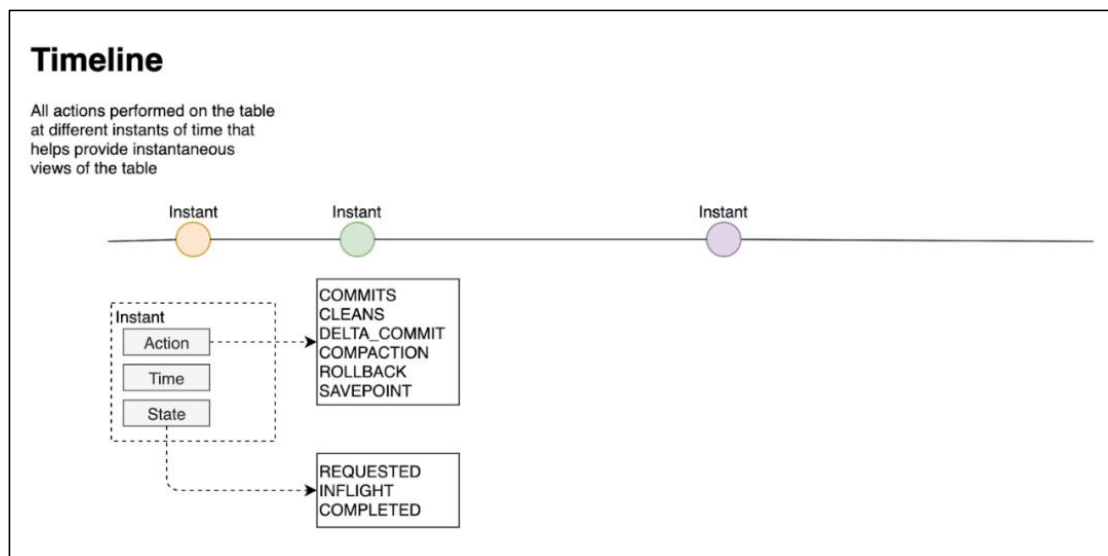
比如，flink 与 hudi 的包：

```
[atguigu@hadoop1 target]$ pwd
/opt/software/hudi-0.12.0/packaging/hudi-flink-bundle/target
[atguigu@hadoop1 target]$ ls
classes                                maven-shared-archive-resources
dependency-reduced-pom.xml            maven-status
generated-sources                    original-hudi-flink1.13-bundle-0.12.0.jar
hudi-flink1.13-bundle-0.12.0.jar      original-hudi-flink1.13-bundle-0.12.0-sources.jar
hudi-flink1.13-bundle-0.12.0-sources.jar
maven-archiver                       rat.txt
test-classes
```

第 3 章 核心概念

3.1 基本概念

3.1.1 时间轴（TimeLine）



Hudi 的核心是维护表上在不同的**即时时间（instants）**执行的所有操作的**时间轴（timeline）**，这有助于提供表的即时视图，同时还有效地支持按到达顺序检索数据。一个 instant 由以下三个部分组成：

1) Instant action: 在表上执行的操作类型

- **COMMITTS**: 一次 commit 表示将一批数据原子性地写入一个表。
- **CLEANS**: 清除表中不再需要的旧版本文件的后台活动。
- **DELTA_COMMIT**: 增量提交指的是将一批数据原子性地写入一个 MergeOnRead

更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可百度访问：[尚硅谷官网](#)

类型的表，其中部分或所有数据可以写入增量日志。

- **COMPACTION**: 合并 Hudi 内部差异数据结构的后台活动，例如:将更新操作从基于行的 log 日志文件合并到列式存储的数据文件。在内部，COMPACTION 体现为时间轴上的特殊提交。
- **ROLLBACK**: 表示当提交不成功时进行回滚，其会删除在写入过程中产生的部分文件。
- **SAVEPOINT**: 将某些文件组标记为已保存，以便其不会被删除。在发生灾难需要恢复数据的情况下，它有助于将数据集还原到时间轴上的某个点。

2) Instant time

通常是一个时间戳（例如：20190117010349），它按照动作开始时间的顺序单调增加。

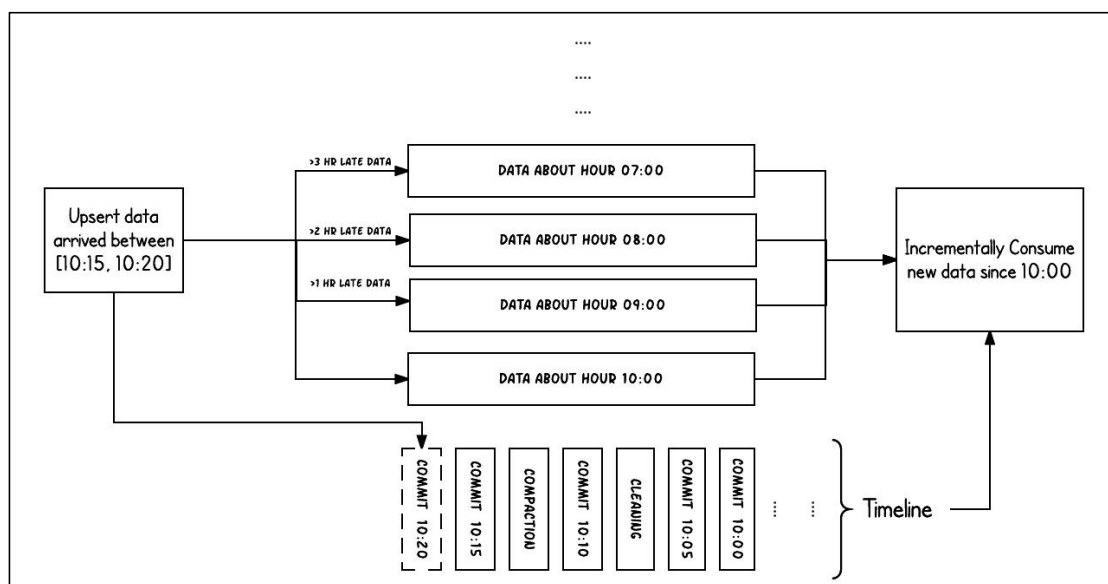
3) State

- **REQUESTED**: 表示某个动作已经调度，但尚未执行。
- **INFLIGHT**: 表示动作当前正在执行。
- **COMPLETED**: 表示时间轴上的动作已经完成。

4) 两个时间概念

区分两个重要的时间概念：

- **Arrival time**: 数据到达 Hudi 的时间。
- **Event time**: 数据中记录的时间。

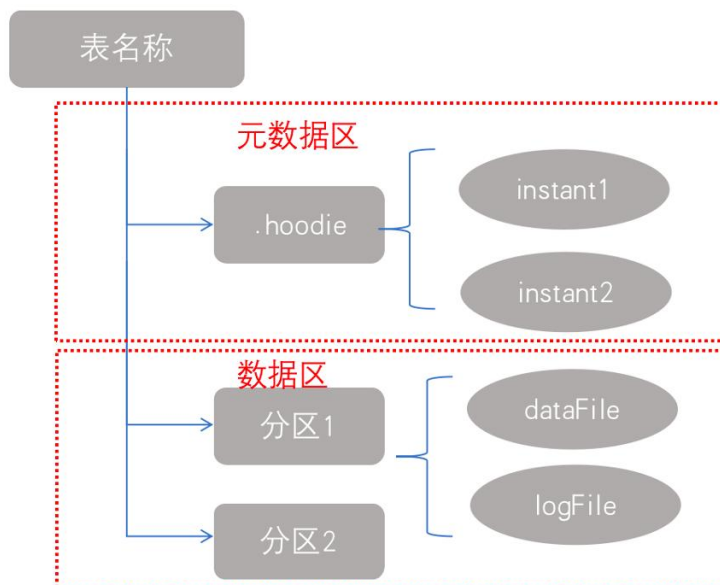


上图中采用时间（小时）作为分区字段，从 10:00 开始陆续产生各种提交，10:20 来

了一条 9:00 的数据，根据事件时间该数据仍然可以落到 9:00 对应的分区，通过时间轴直接消费 10:00（commit time）之后的增量更新（只消费有新 commits 的 group），那么这条延迟的数据仍然可以被消费到。

3.1.2 文件布局（File Layout）

Hudi 将一个表映射为如下文件结构



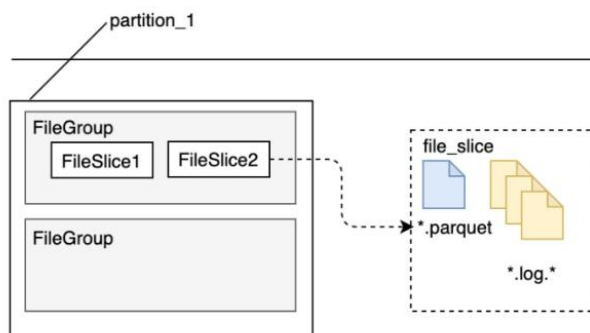
Hudi 存储分为两个部分：

(1) 元数据：.hoodie 目录对应着表的元数据信息，包括表的版本管理（Timeline）、归档目录（存放过时的版本），一个版本记录了一次提交（commit）的行为、时间戳和状态，Hudi 以时间轴的形式维护了在数据集上执行的所有操作的元数据；

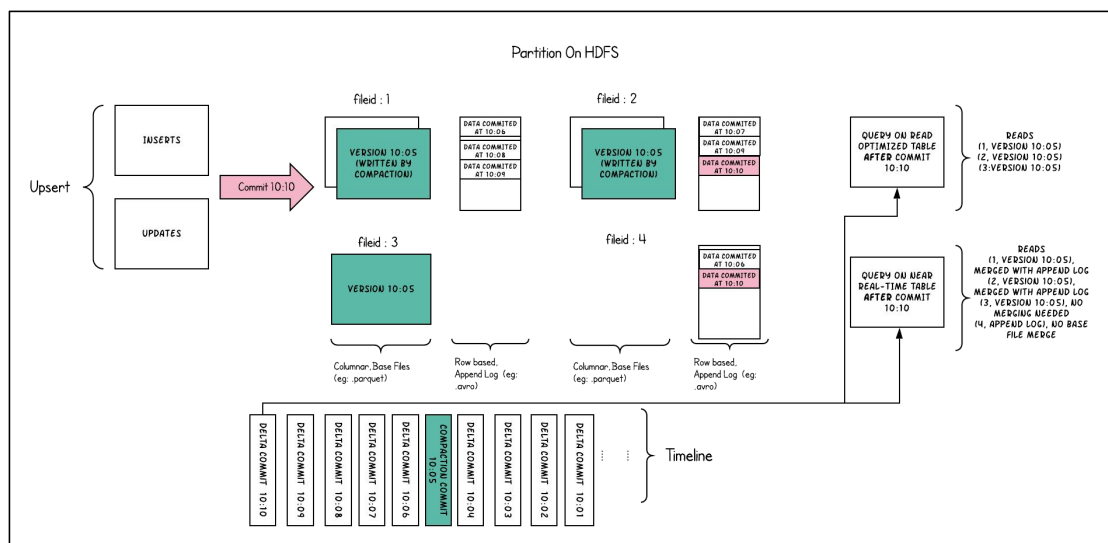
(2) 数据：和 hive 一样，以分区方式存放数据；分区里面存放着 Base File（.parquet）和 Log File（.log.*）；

FileManage

A base path have several partition DIRs, each DIR has several file groups with unique ID, each file group has several file slices.

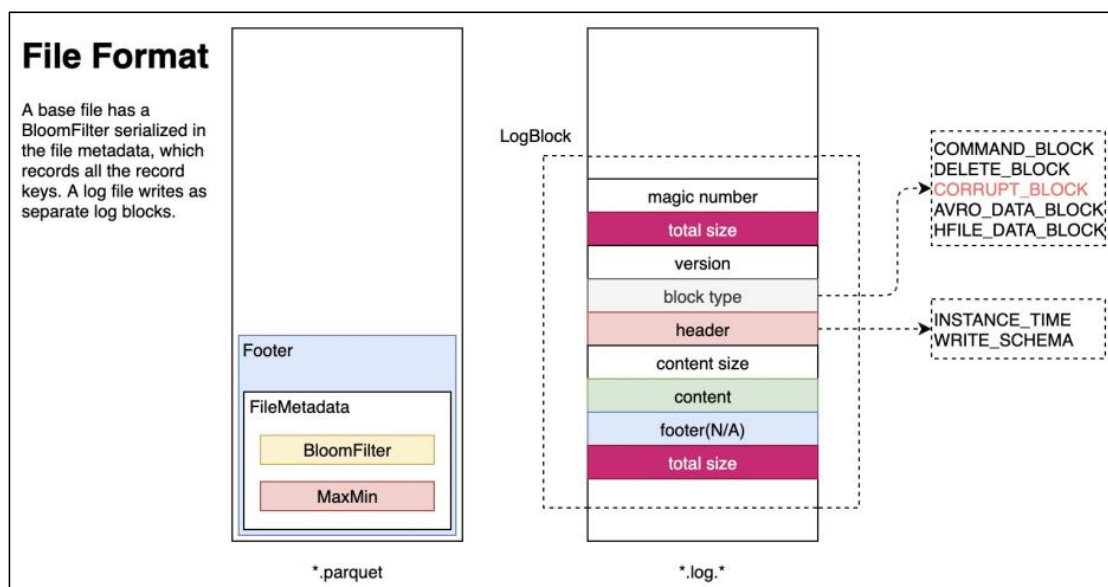


- (1) Hudi 将数据表组织成分布式文件系统基本路径（basepath）下的目录结构
- (2) 表被划分为多个分区，这些分区是包含该分区的数据文件的文件夹，非常类似于 Hive 表
- (3) 在每个分区中，文件被组织成文件组，由文件 ID 唯一标识
- (4) 每个文件组包含几个文件片（FileSlice）
- (5) 每个文件片包含：
 - 一个基本文件(.parquet)：在某个提交或者合并的处理时间（instant time）生成的（MOR 可能没有）
 - 多个日志文件(.log.*), 这些日志文件包含自生成基本文件以来对基本文件的插入/更新（COW 没有）
- (6) Hudi 采用了多版本并发控制(Multiversion Concurrency Control, MVCC)
 - compaction 操作：合并日志和基本文件以产生新的文件片
 - clean 操作：清除不使用的/旧的文件片以回收文件系统上的空间



(7) Hudi 的 base 文件（parquet 文件）在元数据中记录了数据的主键(key)组成的布隆过滤器，用于在读取数据中实现高效率的主键包含检测。只有不在布隆过滤器的 key 才需要扫描整个文件消灭假阳。

(8) Hudi 的 log（avro 文件）是自己编码的，通过积攒数据 buffer 以 LogBlock 为单位写出，每个 LogBlock 包含 magic number、size、content、footer 等信息，用于数据读、校验和过滤。

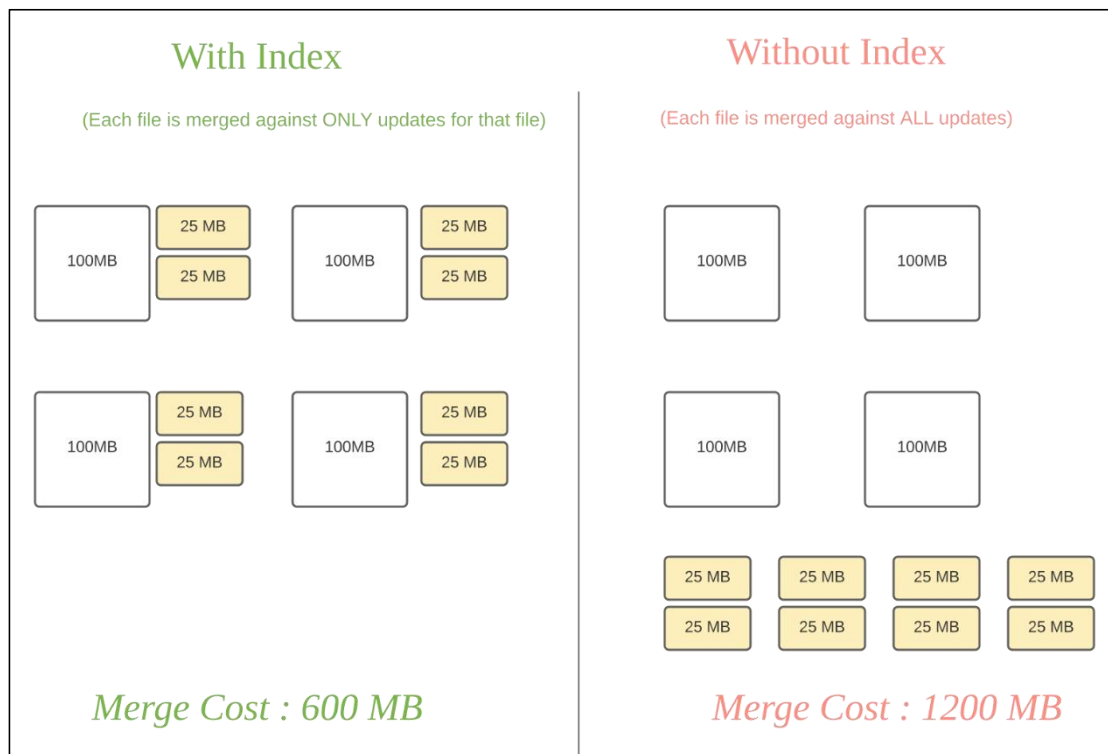


3.1.3 索引 (Index)

1) 原理

Hudi 通过索引机制提供高效的 upserts, 具体是将给定的 hoodie key(record key + partition

path)与文件 id (文件组) 建立唯一映射。这种映射关系, 数据第一次写入文件后保持不变, 所以, 一个文件组包含了一批数据的所有版本记录。Index 用于区分消息是 INSERT 还是 UPDATE。



Hudi 为了消除不必要的读写, 引入了索引的实现。在有了索引之后, 更新的数据可以快速被定位到对应的文件组。上图为, 白色是基本文件, 黄色是更新数据, 有了索引机制, 可以做到: 避免读取不需要的文件、避免更新不必要的文件、无需将更新数据与历史数据做分布式关联, 只需要在文件组内做合并。

2) 索引选项

Index 类型	原理	优点	缺点
Bloom Index	默认配置, 使用布隆过滤器来判断记录存在与否, 也可选使用 record key 的范围裁剪需要的文件	效率高, 不依赖外部系统, 数据和索引保持一致性	因假阳性问题, 还需回溯原文件再查找一遍
Simple Index	把 update/delete 操作的新数据和老数据进行 join	实现最简单, 无需额外的资源	性能比较差
HBase Index	把 index 存放在 HBase 里面。在插入 File Group 定位阶段所有 task 向 HBase 发送 Batch Get 请求, 获取 Record Key 的 Mapping 信息	对于小批次的 keys, 查询效率高	需要外部的系统, 增加了运维压力

Flink State-based Index	HUDI 在 0.8.0 版本中实现的 Flink writer, 采用了 Flink 的 state 作为底层的 index 存储, 每个 records 在写入之前都会先计算目标 bucket ID。	不同于 BloomFilter Index, 避免了每次重复的文件 index 查找	
-------------------------	--	--	--

注意: Flink 只有一种 state based index (和 bucket_index), 其他索引是 Spark 可选配置。

3) 全局索引与非全局索引

全局索引: 全局索引在全表的所有分区范围下强制要求键的唯一性, 也就是确保对给定的键有且只有一个对应的记录。全局索引提供了更强的保证, 但是随着表增大, update/delete 操作损失的性能越高, 因此更适用于小表。

非全局索引: 默认的索引实现, 只能保证数据在分区的唯一性。非全局索引依靠写入器为同一个记录的 update/delete 提供一致的分区路径, 同时大幅提高了效率, 更适用于大表。

从 index 的维护成本和写入性能的角度考虑, 维护一个 global index 的难度更大, 对写入性能的影响也更大, 所以需要 non-global index。

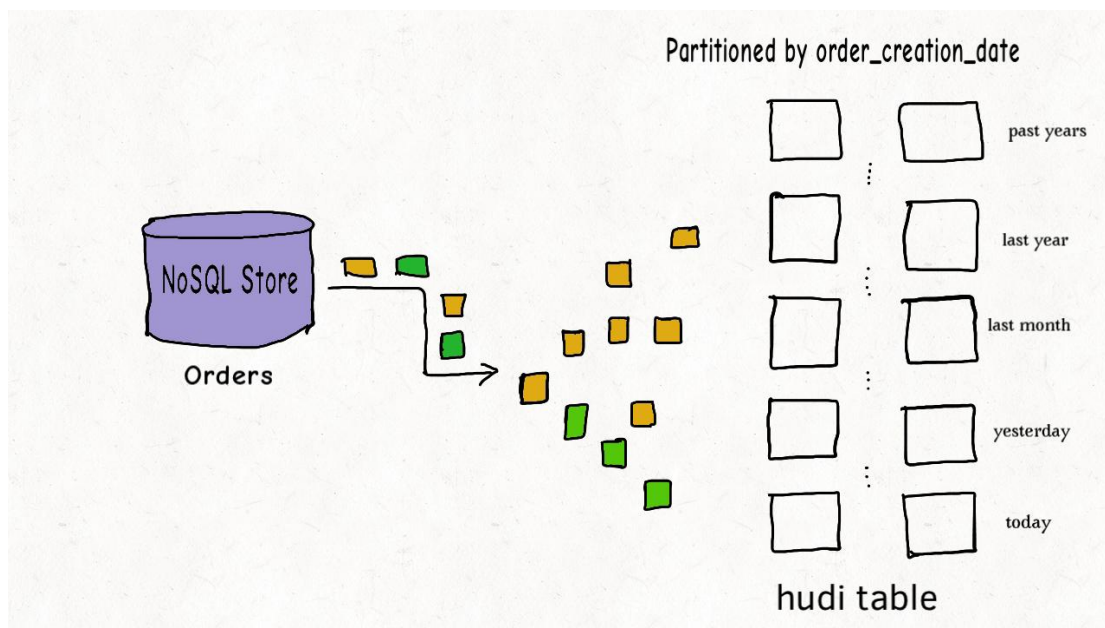
HBase 索引本质上是一个全局索引, bloom 和 simple index 都有全局选项:

- hoodie.index.type=GLOBAL_BLOOM
- hoodie.index.type=GLOBAL_SIMPLE

4) 索引的选择策略

(1) 对事实表的延迟更新

许多公司会在 NoSQL 数据存储中存放大量的交易数据。例如共享出行的行程表、股票买卖记录的表、和电商的订单表。这些表通常一直在增长, 且大部分的更新随机发生在较新的记录上, 而对旧记录有着长尾分布型的更新。这通常是源于交易关闭或者数据更正的延迟性。换句话说, 大部分更新会发生在最新的几个分区上而小部分会在旧的分区。

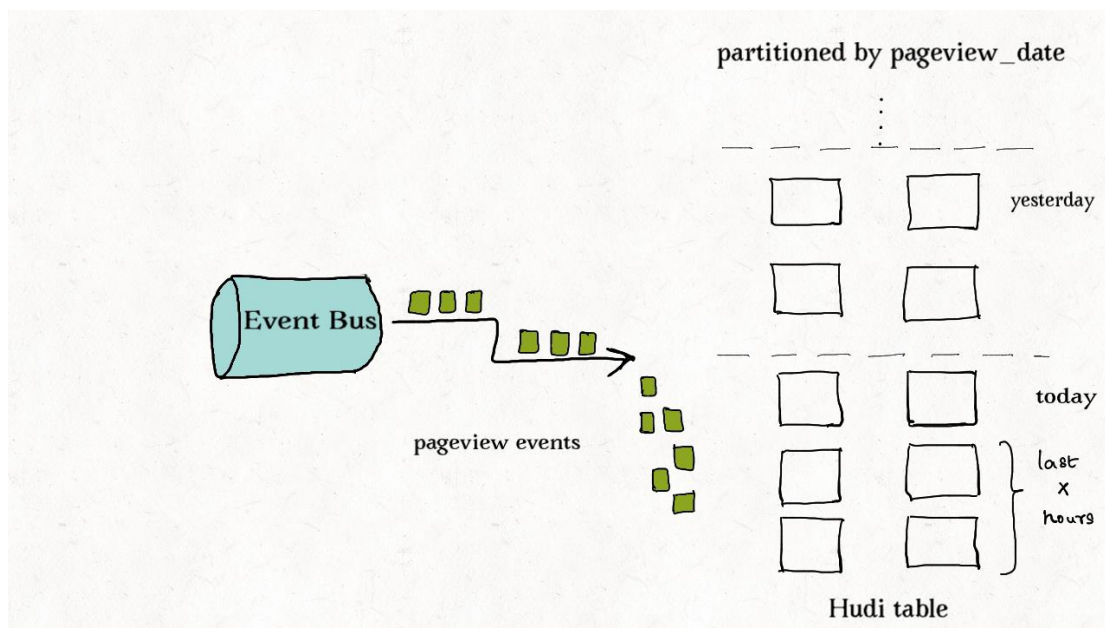


对于这样的作业模式，布隆索引就能表现地很好，因为查询索引可以靠设置得当的布隆过滤器来裁剪很多数据文件。另外，如果生成的键可以以某种顺序排列，参与比较的文件数会进一步通过范围裁剪而减少。Hudi 用所有文件的键域来构造区间树，这样能来高效地依据输入的更新记录的键域来排除不匹配的文件。

为了高效地把记录键和布隆过滤器进行比对，即尽量减少过滤器的读取和均衡执行器间的工作量，Hudi 缓存了输入记录并使用了自定义分区器和统计规律来解决数据的偏斜。有时，如果布隆过滤器的假阳性率过高，查询会增加数据的打乱操作。Hudi 支持动态布隆过滤器（设置 `hoodie.bloom.index.filter.type=DYNAMIC_V0`）。它可以根据文件里存放的记录数量来调整大小从而达到设定的假阳性率。

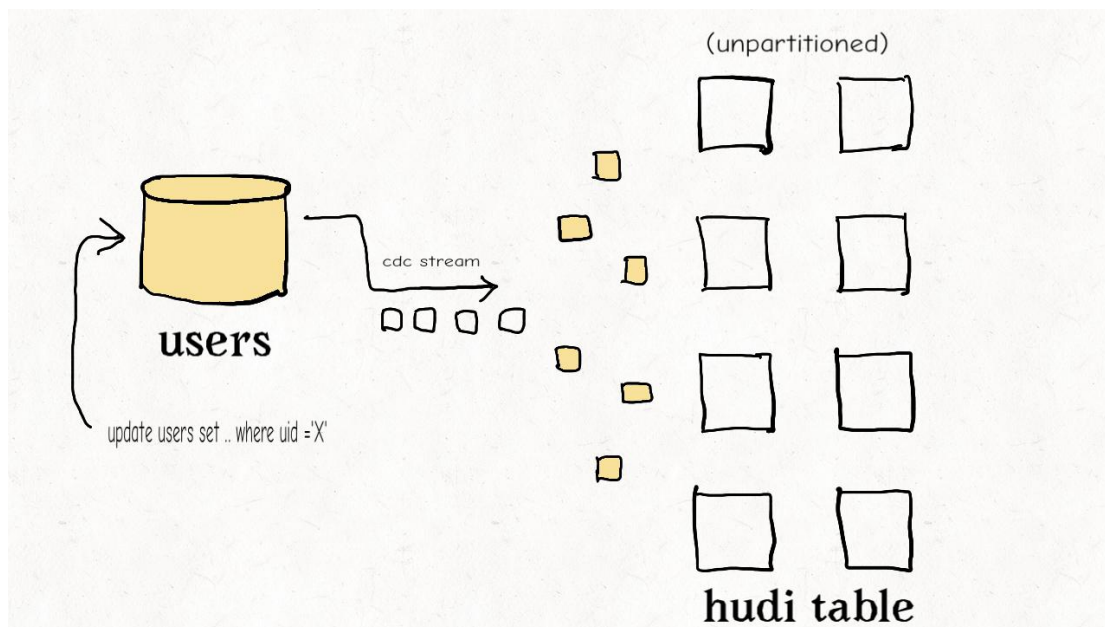
（2）对事件表的去重

事件流无处不在。从 Apache Kafka 或其他类似的消息总线发出的事件数通常是事实表大小的 10-100 倍。事件通常把时间（到达时间、处理时间）作为首要处理对象，比如物联网的事件流、点击流数据、广告曝光数等等。由于这些大部分都是仅追加的数据，插入和更新只存在于最新的几个分区中。由于重复事件可能发生在整个数据管道的任一节点，在存放数据湖前去重是一个常见的需求。



总的来说，低消耗去重是一个非常有挑战的工作。虽然可以用一个键值存储来实现去重（即 HBase 索引），但索引存储的消耗会随着事件数增长而线性增长以至于变得不可行。事实上，有范围裁剪功能的布隆索引是最佳的解决方案。我们可以利用作为首类处理对象的时间来构造由事件时间戳和事件 id (`event_ts+event_id`)组成的键，这样插入的记录就有了单调增长的键。这会在最新的几个分区里大幅提高裁剪文件的效益。

(3) 对维度表的随机更删



正如之前提到的，如果范围比较不能裁剪许多文件的话，那么布隆索引并不能带来很好的效益。在这样一个随机写入的作业场景下，更新操作通常会触及表里大多数文件从而导致布隆过滤器依据输入的更新对所有文件标明阳性。最终会导致，即使采用了范围比较，也还更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可百度访问：[尚硅谷官网](#)

是检查了所有文件。使用简单索引对此场景更合适，因为它不采用提前的裁剪操作，而是直接和所有文件的所需字段连接。如果额外的运维成本可以接受的话，也可以采用 HBase 索引，其对这些表能提供更加优越的查询效率。

当使用全局索引时，也可以考虑通过设置 `hoodie.bloom.index.update.partition.path=true` 或 `hoodie.simple.index.update.partition.path=true` 来处理的情况；例如对于以所在城市分区的用户表，会有用户迁至另一座城市的情况。这些表也非常适合采用 Merge-On-Read 表型。

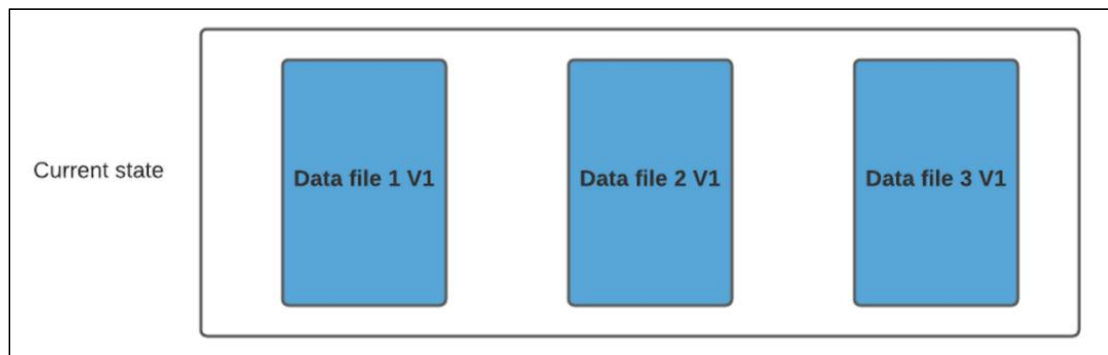
3.1.4 表类型（Table Types）

1) Copy On Write

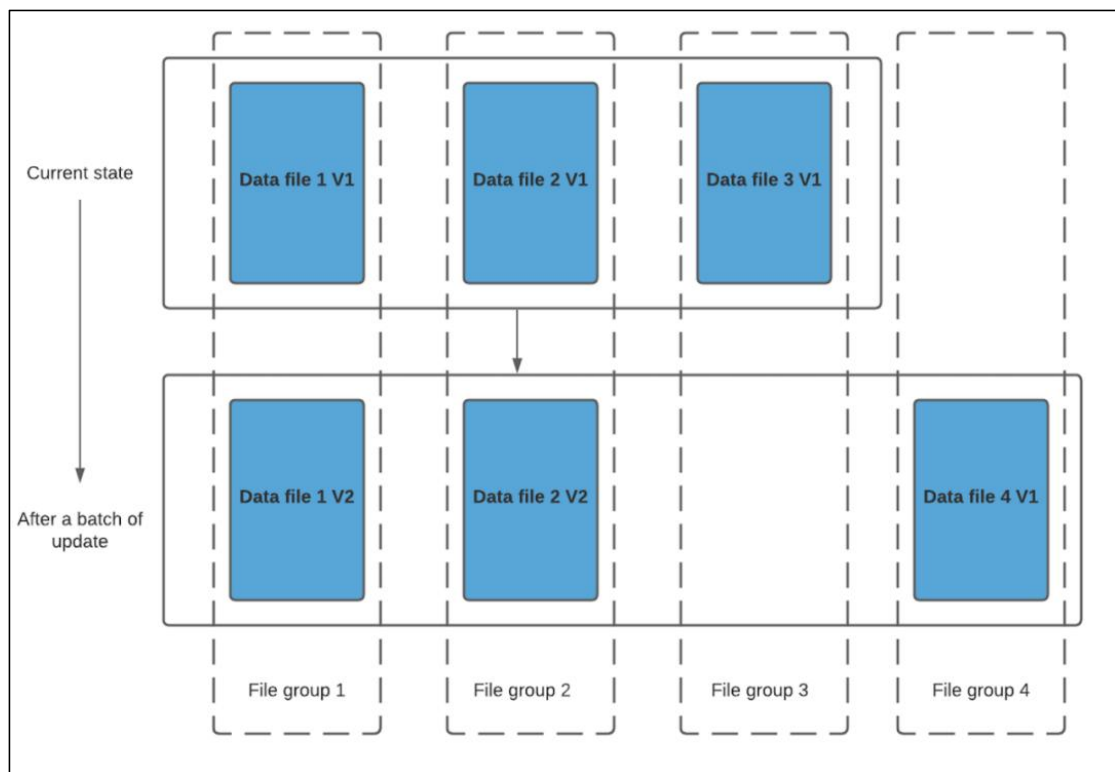
在 COW 表中，只有数据文件/基本文件（.parquet），没有增量日志文件（.log.*）。

对每一个新批次写入都将创建相应数据文件的新版本（新的 FileSlice），新版本文件包括旧版本文件的记录以及来自传入批次的记录（全量最新）。

假设我们有 3 个文件组，其中包含如下数据文件。



我们进行一批新的写入，在索引后，我们发现这些记录与 File group 1 和 File group 2 匹配，然后有新的插入，我们将为其创建一个新的文件组（File group 4）。



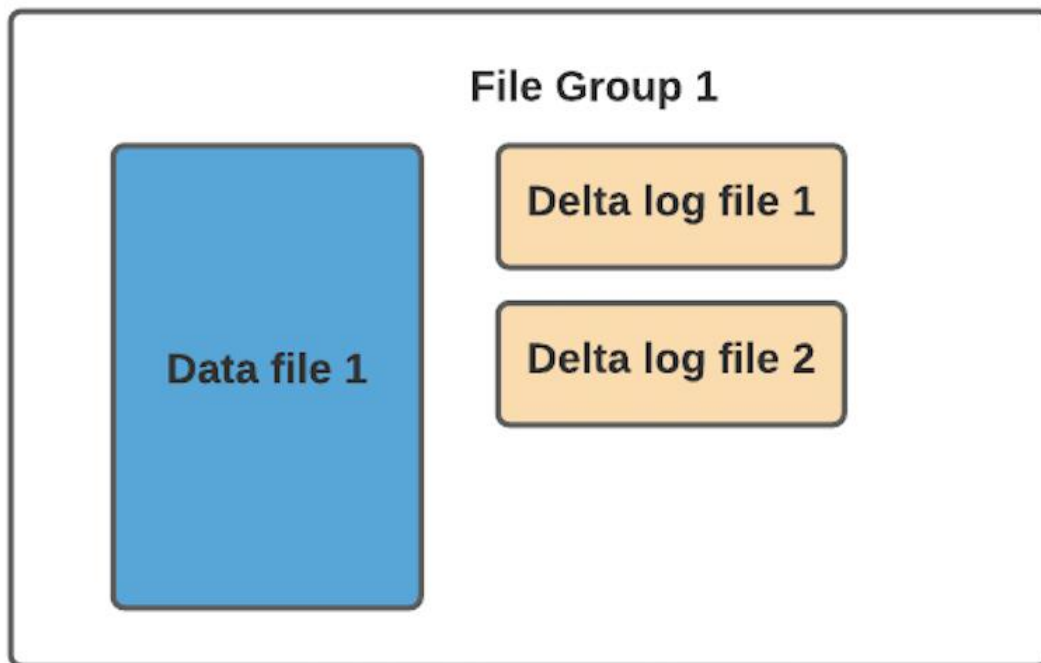
因此 data_file1 和 data_file2 都将创建更新的版本, data_file1 V2 是 data_file1 V1 的内容与 data_file1 中传入批次匹配记录的记录合并。

由于在写入期间进行合并, COW 会产生一些写入延迟。但是 COW 的优势在于它的简单性, 不需要其他表服务 (如压缩), 也相对容易调试。

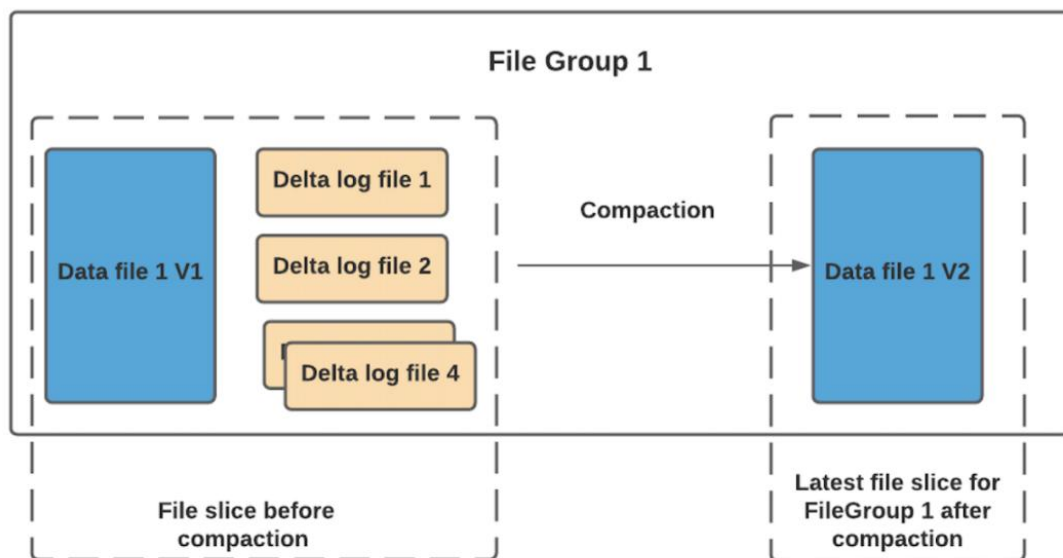
2) Merge On Read

MOR 表中, 包含列存的基本文件 (.parquet) 和行存的增量日志文件 (基于行的 avro 格式, .log.*)。

顾名思义, MOR 表的合并成本在读取端。因此在写入期间我们不会合并或创建较新的数据文件版本。标记/索引完成后, 对于具有要更新记录的现有数据文件, Hudi 创建增量日志文件并适当命名它们, 以便它们都属于一个文件组。



读取端将实时合并基本文件及其各自的增量日志文件。每次的读取延迟都比较高（因为查询时进行合并），所以 Hudi 使用压缩机制来将数据文件和日志文件合并在一起并创建更新版本的数据文件。



用户可以选择内联或异步模式运行压缩。Hudi 也提供了不同的压缩策略供用户选择，最常用的一种是基于提交的数量。例如可以将压缩的最大增量日志配置为 4。这意味着在进行 4 次增量写入后，将对数据文件进行压缩并创建更新版本的数据文件。压缩完成后，读取端只需要读取最新的数据文件，而不必关心旧版本文件。

MOR 表的写入行为，依据 index 的不同会有细微的差别：

- 对于 BloomFilter 这种无法对 log file 生成 index 的索引方案，对于 INSERT 消息仍然会写 base file（parquet format），只有 UPDATE 消息会 append log 文件（因为 base file 已经记录了该 UPDATE 消息的 FileGroup ID）。
- 对于可以对 log file 生成 index 的索引方案，例如 Flink writer 中基于 state 的索引，每次写入都是 log format，并且会不断追加和滚动。

3) COW 与 MOR 的对比

	CopyOnWrite	MergeOnRead
写入延迟	高	低
查询延迟	低	高
Update(I/O) 更新成本	高（重写整个 Parquet 文件）	低（追加到增量日志）
Parquet 文件大小	低（更新成本 I/O 高）	较大（低更新成本）
写放大	大	低（取决于压缩策略）

3.1.5 查询类型（Query Types）

Hudi 支持如下三种查询类型：

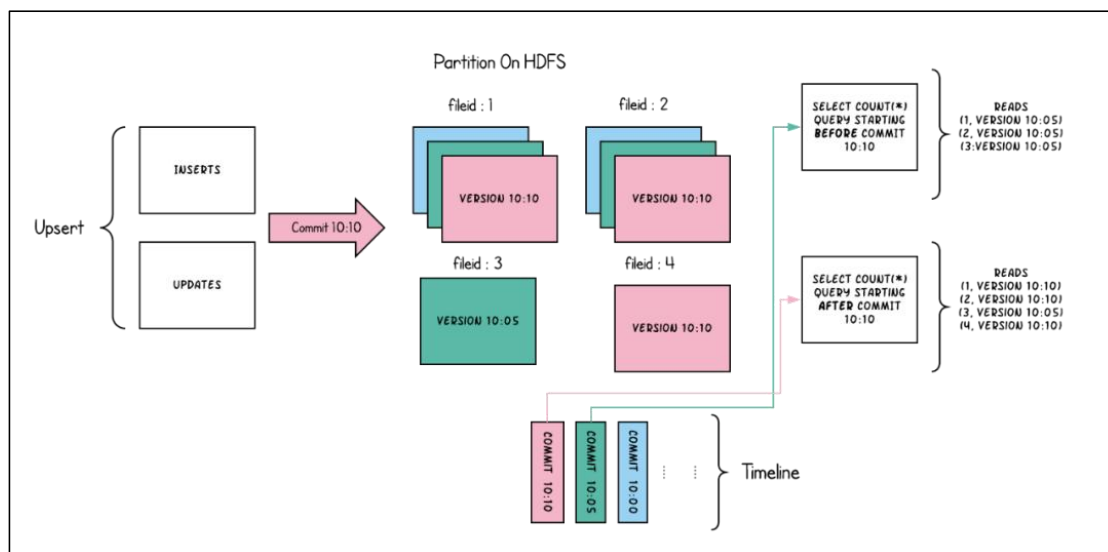
1) Snapshot Queries

快照查询，可以查询指定 commit/delta commit 即时操作后表的最新快照。

在读时合并（MOR）表的情况下，它通过即时合并最新文件片的基本文件和增量文件来提供近实时表（几分钟）。

对于写时复制（COW），它可以替代现有的 parquet 表（或相同基本文件类型的表），同时提供 upsert/delete 和其他写入方面的功能，可以理解为查询最新版本的 Parquet 数据文件。

下图是 COW 的快照查询：



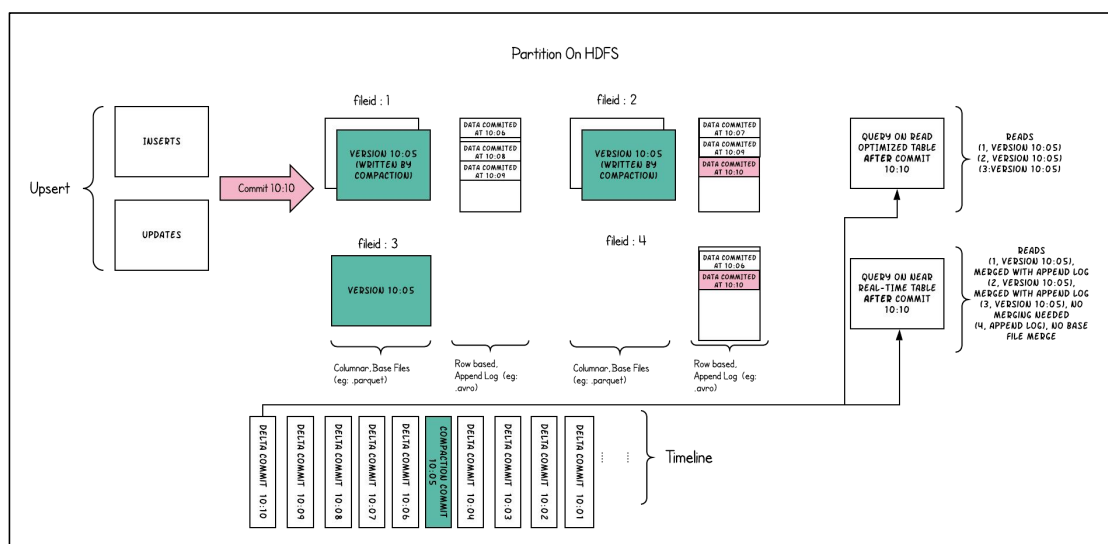
2) Incremental Queries

增量查询，可以查询给定 commit/delta commit 即时操作以来新写入的数据。有效的提供变更流来启用增量数据管道。

3) Read Optimized Queries

读优化查询，可查看给定的 commit/compact 即时操作的表的最新快照。仅将最新文件片的基本/列文件暴露给查询，并保证与非 Hudi 表相同的列查询性能。

下图是 MOR 表的快照查询与读优化查询的对比：



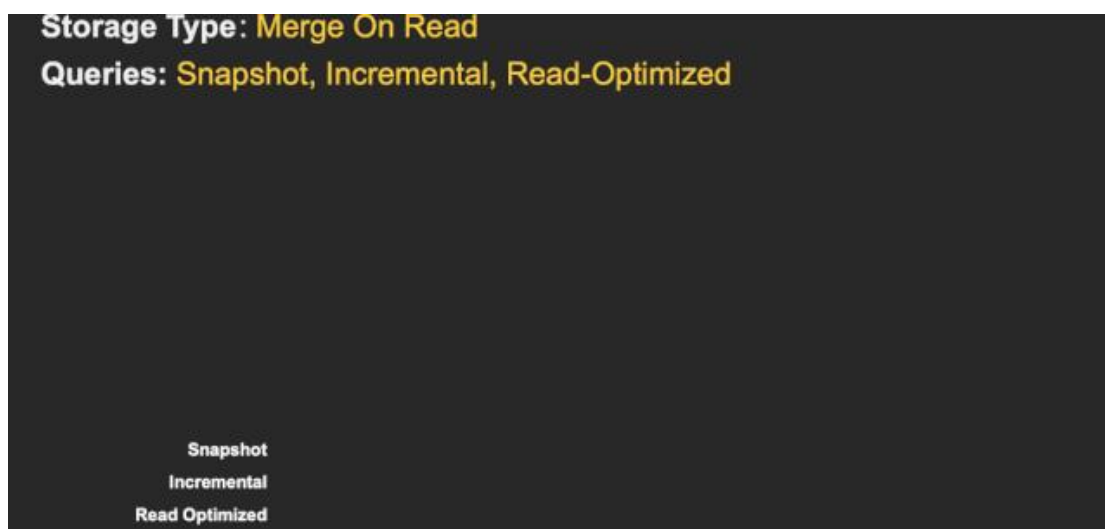
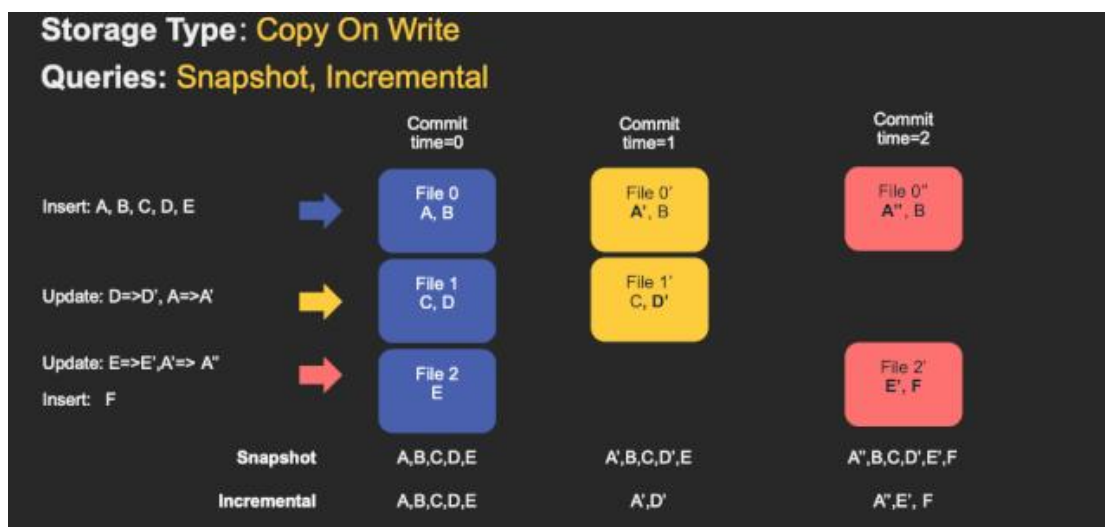
Read Optimized Queries 是对 Merge On Read 表类型快照查询的优化。

	Snapshot	Read Optimized
数据延迟	低	高

查询延迟	高（合并列式基础文件+行式增量日志文件）	低(原始列式基础文件)
------	----------------------	-------------

4) 不同表支持的查询类型

Table Type	Supported Query types
Copy On Write	Snapshot Queries + Incremental Queries
Merge On Read	Snapshot Queries + Incremental Queries + Read Optimized Queries



3.2 数据写

3.2.1 写操作

(1) UPSERT: 默认行为，数据先通过 index 打标(INSERT/UPDATE)，有一些启发式算法决定消息的组织以优化文件的大小 => CDC 导入

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

(2) INSERT: 跳过 index, 写入效率更高 => Log Deduplication

(3) BULK_INSERT: 写排序, 对大数据量的 Hudi 表初始化友好, 对文件大小的限制 best effort (写 HFile)

3.2.2 写流程 (UPSERT)

1) Copy On Write

(1) 先对 records 按照 record key 去重

(2) 首先对这批数据创建索引 (HoodieKey => HoodieRecordLocation); 通过索引区分哪些 records 是 update, 哪些 records 是 insert (key 第一次写入)

(3) 对于 update 消息, 会直接找到对应 key 所在的最新 FileSlice 的 base 文件, 并做 merge 后写新的 base file (新的 FileSlice)

(4) 对于 insert 消息, 会扫描当前 partition 的所有 SmallFile (小于一定大小的 base file), 然后 merge 写新的 FileSlice; 如果没有 SmallFile, 直接写新的 FileGroup + FileSlice

2) Merge On Read

(1) 先对 records 按照 record key 去重 (可选)

(2) 首先对这批数据创建索引 (HoodieKey => HoodieRecordLocation); 通过索引区分哪些 records 是 update, 哪些 records 是 insert (key 第一次写入)

(3) 如果是 insert 消息, 如果 log file 不可建索引 (默认), 会尝试 merge 分区内最小的 base file (不包含 log file 的 FileSlice), 生成新的 FileSlice; 如果没有 base file 就新写一个 FileGroup + FileSlice + base file; 如果 log file 可建索引, 尝试 append 小的 log file, 如果没有就新写一个 FileGroup + FileSlice + base file

(4) 如果是 update 消息, 写对应的 file group + file slice, 直接 append 最新的 log file (如果碰巧是当前最小的小文件, 会 merge base file, 生成新的 file slice)

(5) log file 大小达到阈值会 roll over 一个新的

3.2.3 写流程 (INSERT)

1) Copy On Write

(1) 先对 records 按照 record key 去重 (可选)

(2) 不会创建 Index

(3) 如果有小的 base file 文件, merge base file, 生成新的 FileSlice + base file, 否则

直接写新的 FileSlice + base file

2) Merge On Read

- (1) 先对 records 按照 record key 去重（可选）
- (2) 不会创建 Index
- (3) 如果 log file 可索引，并且有小的 FileSlice，尝试追加或写最新的 log file；如果 log file 不可索引，写一个新的 FileSlice + base file

3.2.4 写流程（INSERT OVERWRITE）

在同一分区中创建新的文件组集。现有的文件组被标记为 "删除"。根据新记录的数量创建新的文件组

1) COW

在插入分区之前	插入相同数量的记录覆盖	插入覆盖更多的记录	插入重写 1 条记录
分区包含 file1-t0.parquet, file2-t0.parquet。	分区将添加 file3-t1.parquet, file4-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。	分区将添加 file3-t1.parquet, file4-t1.parquet, file5-t1.parquet, ..., fileN-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效	分区将添加 file3-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。

2) MOR

在插入分区之前	插入相同数量的记录覆盖	插入覆盖更多的记录	插入重写 1 条记录
分区包含 file1-t0.parquet, file2-t0.parquet。 .file1-t00.log	file3-t1.parquet, file4-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。	file3-t1.parquet, file4-t1.parquet ... fileN-t1.parquet file1, file2 在 t1 后的元数据中被标记为无效	分区将添加 file3-t1.parquet。 file1, file2 在 t1 后的元数据中被标记为无效。

3) 优点

- (1) COW 和 MOR 在执行方面非常相似。不干扰 MOR 的 compaction。
- (2) 减少 parquet 文件大小。
- (3) 不需要更新关键路径中的外部索引。索引实现可以检查文件组是否无效（类似于在 HBaseIndex 中检查 commit 是否无效的方式）。

(4) 可以扩展清理策略，在一定的时间窗口后删除旧文件组。

4) 缺点

(1) 需要转发以前提交的元数据。

- 在 t1，比如 file1 被标记为无效，我们在 t1.commit 中存储 "invalidFiles=file1"(或者在 MOR 中存储 deltacommit)
- 在 t2，比如 file2 也被标记为无效。我们转发之前的文件，并在 t2.commit 中标记 "invalidFiles=file1, file2" (或 MOR 的 deltacommit)

(2) 忽略磁盘中存在的 parquet 文件也是 Hudi 的一个新行为，可能容易出错,我们必须认识到新的行为，并更新文件系统的所有视图来忽略它们。这一点可能会在实现其他功能时造成问题。

3.2.5 Key 生成策略

用来生成 HoodieKey (record key + partition path)，目前支持以下策略：

- 支持多个字段组合 record keys
- 支持多个字段组合的 partition path (可定制时间格式，Hive style path name)
- 非分区表

3.2.6 删除策略

1) 逻辑删：将 value 字段全部标记为 null

2) 物理删：

(1) 通过 OPERATION_OPT_KEY 删除所有的输入记录

(2) 配置 PAYLOAD_CLASS_OPT_KEY = org.apache.hudi.EmptyHoodieRecordPayload

删除所有的输入记录

(3) 在输入记录添加字段：_hoodie_is_deleted

3.2.7 总结

通过对写流程的梳理可以了解到 Apache Hudi 相对于其他数据湖方案的核心优势：

(1) 写入过程充分优化了文件存储的小文件问题，Copy On Write 写会一直将一个 bucket (FileGroup) 的 base 文件写到设定的阈值大小才会划分新的 bucket; Merge On Read 写在同一个 bucket 中，log file 也是一直 append 直到大小超过设定的阈值 roll over。

(2) 对 UPDATE 和 DELETE 的支持非常高效，一条 record 的整个生命周期操作都更多 [Java - 大数据 - 前端 - python 人工智能资料下载](#)，可百度访问：[尚硅谷官网](#)

发生在同一个 bucket，不仅减少小文件数量，也提升了数据读取的效率（不必要的 join 和 merge）。

第 4 章 集成 Spark

4.1 环境准备

4.1.1 安装 Spark

1) Hudi 支持的 Spark 版本

Hudi	Supported Spark 3 version
0.12.x	3.3.x, 3.2.x, 3.1.x
0.11.x	3.2.x (default build, Spark bundle only) , 3.1.x
0.10.x	3.1.x(default build), 3.0.x
0.7.0-0.9.0	3.0.x
0.6.0 and prior	Not supported

2) 下载 Spark 安装包，解压

```
cd /opt/software/
wget
https://dlcdn.apache.org/spark/spark-3.2.2/spark-3.2.2-bin-hadoop3.2.tgz
tar -zxvf spark-3.2.2-bin-hadoop3.2.tgz -C /opt/module/
mv /opt/module/spark-3.2.2-bin-hadoop3.2 /opt/module/spark-3.2.2
```

3) 配置环境变量

```
sudo vim /etc/profile.d/my_env.sh

export SPARK_HOME=/opt/module/spark-3.2.2
export PATH=$PATH:$SPARK_HOME/bin

source /etc/profile.d/my_env.sh
```

4) 拷贝编译好的包到 spark 的 jars 目录

```
cp
/opt/software/hudi-0.12.0/packaging/hudi-spark-bundle/target/hudi-spark3.2-bundle_2.12-0.12.0.jar /opt/module/spark-3.2.2/jars
```

4.1.2 启动 Hadoop（略）

4.2 spark-shell 方式

4.2.1 启动 spark-shell

1) 启动命令

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
#针对 Spark 3.2
spark-shell \
  --conf
'spark.serializer=org.apache.spark.serializer.KryoSerializer' \
  --conf
'spark.sql.catalog.spark_catalog=org.apache.spark.sql.hudi.catalog.HoodieCatalog' \
  --conf
'spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension'
```

2) 设置表名, 基本路径和数据生成器

```
import org.apache.hudi.QuickstartUtils._
import scala.collection.JavaConversions._
import org.apache.spark.sql.SaveMode._
import org.apache.hudi.DataSourceReadOptions._
import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.config.HoodieWriteConfig._

val tableName = "hudi_trips_cow"
val basePath = "file:///tmp/hudi_trips_cow"
val dataGen = new DataGenerator
```

不需要单独的建表。如果表不存在, 第一批写表将创建该表。

4.2.2 插入数据

新增数据, 生成一些数据, 将其加载到 `DataFrame` 中, 然后将 `DataFrame` 写入 Hudi 表。

```
val inserts = convertToStringList(dataGen.generateInserts(10))
val df = spark.read.json(spark.sparkContext.parallelize(inserts, 2))
df.write.format("hudi").
  options(getQuickstartWriteConfigs).
  option(PRECOMBINE_FIELD_OPT_KEY, "ts").
  option(RECORDKEY_FIELD_OPT_KEY, "uuid").
  option(PARTITIONPATH_FIELD_OPT_KEY, "partitionpath").
  option(TABLE_NAME, tableName).
  mode(Overwrite).
  save(basePath)
```

Mode (overwrite) 将覆盖重新创建表 (如果已存在)。可以检查 `/tmp/hudi_trips_cow` 路径下是否有数据生成。

```
cd /tmp/hudi_trips_cow/
ls
```

```
[atguigu@hadoop1 hudi_trips_cow]$ pwd
/tmp/hudi_trips_cow
[atguigu@hadoop1 hudi_trips_cow]$ ls
americas  asia
```

```
[atguigu@hadoop1 sao_paulo]$ pwd
/tmp/hudi_trips_cow/americas/brazil/sao_paulo
[atguigu@hadoop1 sao_paulo]$ ls
a636e7f8-ebaf-44c1-b560-b8711e7313d8-0_0-28-34_20220930032438342.parquet
```

数据文件的命名规则, 源码如下:

```
public static String makeBaseFileName(String instantTime, String writeToken, String fileId, String fileExtension) {
    return String.format("%s_%s_%s", fileId, writeToken, instantTime, fileExtension);
}
```

4.2.3 查询数据

1) 转换成 DF

```
val tripsSnapshotDF = spark.
    read.
    format("hudi").
    load(basePath)
tripsSnapshotDF.createOrReplaceTempView("hudi_trips_snapshot")
```

注意：该表有三级分区（区域/国家/城市），在 0.9.0 版本以前的 hudi，在 load 中的路径需要按照分区目录拼接"*"，如：load(basePath + "/*/*/*"), 当前版本不需要。

2) 查询

```
spark.sql("select fare, begin_lon, begin_lat, ts from hudi_trips_snapshot where fare > 20.0").show()
```

```
scala> spark.sql("select fare, begin_lon, begin_lat, ts from hudi_trips_snapshot where fare > 20.0").show()
ANTLR Tool version 4.7 used for code generation does not match the current runtime version 4.8ANTLR Tool version 4.7 used for code generation does not match the current runtime version 4.8
```

4.8+	fare	begin_lon	begin_lat	ts
27.79478688582596	0.6273212202489661	0.11488393157088261	1658898381340	
33.92216483948643	0.9694586417848392	0.1856488085068272	1659266366561	
64.27696295884016	0.4923479652912024	0.5731835407930634	1658833019905	
93.56018115236618	0.14285051259466197	0.21624150367601136	1658939243001	
34.158284716382845	0.46157858450465483	0.4726905879569653	1659195333214	
43.4923811219014	0.8779402295427752	0.6100070562136587	1659045643675	
66.62084366450246	0.03844104444445928	0.0750588760043035	1659165071870	
41.06290929046368	0.8192868687714224	0.651058505660742	1659095381308	

```
spark.sql("select _hoodie_commit_time, _hoodie_record_key, _hoodie_partition_path, rider, driver, fare from hudi_trips_snapshot").show()
```

```
scala> spark.sql("select _hoodie_commit_time, _hoodie_record_key, _hoodie_partition_path, rider, driver, fare from hudi_trips_snapshot").show()
```

_hoodie_commit_time	_hoodie_record_key	_hoodie_partition_path	rider	driver	fare
20220802061035097	81b1bb5c-ae0a-4d1...	americas/united_s...	rider-213	driver-213	27.79478688582596
20220802061035097	ff21c777-551b-4cb...	americas/united_s...	rider-213	driver-213	33.92216483948643
20220802061035097	69d62d2b-81d7-431...	americas/united_s...	rider-213	driver-213	64.27696295884016
20220802061035097	ed66329e-c5bc-49a...	americas/united_s...	rider-213	driver-213	19.179139106643607
20220802061035097	48dd8d6a-dbd7-47d...	americas/united_s...	rider-213	driver-213	93.56018115236618
20220802061035097	8dc69b69-7c8c-42d...	americas/brazil/s...	rider-213	driver-213	34.158284716382845
20220802061035097	c07f6c98-f011-4ba...	americas/brazil/s...	rider-213	driver-213	43.4923811219014
20220802061035097	ddf489f3-6c7d-4c6...	americas/brazil/s...	rider-213	driver-213	66.62084366450246
20220802061035097	cc747af1-fed0-47a...	asia/india/chennai	rider-213	driver-213	17.851135255091155
20220802061035097	7e033609-e6a0-403...	asia/india/chennai	rider-213	driver-213	41.06290929046368

3) 时间旅行查询

Hudi 从 0.9.0 开始就支持时间旅行查询。目前支持三种查询时间格式，如下所示。

```
spark.read.
    format("hudi").
    option("as.of.instant", "20210728141108100").
    load(basePath)

spark.read.
    format("hudi").
    option("as.of.instant", "2021-07-28 14:11:08.200").
    load(basePath)

// 表示 "as.of.instant = 2021-07-28 00:00:00"
spark.read.
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
format("hudi").  
option("as.of.instant", "2021-07-28").  
load(basePath)
```

4.2.4 更新数据

类似于插入新数据，使用数据生成器生成新数据对历史数据进行更新。将数据加载到 `DataFrame` 中并将 `DataFrame` 写入 Hudi 表中。

```
val updates = convertToStringList(dataGen.generateUpdates(10))  
val df = spark.read.json(spark.sparkContext.parallelize(updates, 2))  
df.write.format("hudi").  
  options(getQuickstartWriteConfigs).  
  option(PRECOMBINE_FIELD_OPT_KEY, "ts").  
  option(RECORDKEY_FIELD_OPT_KEY, "uuid").  
  option(PARTITIONPATH_FIELD_OPT_KEY, "partitionpath").  
  option(TABLE_NAME, tableName).  
  mode(Append).  
  save(basePath)
```

注意：保存模式现在是 `Append`。通常，除非是第一次创建表，否则请始终使用追加模式。现在再次查询数据将显示更新的行程数据。每个写操作都会生成一个用时间戳表示的新提交。查找以前提交中相同的 `_hoodie_record_keys` 在该表的 `_hoodie_commit_time`、`rider`、`driver` 字段中的变化。

查询更新后的数据，要重新加载该 hudi 表：

```
val tripsSnapshotDF = spark.  
  read.  
  format("hudi").  
  load(basePath)  
tripsSnapshotDF1.createOrReplaceTempView("hudi_trips_snapshot")  
  
spark.sql("select      _hoodie_commit_time,      _hoodie_record_key,  
  _hoodie_partition_path,      rider,      driver,      fare      from  
hudi_trips_snapshot").show()
```

4.2.5 增量查询

Hudi 还提供了增量查询的方式，可以获取从给定提交时间戳以来更改的数据流。需要指定增量查询的 `beginTime`，选择性指定 `endTime`。如果我們希望在给定提交之后进行所有更改，则不需要指定 `endTime`（这是常见的情况）。

1) 重新加载数据

```
spark.  
  read.  
  format("hudi").  
  load(basePath).  
  createOrReplaceTempView("hudi_trips_snapshot")
```

2) 获取指定 `beginTime`

```
val commits = spark.sql("select distinct(_hoodie_commit_time) as
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
commitTime from hudi_trips_snapshot order by commitTime").map(k =>
k.getString(0)).take(50)
val beginTime = commits(commits.length - 2)
```

3) 创建增量查询的表

```
val tripsIncrementalDF = spark.read.format("hudi").
  option(QUERY_TYPE_OPT_KEY, QUERY_TYPE_INCREMENTAL_OPT_VAL).
  option(BEGIN_INSTANTTIME_OPT_KEY, beginTime).
  load(basePath)
tripsIncrementalDF.createOrReplaceTempView("hudi_trips_incremental")
```

4) 查询增量表

```
spark.sql("select `hoodie_commit_time`, fare, begin_lon, begin_lat,
ts from hudi_trips_incremental where fare > 20.0").show()
```

这将过滤出 beginTime 之后提交且 fare>20 的数据。

利用增量查询，我们能在批处理数据上创建 streaming pipelines。

4.2.6 指定时间点查询

查询特定时间点的数据，可以将 endTime 指向特定时间，beginTime 指向 000（表示最早提交时间）

1) 指定 beginTime 和 endTime

```
val beginTime = "000"
val endTime = commits(commits.length - 2)
```

2) 根据指定时间创建表

```
val tripsPointInTimeDF = spark.read.format("hudi").
  option(QUERY_TYPE_OPT_KEY, QUERY_TYPE_INCREMENTAL_OPT_VAL).
  option(BEGIN_INSTANTTIME_OPT_KEY, beginTime).
  option(END_INSTANTTIME_OPT_KEY, endTime).
  load(basePath)
tripsPointInTimeDF.createOrReplaceTempView("hudi_trips_point_in_time")
```

3) 查询

```
spark.sql("select `hoodie_commit_time`, fare, begin_lon, begin_lat,
ts from hudi_trips_point_in_time where fare > 20.0").show()
```

4.2.7 删除数据

根据传入的 HoodieKeys 来删除（uuid + partitionpath），只有 append 模式，才支持删除功能。

1) 获取总行数

```
spark.sql("select uuid, partitionpath from
hudi_trips_snapshot").count()
```

2) 取其中 2 条用来删除

```
val ds = spark.sql("select uuid, partitionpath from
hudi_trips_snapshot").limit(2)
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

3) 将待删除的 2 条数据构建 DF

```
val deletes = dataGen.generateDeletes(ds.collectAsList())  
val df = spark.read.json(spark.sparkContext.parallelize(deletes, 2))
```

4) 执行删除

```
df.write.format("hudi").  
  options(getQuickstartWriteConfigs).  
  option(OPERATION_OPT_KEY, "delete").  
  option(PRECOMBINE_FIELD_OPT_KEY, "ts").  
  option(RECORDKEY_FIELD_OPT_KEY, "uuid").  
  option(PARTITIONPATH_FIELD_OPT_KEY, "partitionpath").  
  option(TABLE_NAME, tableName).  
  mode(Append).  
  save(basePath)
```

5) 统计删除数据后的行数，验证删除是否成功

```
val roAfterDeleteViewDF = spark.  
  read.  
  format("hudi").  
  load(basePath)  
  
roAfterDeleteViewDF.registerTempTable("hudi_trips_snapshot")  
  
// 返回的总行数应该比原来少 2 行  
spark.sql("select          uuid,          partitionpath          from  
hudi_trips_snapshot").count()
```

4.2.8 覆盖数据

对于表或分区来说，如果大部分记录在每个周期都发生变化，那么做 **upsert** 或 **merge** 的效率就很低。我们希望类似 hive 的 "insert overwrite" 操作，以忽略现有数据，只用提供的新数据创建一个提交。

也可以用于某些操作任务，如修复指定的问题分区。我们可以用源文件中的记录对该分区进行'插入覆盖'。对于某些数据源来说，这比还原和重放要快得多。

Insert overwrite 操作可能比批量 ETL 作业的 **upsert** 更快，批量 ETL 作业是每一批次都要重新计算整个目标分区（包括索引、预组合和其他重分区步骤）。

1) 查看当前表的 key

```
spark.  
  read.format("hudi").  
  load(basePath).  
  select("uuid", "partitionpath").  
  sort("partitionpath", "uuid").  
  show(100, false)
```

2) 生成一些新的行程数据

```
val inserts = convertToStringList(dataGen.generateInserts(10))  
val df = spark.  
  read.json(spark.sparkContext.parallelize(inserts, 2)).
```

```
filter("partitionpath = 'americas/united_states/san_francisco'")
```

3) 覆盖指定分区

```
df.write.format("hudi").
  options(getQuickstartWriteConfigs()).
  option(OPERATION.key(), "insert_overwrite").
  option(PRECOMBINE_FIELD.key(), "ts").
  option(RECORDKEY_FIELD.key(), "uuid").
  option(PARTITIONPATH_FIELD.key(), "partitionpath").
  option(TBL_NAME.key(), tableName).
  mode(Append).
  save(basePath)
```

4) 查询覆盖后的 key，发生了变化

```
spark.
  read.format("hudi").
  load(basePath).
  select("uuid", "partitionpath").
  sort("partitionpath", "uuid").
  show(100, false)
```

4.3 Spark SQL 方式

4.3.1 创建表

1) 启动 Hive 的 Metastore

```
nohup hive --service metastore &
```

1) 启动 spark-sql

```
#针对 Spark 3.2
spark-sql \
  --conf
'spark.serializer=org.apache.spark.serializer.KryoSerializer' \
  --conf
'spark.sql.catalog.spark_catalog=org.apache.spark.sql.hudi.catalog.HoodieCatalog' \
  --conf
'spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension'
```

如果没有配置 hive 环境变量，手动拷贝 hive-site.xml 到 spark 的 conf 下

2) 建表参数

参数名	默认值	说明
primaryKey	uuid	表的主键名，多个字段用逗号分隔。 同 hoodie.datasource.write.recordkey.field
preCombineField		表的预合并字段。 同 hoodie.datasource.write.precombine.field
type	cow	创建的表类型：

		type = 'cow' type = 'mor' 同 hoodie.datasource.write.table.type
--	--	--

3) 创建非分区表

(1) 创建一个 cow 表，默认 primaryKey 'uuid'，不提供 preCombineField

```
create table hudi_cow_nonpcf_tbl (
  uuid int,
  name string,
  price double
) using hudi;
```

(2) 创建一个 mor 非分区表

```
create table hudi_mor_tbl (
  id int,
  name string,
  price double,
  ts bigint
) using hudi
tblproperties (
  type = 'mor',
  primaryKey = 'id',
  preCombineField = 'ts'
);
```

4) 创建分区表

创建一个 cow 分区外部表，指定 primaryKey 和 preCombineField

```
create table hudi_cow_pt_tbl (
  id bigint,
  name string,
  ts bigint,
  dt string,
  hh string
) using hudi
tblproperties (
  type = 'cow',
  primaryKey = 'id',
  preCombineField = 'ts'
)
partitioned by (dt, hh)
location '/tmp/hudi/hudi_cow_pt_tbl';
```

5) 在已有的 hudi 表上创建新表

不需要指定模式和非分区列（如果存在）之外的任何属性，Hudi 可以自动识别模式和配置。

(1) 非分区表

```
create table hudi_existing_tbl0 using hudi
location 'file:///tmp/hudi/dataframe_hudi_nonpt_table';
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

(2) 分区表

```
create table hudi_existing_tbl1 using hudi
partitioned by (dt, hh)
location 'file:///tmp/hudi/dataframe_hudi_pt_table';
```

6) 通过 CTAS (Create Table As Select)建表

为了提高向 hudi 表加载数据的性能，CTAS 使用批量插入作为写操作。

(1) 通过 CTAS 创建 cow 非分区表，不指定 preCombineField

```
create table hudi_ctas_cow_nonpcf_tbl
using hudi
tblproperties (primaryKey = 'id')
as
select 1 as id, 'a1' as name, 10 as price;
```

(2) 通过 CTAS 创建 cow 分区表，指定 preCombineField

```
create table hudi_ctas_cow_pt_tbl
using hudi
tblproperties (type = 'cow', primaryKey = 'id', preCombineField = 'ts')
partitioned by (dt)
as
select 1 as id, 'a1' as name, 10 as price, 1000 as ts, '2021-12-01'
as dt;
```

(3) 通过 CTAS 从其他表加载数据

```
# 创建内部表
create table parquet_mngd using parquet location
'file:///tmp/parquet_dataset/*.parquet';

# 通过 CTAS 加载数据
create table hudi_ctas_cow_pt_tbl2 using hudi location
'file:/tmp/hudi/hudi_tbl/' options (
  type = 'cow',
  primaryKey = 'id',
  preCombineField = 'ts'
)
partitioned by (datestr) as select * from parquet_mngd;
```

4.3.2 插入数据

默认情况下，如果提供了 preCombineKey，则 insert into 的写操作类型为 upsert，否则使用 insert。

1) 向非分区表插入数据

```
insert into hudi_cow_nonpcf_tbl select 1, 'a1', 20;
insert into hudi_mor_tbl select 1, 'a1', 20, 1000;
```

2) 向分区表动态分区插入数据

```
insert into hudi_cow_pt_tbl partition (dt, hh)
select 1 as id, 'a1' as name, 1000 as ts, '2021-12-09' as dt, '10'
as hh;
```

3) 向分区表静态分区插入数据

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
insert into hudi_cow_pt_tbl partition(dt = '2021-12-09', hh='11')
select 2, 'a2', 1000;
```

4) 使用 bulk_insert 插入数据

hudi 支持使用 bulk_insert 作为写操作的类型，只需要设置两个配置：

hoodie.sql.bulk.insert.enable 和 hoodie.sql.insert.mode。

```
-- 向指定 preCombineKey 的表插入数据，则写操作为 upsert
insert into hudi_mor_tbl select 1, 'a1_1', 20, 1001;
select id, name, price, ts from hudi_mor_tbl;
1  a1_1    20.0    1001

-- 向指定 preCombineKey 的表插入数据，指定写操作为 bulk_insert
set hoodie.sql.bulk.insert.enable=true;
set hoodie.sql.insert.mode=non-strict;

insert into hudi_mor_tbl select 1, 'a1_2', 20, 1002;
select id, name, price, ts from hudi_mor_tbl;
1  a1_1    20.0    1001
1  a1_2    20.0    1002
```

4.3.3 查询数据

1) 查询

```
select fare, begin_lon, begin_lat, ts from hudi_trips_snapshot where
fare > 20.0
```

2) 时间旅行查询

Hudi 从 0.9.0 开始就支持时间旅行查询。Spark SQL 方式要求 Spark 版本 3.2 及以上。

```
-- 关闭前面开启的 bulk_insert
set hoodie.sql.bulk.insert.enable=false;

create table hudi_cow_pt_tbl1 (
  id bigint,
  name string,
  ts bigint,
  dt string,
  hh string
) using hudi
tblproperties (
  type = 'cow',
  primaryKey = 'id',
  preCombineField = 'ts'
)
partitioned by (dt, hh)
location '/tmp/hudi/hudi_cow_pt_tbl1';

-- 插入一条 id 为 1 的数据
insert into hudi_cow_pt_tbl1 select 1, 'a0', 1000, '2021-12-09', '10';
select * from hudi_cow_pt_tbl1;

-- 修改 id 为 1 的数据
```

```
insert into hudi_cow_pt_tbl1 select 1, 'a1', 1001, '2021-12-09', '10';
select * from hudi_cow_pt_tbl1;

-- 基于第一次提交时间进行时间旅行
select * from hudi_cow_pt_tbl1 timestamp as of '20220307091628793'
where id = 1;

-- 其他时间格式的时间旅行写法
select * from hudi_cow_pt_tbl1 timestamp as of '2022-03-07
09:16:28.100' where id = 1;

select * from hudi_cow_pt_tbl1 timestamp as of '2022-03-08' where id
= 1;
```

4.3.4 更新数据

1) update

更新操作需要指定 `preCombineField`。

(1) 语法

```
UPDATE tableIdentifier SET column = EXPRESSION [, column = EXPRESSION]
[ WHERE boolExpression]
```

(2) 执行更新

```
update hudi_mor_tbl set price = price * 2, ts = 1111 where id = 1;
update hudi_cow_pt_tbl1 set name = 'a1_1', ts = 1001 where id = 1;

-- update using non-PK field
update hudi_cow_pt_tbl1 set ts = 1111 where name = 'a1_1';
```

2) MergeInto

(1) 语法

```
MERGE INTO tableIdentifier AS target_alias
USING (sub_query | tableIdentifier) AS source_alias
ON <merge_condition>
[ WHEN MATCHED [ AND <condition> ] THEN <matched_action> ]
[ WHEN MATCHED [ AND <condition> ] THEN <matched_action> ]
[ WHEN NOT MATCHED [ AND <condition> ] THEN <not_matched_action> ]

<merge_condition> = A equal bool condition
<matched_action> =
    DELETE |
    UPDATE SET * |
    UPDATE SET column1 = expression1 [, column2 = expression2 ...]
<not_matched_action> =
    INSERT * |
    INSERT (column1 [, column2 ...]) VALUES (value1 [, value2 ...])
```

(2) 执行案例

```
-- 1、准备 source 表: 非分区的 hudi 表, 插入数据
create table merge_source (id int, name string, price double, ts bigint)
using hudi
tblproperties (primaryKey = 'id', preCombineField = 'ts');
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
insert into merge_source values (1, "old_a1", 22.22, 2900), (2,
"new_a2", 33.33, 2000), (3, "new_a3", 44.44, 2000);

merge into hudi_mor_tbl as target
using merge_source as source
on target.id = source.id
when matched then update set *
when not matched then insert *
;

-- 2、准备 source 表: 分区的 parquet 表, 插入数据
create table merge_source2 (id int, name string, flag string, dt string,
hh string) using parquet;
insert into merge_source2 values (1, "new_a1", 'update', '2021-12-09',
'10'), (2, "new_a2", 'delete', '2021-12-09', '11'), (3, "new_a3",
'insert', '2021-12-09', '12');

merge into hudi_cow_pt_tbl1 as target
using (
  select id, name, '2000' as ts, flag, dt, hh from merge_source2
) source
on target.id = source.id
when matched and flag != 'delete' then
  update set id = source.id, name = source.name, ts = source.ts, dt =
source.dt, hh = source.hh
when matched and flag = 'delete' then delete
when not matched then
  insert (id, name, ts, dt, hh) values(source.id, source.name,
source.ts, source.dt, source.hh)
;
```

4.2.5 删除数据

1) 语法

```
DELETE FROM tableIdentifier [ WHERE BOOL_EXPRESSION]
```

2) 案例

```
delete from hudi_cow_nonpcf_tbl where uuid = 1;

delete from hudi_mor_tbl where id % 2 = 0;

-- 使用非主键字段删除
delete from hudi_cow_pt_tbl1 where name = 'a1_1';
```

4.2.6 覆盖数据

- 使用 INSERT_OVERWRITE 类型的写操作覆盖分区表
- 使用 INSERT_OVERWRITE_TABLE 类型的写操作插入覆盖非分区表或分区表(动态分区)

1) insert overwrite 非分区表

```
insert overwrite hudi_mor_tbl select 99, 'a99', 20.0, 900;
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
insert overwrite hudi_cow_nonpcf_tbl select 99, 'a99', 20.0;
```

2) 通过动态分区 **insert overwrite table** 到分区表

```
insert overwrite table hudi_cow_pt_tbl1 select 10, 'a10', 1100, '2021-12-09', '11';
```

3) 通过静态分区 **insert overwrite** 分区表

```
insert overwrite hudi_cow_pt_tbl1 partition(dt = '2021-12-09', hh='12') select 13, 'a13', 1100;
```

4.2.7 修改表结构 (Alter Table)

1) 语法

```
-- Alter table name
ALTER TABLE oldTableName RENAME TO newTableName

-- Alter table add columns
ALTER TABLE tableIdentifier ADD COLUMNS(colAndType (,colAndType)*)

-- Alter table column type
ALTER TABLE tableIdentifier CHANGE COLUMN colName colName colType

-- Alter table properties
ALTER TABLE tableIdentifier SET TBLPROPERTIES (key = 'value')
```

2) 案例

```
--rename to:
ALTER TABLE hudi_cow_nonpcf_tbl RENAME TO hudi_cow_nonpcf_tbl2;

--add column:
ALTER TABLE hudi_cow_nonpcf_tbl2 add columns(remark string);

--change column:
ALTER TABLE hudi_cow_nonpcf_tbl2 change column uuid uuid int;

--set properties;
alter table hudi_cow_nonpcf_tbl2 set tblproperties
(hoodie.keep.max.commits = '10');
```

4.2.8 修改分区

1) 语法

```
-- Drop Partition
ALTER TABLE tableIdentifier DROP PARTITION ( partition_col_name =
partition_col_val [ , ... ] )

-- Show Partitions
SHOW PARTITIONS tableIdentifier
```

2) 案例

```
--show partition:
show partitions hudi_cow_pt_tbl1;

--drop partition:
alter table hudi_cow_pt_tbl1 drop partition (dt='2021-12-09',
```

```
hh='10');
```

注意：show partition 结果是基于文件系统表路径的。删除整个分区数据或直接删除某个分区目录并不精确。

4.2.9 存储过程（Procedures）

1) 语法

```
--Call procedure by positional arguments
CALL system.procedure_name(arg_1, arg_2, ... arg_n)

--Call procedure by named arguments
CALL system.procedure_name(arg_name_2 => arg_2, arg_name_1 =>
arg_1, ... arg_name_n => arg_n)
```

2) 案例

可用的存储过程：<https://hudi.apache.org/docs/procedures/>

```
--show commit's info
call show_commits(table => 'hudi_cow_pt_tbl1', limit => 10);
```

4.4 IDEA 编码方式

除了用 shell 交互式的操作，还可以自己编写 Spark 程序，打包提交。

4.4.1 环境准备

创建 Maven 工程，pom 文件：

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.atguigu.hudi</groupId>
  <artifactId>spark-hudi-demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <scala.version>2.12.10</scala.version>
    <scala.binary.version>2.12</scala.binary.version>
    <spark.version>3.2.2</spark.version>
    <hadoop.version>3.1.3</hadoop.version>
    <hudi.version>0.12.0</hudi.version>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
  </properties>

  <dependencies>
    <!-- 依赖 Scala 语言 -->
    <dependency>
      <groupId>org.scala-lang</groupId>
      <artifactId>scala-library</artifactId>
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

```
        <version>${scala.version}</version>
      </dependency>
      <!-- Spark Core 依赖 -->
      <dependency>
        <groupId>org.apache.spark</groupId>

<artifactId>spark-core_${scala.binary.version}</artifactId>
        <version>${spark.version}</version>
        <scope>provided</scope>
      </dependency>
      <!-- Spark SQL 依赖 -->
      <dependency>
        <groupId>org.apache.spark</groupId>

<artifactId>spark-sql_${scala.binary.version}</artifactId>
        <version>${spark.version}</version>
        <scope>provided</scope>
      </dependency>

      <dependency>
        <groupId>org.apache.spark</groupId>

<artifactId>spark-hive_${scala.binary.version}</artifactId>
        <version>${spark.version}</version>
        <scope>provided</scope>
      </dependency>

      <!-- Hadoop Client 依赖 -->
      <dependency>
        <groupId>org.apache.hadoop</groupId>
        <artifactId>hadoop-client</artifactId>
        <version>${hadoop.version}</version>
        <scope>provided</scope>
      </dependency>

      <!-- hudi-spark3.2 -->
      <dependency>
        <groupId>org.apache.hudi</groupId>

<artifactId>hudi-spark3.2-bundle_${scala.binary.version}</artifac
tId>
        <version>${hudi.version}</version>
        <scope>provided</scope>
      </dependency>

    </dependencies>

    <build>
      <plugins>
        <!-- assembly 打包插件 -->
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-assembly-plugin</artifactId>
          <version>3.0.0</version>
          <executions>
            <execution>
```

```
        <id>make-assembly</id>
        <phase>package</phase>
        <goals>
            <goal>single</goal>
        </goals>
    </execution>
</executions>
<configuration>
    <archive>
        <manifest>
            </manifest>
        </archive>
    <descriptorRefs>

<descriptorRef>jar-with-dependencies</descriptorRef>
    </descriptorRefs>
    </configuration>
</plugin>

<!--Maven 编译 scala 所需依赖-->
<plugin>
    <groupId>net.alchim31.maven</groupId>
    <artifactId>scala-maven-plugin</artifactId>
    <version>3.2.2</version>
    <executions>
        <execution>
            <goals>
                <goal>compile</goal>
                <goal>testCompile</goal>
            </goals>
        </execution>
    </executions>
</plugin>
</plugins>
</build>

</project>
```

4.4.2 插入数据

```
package com.atguigu.hudi.spark

import org.apache.hudi.QuickstartUtils._
import org.apache.spark.SparkConf
import org.apache.spark.sql._
import scala.collection.JavaConversions._
import org.apache.spark.sql.SaveMode._
import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.config.HoodieWriteConfig._

object InsertDemo {
    def main( args: Array[String] ): Unit = {
        // 创建 SparkSession
        val sparkConf = new SparkConf()
            .setAppName(this.getClass.getSimpleName)
            .setMaster("local[*]")
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
.set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
val sparkSession = SparkSession.builder()
    .config(sparkConf)
    .enableHiveSupport()
    .getOrCreate()

val tableName = "hudi_trips_cow"
val basePath = "hdfs://hadoop1:8020/tmp/hudi_trips_cow"
val dataGen = new DataGenerator

val inserts = convertToStringList(dataGen.generateInserts(10))
val df = sparkSession.read.json(sparkSession.sparkContext.parallelize(inserts, 2))
df.write.format("hudi").
    options(getQuickstartWriteConfigs).
    option(PRECOMBINE_FIELD.key(), "ts").
    option(RECORDKEY_FIELD.key(), "uuid").
    option(PARTITIONPATH_FIELD.key(), "partitionpath").
    option(TBL_NAME.key(), tableName).
    mode(Overwrite).
    save(basePath)
}
```

4.4.3 查询数据

```
package com.atguigu.hudi.spark

import org.apache.spark.SparkConf
import org.apache.spark.sql._

object QueryDemo {
    def main( args: Array[String] ): Unit = {
        // 创建 SparkSession
        val sparkConf = new SparkConf()
            .setAppName(this.getClass.getSimpleName)
            .setMaster("local[*]")
            .set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
        val sparkSession = SparkSession.builder()
            .config(sparkConf)
            .enableHiveSupport()
            .getOrCreate()

        val basePath = "hdfs://hadoop1:8020/tmp/hudi_trips_cow"

        val tripsSnapshotDF = sparkSession.
            read.
            format("hudi").
            load(basePath)

        // 时间旅行查询写法一
        // sparkSession.read.
        // format("hudi").
```

```
//      option("as.of.instant", "20210728141108100").
//      load(basePath)
//
//      时间旅行查询写法二
//      sparkSession.read.
//      format("hudi").
//      option("as.of.instant", "2021-07-28 14:11:08.200").
//      load(basePath)
//
//      时间旅行查询写法三：等价于"as.of.instant = 2021-07-28 00:00:00"
//      sparkSession.read.
//      format("hudi").
//      option("as.of.instant", "2021-07-28").
//      load(basePath)

tripsSnapshotDF.createOrReplaceTempView("hudi_trips_snapshot")

sparkSession
  .sql("select    fare,    begin_lon,    begin_lat,    ts    from
hudi_trips_snapshot where fare > 20.0")
  .show()

}
}
```

4.4.4 更新数据

```
package com.atguigu.hudi.spark

import org.apache.hudi.QuickstartUtils._
import org.apache.spark.SparkConf
import org.apache.spark.sql._
import scala.collection.JavaConversions._
import org.apache.spark.sql.SaveMode._
import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.config.HoodieWriteConfig._

object UpdateDemo {
  def main( args: Array[String] ): Unit = {
    // 创建 SparkSession
    val sparkConf = new SparkConf()
      .setAppName(this.getClass.getSimpleName)
      .setMaster("local[*]")
      .set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
    val sparkSession = SparkSession.builder()
      .config(sparkConf)
      .enableHiveSupport()
      .getOrCreate()

    val tableName = "hudi_trips_cow"
    val basePath = "hdfs://hadoop1:8020/tmp/hudi_trips_cow"

    val dataGen = new DataGenerator
    val updates = convertToStringList(dataGen.generateUpdates(10))
    val df =
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
sparkSession.read.json(sparkSession.sparkContext.parallelize(updates, 2))
df.write.format("hudi").
  options(getQuickstartWriteConfigs).
  option(PRECOMBINE_FIELD.key(), "ts").
  option(RECORDKEY_FIELD.key(), "uuid").
  option(PARTITIONPATH_FIELD.key(), "partitionpath").
  option(TBL_NAME.key(), tableName).
  mode(Append).
  save(basePath)

// val tripsSnapshotDF = sparkSession.
//   read.
//     format("hudi").
//     load(basePath)
//
tripsSnapshotDF.createOrReplaceTempView("hudi_trips_snapshot")
//
// sparkSession
//   .sql("select fare, begin_lon, begin_lat, ts from
hudi_trips_snapshot where fare > 20.0")
//   .show()

}
}
```

4.4.5 指定时间点查询

```
package com.atguigu.hudi.spark

import org.apache.hudi.DataSourceReadOptions._
import org.apache.spark.SparkConf
import org.apache.spark.sql._

object PointInTimeQueryDemo {
  def main( args: Array[String] ): Unit = {
    // 创建 SparkSession
    val sparkConf = new SparkConf()
      .setAppName(this.getClass.getSimpleName)
      .setMaster("local[*]")
      .set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
    val sparkSession = SparkSession.builder()
      .config(sparkConf)
      .enableHiveSupport()
      .getOrCreate()

    val basePath = "hdfs://hadoop1:8020/tmp/hudi_trips_cow"

    import sparkSession.implicits._
    val commits = sparkSession.sql("select
distinct( _hoodie_commit_time) as commitTime from
hudi_trips_snapshot order by commitTime").map(k =>
k.getString(0)).take(50)
    val beginTime = "000"
```

```
val endTime = commits(commits.length - 2)

val tripsIncrementalDF = sparkSession.read.format("hudi").
  option(QUERY_TYPE.key(), QUERY_TYPE_INCREMENTAL_OPT_VAL).
  option(BEGIN_INSTANTTIME.key(), beginTime).
  option(END_INSTANTTIME.key(), endTime).
  load(basePath)

tripsIncrementalDF.createOrReplaceTempView("hudi_trips_point_in_time")

sparkSession.
  sql("select `_hoodie_commit_time`, fare, begin_lon, begin_lat,
ts from hudi_trips_point_in_time where fare > 20.0").
  show()

}
}
```

4.4.6 增量查询

```
package com.atguigu.hudi.spark

import org.apache.hudi.DataSourceReadOptions._
import org.apache.spark.SparkConf
import org.apache.spark.sql._

object IncrementalQueryDemo {
  def main( args: Array[String] ): Unit = {
    // 创建 SparkSession
    val sparkConf = new SparkConf()
      .setAppName(this.getClass.getSimpleName)
      .setMaster("local[*]")
      .set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
    val sparkSession = SparkSession.builder()
      .config(sparkConf)
      .enableHiveSupport()
      .getOrCreate()

    val basePath = "hdfs://hadoop1:8020/tmp/hudi_trips_cow"

    import sparkSession.implicits._
    val commits = sparkSession.sql("select
distinct(_hoodie_commit_time) as commitTime from
hudi_trips_snapshot order by commitTime").map(k =>
k.getString(0)).take(50)
    val beginTime = commits(commits.length - 2)

    val tripsIncrementalDF = sparkSession.read.format("hudi").
      option(QUERY_TYPE.key(), QUERY_TYPE_INCREMENTAL_OPT_VAL).
      option(BEGIN_INSTANTTIME.key(), beginTime).
      load(basePath)
```

```
tripsIncrementalDF.createOrReplaceTempView("hudi_trips_incremental")

    sparkSession.sql("select `hoodie_commit_time`, fare, begin_lon,
begin_lat, ts from hudi_trips_incremental where fare > 20.0").show()

}
}
```

4.4.7 删除数据

```
package com.atguigu.hudi.spark

import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.QuickstartUtils._
import org.apache.hudi.config.HoodieWriteConfig._
import org.apache.spark.SparkConf
import org.apache.spark.sql.SaveMode._
import org.apache.spark.sql._

import scala.collection.JavaConversions._

object DeleteDemo {
  def main( args: Array[String] ): Unit = {
    // 创建 SparkSession
    val sparkConf = new SparkConf()
      .setAppName(this.getClass.getSimpleName)
      .setMaster("local[*]")
      .set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
    val sparkSession = SparkSession.builder()
      .config(sparkConf)
      .enableHiveSupport()
      .getOrCreate()

    val tableName = "hudi_trips_cow"
    val basePath = "hdfs://hadoop1:8020/tmp/hudi_trips_cow"
    val dataGen = new DataGenerator

    sparkSession.
      read.
      format("hudi").
      load(basePath).
      createOrReplaceTempView("hudi_trips_snapshot")

    sparkSession.sql("select      uuid,      partitionpath      from
hudi_trips_snapshot").count()

    val ds = sparkSession.sql("select uuid, partitionpath from
hudi_trips_snapshot").limit(2)

    val deletes = dataGen.generateDeletes(ds.collectAsList())
    val df =
sparkSession.read.json(sparkSession.sparkContext.parallelize(deletes, 2))
  }
}
```

```
df.write.format("hudi").
  options(getQuickstartWriteConfigs).
  option(OPERATION.key(), "delete").
  option(PRECOMBINE_FIELD.key(), "ts").
  option(RECORDKEY_FIELD.key(), "uuid").
  option(PARTITIONPATH_FIELD.key(), "partitionpath").
  option(TBL_NAME.key(), tableName).
  mode(Append).
  save(basePath)

val roAfterDeleteViewDF = sparkSession.
  read.
  format("hudi").
  load(basePath)

roAfterDeleteViewDF.createOrReplaceTempView("hudi_trips_snapshot"
)

// 返回的总行数应该比原来少 2 行
sparkSession.sql("select      uuid,      partitionpath      from
hudi_trips_snapshot").count()

}
}
```

4.4.8 覆盖数据

```
package com.atguigu.hudi.spark

import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.QuickstartUtils._
import org.apache.hudi.config.HoodieWriteConfig._
import org.apache.spark.SparkConf
import org.apache.spark.sql.SaveMode._
import org.apache.spark.sql._

import scala.collection.JavaConversions._

object InsertOverwriteDemo {
  def main( args: Array[String] ): Unit = {
    // 创建 SparkSession
    val sparkConf = new SparkConf()
      .setAppName(this.getClass.getSimpleName)
      .setMaster("local[*]")
      .set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
    val sparkSession = SparkSession.builder()
      .config(sparkConf)
      .enableHiveSupport()
      .getOrCreate()

    val tableName = "hudi_trips_cow"
    val basePath = "hdfs://hadoop1:8020/tmp/hudi_trips_cow"
    val dataGen = new DataGenerator
```

```
sparkSession.  
  read.format("hudi").  
  load(basePath).  
  select("uuid","partitionpath").  
  sort("partitionpath","uuid").  
  show(100, false)  
  
val inserts = convertToStringList(dataGen.generateInserts(10))  
val df =  
sparkSession.read.json(sparkSession.sparkContext.parallelize(inserts, 2)).  
  filter("partitionpath =  
'americas/united_states/san_francisco'")  
  
df.write.format("hudi").  
  options(getQuickstartWriteConfigs).  
  option(OPERATION.key(), "insert_overwrite").  
  option(PRECOMBINE_FIELD.key(), "ts").  
  option(RECORDKEY_FIELD.key(), "uuid").  
  option(PARTITIONPATH_FIELD.key(), "partitionpath").  
  option(TBL_NAME.key(), tableName).  
  mode(Append).  
  save(basePath)  
  
sparkSession.  
  read.format("hudi").  
  load(basePath).  
  select("uuid","partitionpath").  
  sort("partitionpath","uuid").  
  show(100, false)  
}
```

4.4.9 提交运行

将代码打成 jar 包，上传到目录 myjars，执行提交命令（QueryDemo 为例）：

```
spark-submit \  
--class com.atguigu.hudi.spark.QueryDemo \  
/opt/module/spark-3.2.2/myjars/spark-hudi-demo-1.0-SNAPSHOT-jar-with-dependencies.jar
```

4.5 DeltaStreamer 导入工具

HoodieDeltaStreamer 工具 (hudi-utilities-bundle 中的一部分) 提供了从 DFS 或 Kafka 等不同来源进行摄取的方式，并具有以下功能：

- 精准一次从 Kafka 采集新数据，从 Sqoop、HiveIncrementalPuller 的输出或 DFS 文件夹下的文件增量导入。
- 导入的数据支持 json、avro 或自定义数据类型。
- 管理检查点，回滚和恢复。

- 利用 DFS 或 Confluent schema registry 的 Avro Schema。
- 支持自定义转换操作。

4.5.1 命令说明

执行如下命令，查看帮助文档：

```
spark-submit --class org.apache.hudi.utilities.deltastreamer.HoodieDeltaStreamer /opt/software/hudi-0.12.0/packaging/hudi-utilities-bundle/target/hudi-utilities-bundle_2.12-0.12.0.jar --help
```

Schema Provider 和 Source 配置项：https://hudi.apache.org/docs/hoodie_deltastreamer

下面以 File Based Schema Provider 和 JsonKafkaSource 为例：

4.5.2 准备 Kafka 数据

(1) 启动 kafka 集群，创建测试用的 topic

```
bin/kafka-topics.sh --bootstrap-server hadoop1:9092 --create --topic hudi_test
```

(2) 准备 java 生产者代码往 topic 发送测试数据

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-clients</artifactId>
  <version>2.4.1</version>
</dependency>

<!--fastjson <= 1.2.80 存在安全漏洞,-->
<dependency>
  <groupId>com.alibaba</groupId>
  <artifactId>fastjson</artifactId>
  <version>1.2.83</version>
</dependency>

package com.atguigu.util;

import com.alibaba.fastjson.JSONObject;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;

import java.util.Properties;
import java.util.Random;

public class TestKafkaProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers",
"hadoop1:9092,hadoop2:9092,hadoop3:9092");
        props.put("acks", "-1");
        props.put("batch.size", "1048576");
        props.put("linger.ms", "5");
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
        props.put("compression.type", "snappy");
        props.put("buffer.memory", "33554432");
        props.put("key.serializer",
"org.apache.kafka.common.serialization.StringSerializer");
        props.put("value.serializer",
"org.apache.kafka.common.serialization.StringSerializer");
        KafkaProducer<String, String> producer = new
KafkaProducer<String, String>(props);
        Random random = new Random();
        for (int i = 0; i < 1000; i++) {
            JSONObject model = new JSONObject();
            model.put("userid", i);
            model.put("username", "name" + i);
            model.put("age", 18);
            model.put("partition", random.nextInt(100));
            producer.send(new ProducerRecord<String,
String>("hudi_test", model.toJSONString()));
        }
        producer.flush();
        producer.close();
    }
}
```

4.5.3 准备配置文件

(1) 定义 arvo 所需 schema 文件（包括 source 和 target）

```
mkdir /opt/module/hudi-props/
vim /opt/module/hudi-props/source-schema-json.avsc
{
  "type": "record",
  "name": "Profiles",
  "fields": [
    {
      "name": "userid",
      "type": [ "null", "string" ],
      "default": null
    },
    {
      "name": "username",
      "type": [ "null", "string" ],
      "default": null
    },
    {
      "name": "age",
      "type": [ "null", "string" ],
      "default": null
    },
    {
      "name": "partition",
      "type": [ "null", "string" ],
      "default": null
    }
  ]
}
```

```
cp source-schema-json.avsc target-schema-json.avsc
```

(2) 拷贝 hudi 配置 base.properties

```
cp
/opt/software/hudi-0.12.0/hudi-utilities/src/test/resources/delta-
streamer-config/base.properties /opt/module/hudi-props/
```

(3) 根据源码里提供的模板, 编写自己的 kafka source 的配置文件

```
cp
/opt/software/hudi-0.12.0/hudi-utilities/src/test/resources/delta-
streamer-config/kafka-source.properties /opt/module/hudi-props/

vim /opt/module/hudi-props/kafka-source.properties

include=hdfs://hadoop1:8020/hudi-props/base.properties
# Key fields, for kafka example
hoodie.datasource.write.recordkey.field=userid
hoodie.datasource.write.partitionpath.field=partition
# schema provider configs
hoodie.deltastreamer.schemaprovider.source.schema.file=hdfs://had
oop1:8020/hudi-props/source-schema-json.avsc
hoodie.deltastreamer.schemaprovider.target.schema.file=hdfs://had
oop1:8020/hudi-props/target-schema-json.avsc
# Kafka Source
hoodie.deltastreamer.source.kafka.topic=hudi_test
#Kafka props
bootstrap.servers=hadoop1:9092,hadoop2:9092,hadoop3:9092
auto.offset.reset=earliest
group.id=test-group
```

(4) 将配置文件上传到 hdfs

```
hadoop fs -put /opt/module/hudi-props/ /
```

4.5.4 拷贝所需 jar 包到 Spark

```
cp
/opt/software/hudi-0.12.0/packaging/hudi-utilities-bundle/target/
hudi-utilities-bundle_2.12-0.12.0.jar
/opt/module/spark-3.2.2/jars/
```

需要把 hudi-utilities-bundle_2.12-0.12.0.jar 放入 spark 的 jars 路径下, 否则报错找不到一些类和方法。

4.5.5 运行导入命令

```
spark-submit \
--class
org.apache.hudi.utilities.deltastreamer.HoodieDeltaStreamer \
/opt/module/spark-3.2.2/jars/hudi-utilities-bundle_2.12-0.12.0.jar \
--props hdfs://hadoop1:8020/hudi-props/kafka-source.properties \
--schemaprovider-class
org.apache.hudi.utilities.schema.FilebasedSchemaProvider \
--source-class org.apache.hudi.utilities.sources.JsonKafkaSource \
--source-ordering-field userid \
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
--target-base-path hdfs://hadoop1:8020/tmp/hudi/hudi_test \
--target-table hudi_test \
--op BULK_INSERT \
--table-type MERGE_ON_READ
```

4.5.6 查看导入结果

(1) 启动 spark-sql

```
spark-sql \
--conf
'spark.serializer=org.apache.spark.serializer.KryoSerializer' \
--conf
'spark.sql.catalog.spark_catalog=org.apache.spark.sql.hudi.catalog.HoodieCatalog' \
--conf
'spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension'
```

(2) 指定 location 创建 hudi 表

```
use spark_hudi;

create table hudi_test using hudi
location 'hdfs://hadoop1:8020/tmp/hudi/hudi_test'
```

(3) 查询 hudi 表

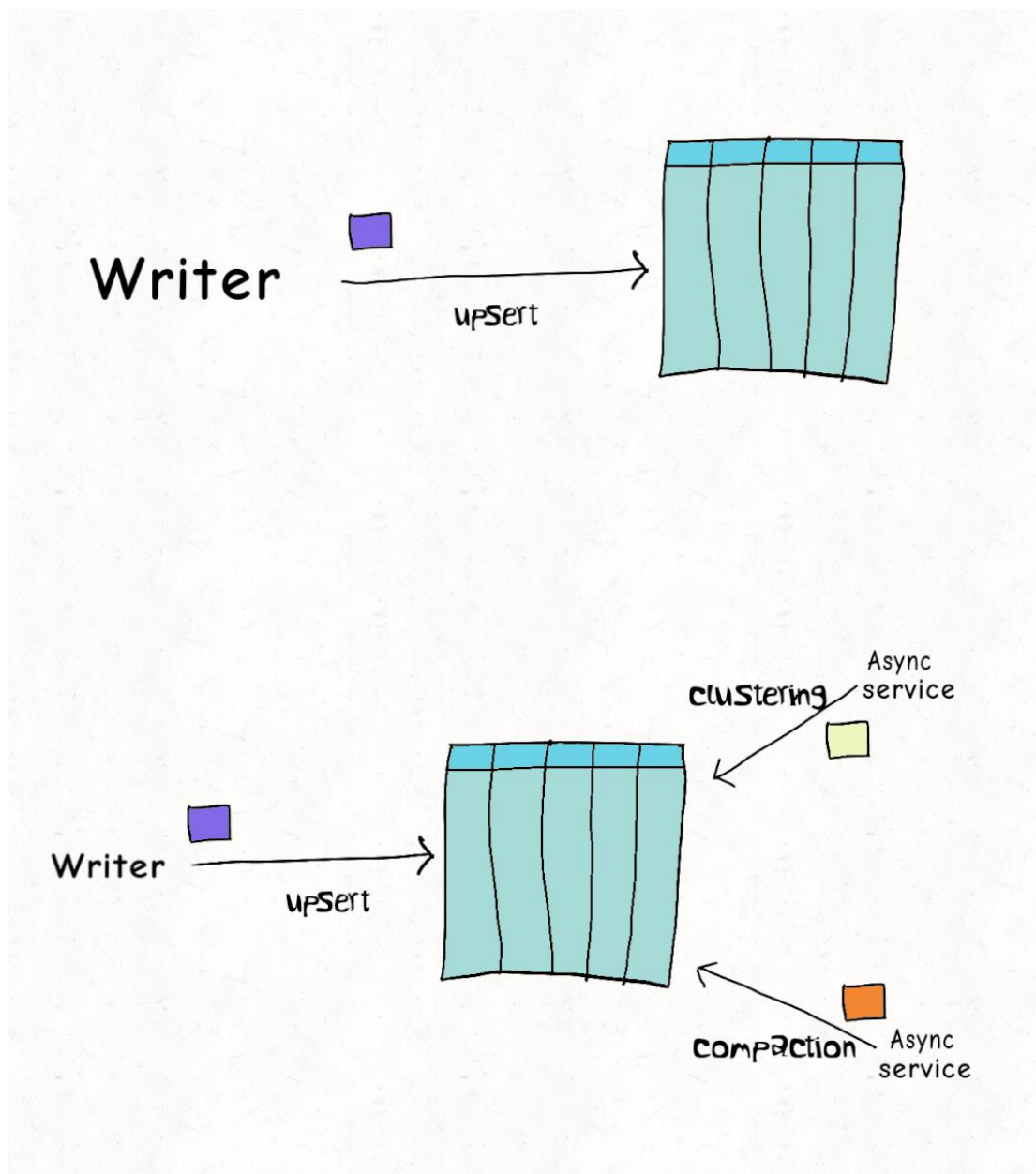
```
select * from hudi_test;
```

4.6 并发控制

4.6.1 Hudi 支持的并发控制

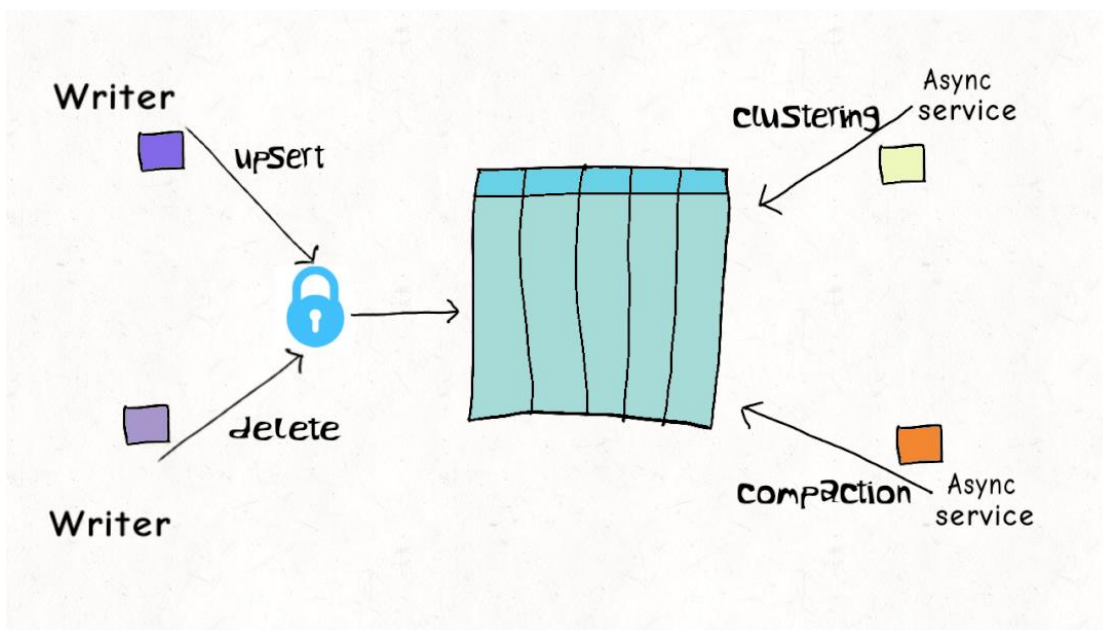
1) MVCC

Hudi 的表操作，如压缩、清理、提交，hudi 会利用多版本并发控制来提供多个表操作写入和查询之间的快照隔离。使用 MVCC 这种模型，Hudi 支持并发任意数量的操作作业，并保证不会发生任何冲突。Hudi 默认这种模型。MVCC 方式所有的 table service 都使用**同一个 writer** 来保证没有冲突，避免竞态条件。



2) OPTIMISTIC CONCURRENCY

针对写入操作（upsert、insert 等）利用乐观并发控制来启用**多个 writer** 将数据写到同一个表中，Hudi 支持文件级的乐观一致性，即对于发生在同一个表中的任何 2 个提交（写入），如果它们没有写入正在更改的重叠文件，则允许两个写入都成功。此功能处于实验阶段，需要用到 Zookeeper 或 HiveMetastore 来获取锁。



4.6.2 使用并发写方式

1) 参数

(1) 如果需要开启乐观并发写入，需要设置以下属性

```
hoodie.write.concurrency.mode=optimistic_concurrency_control
hoodie.cleaner.policy.failed.writes=LAZY
hoodie.write.lock.provider=<lock-provider-classname>
```

Hudi 获取锁的服务提供两种模式使用 zookeeper、HiveMetaStore 或 Amazon DynamoDB

(选一种即可)

(2) 相关 zookeeper 参数

```
hoodie.write.lock.provider=org.apache.hudi.client.transaction.lock.ZookeeperBasedLockProvider
hoodie.write.lock.zookeeper.url
hoodie.write.lock.zookeeper.port
hoodie.write.lock.zookeeper.lock_key
hoodie.write.lock.zookeeper.base_path
```

(3) 相关 HiveMetastore 参数，HiveMetastore URI 是从运行时加载的 hadoop 配置文件中提取的

```
hoodie.write.lock.provider=org.apache.hudi.hive.HiveMetastoreBasedLockProvider
hoodie.write.lock.hivemetastore.database
hoodie.write.lock.hivemetastore.table
```

4.6.3 使用 Spark DataFrame 并发写入

(1) 启动 spark-shell

```
spark-shell \
--conf
'spark.serializer=org.apache.spark.serializer.KryoSerializer' \
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
--conf
'spark.sql.catalog.spark_catalog=org.apache.spark.sql.hudi.catalog.HoodieCatalog' \
--conf
'spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension'
```

(2) 编写代码

```
import org.apache.hudi.QuickstartUtils._
import scala.collection.JavaConversions._
import org.apache.spark.sql.SaveMode._
import org.apache.hudi.DataSourceReadOptions._
import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.config.HoodieWriteConfig._

val tableName = "hudi_trips_cow"
val basePath = "file:///tmp/hudi_trips_cow"
val dataGen = new DataGenerator

val inserts = convertToStringList(dataGen.generateInserts(10))
val df = spark.read.json(spark.sparkContext.parallelize(inserts, 2))
df.write.format("hudi").
  options(getQuickstartWriteConfigs).
  option(PRECOMBINE_FIELD_OPT_KEY, "ts").
  option(RECORDKEY_FIELD_OPT_KEY, "uuid").
  option(PARTITIONPATH_FIELD_OPT_KEY, "partitionpath").
  option("hoodie.write.concurrency.mode",
"optimistic_concurrency_control").
  option("hoodie.cleaner.policy.failed.writes", "LAZY").
  option("hoodie.write.lock.provider",
"org.apache.hudi.client.transaction.lock.ZookeeperBasedLockProvider").
  option("hoodie.write.lock.zookeeper.url",
"hadoop1,hadoop2,hadoop3").
  option("hoodie.write.lock.zookeeper.port", "2181").
  option("hoodie.write.lock.zookeeper.lock_key", "test_table").
  option("hoodie.write.lock.zookeeper.base_path",
"/multiwriter_test").
  option(TABLE_NAME, tableName).
  mode(Append).
  save(basePath)
```

(3) 使用 zk 客户端，验证是否使用了 zk。

```
/opt/module/apache-zookeeper-3.5.7/bin/zkCli.sh
[zk: localhost:2181(CONNECTED) 0] ls /
```

(4) zk 下产生了对应的目录，/multiwriter_test 下的目录，为代码里指定的 lock_key

```
[zk: localhost:2181(CONNECTED) 1] ls /multiwriter_test
```

```
[zk: localhost:2181(CONNECTED) 0] ls /
[aliases.json, clickhouse, clusterstate.json, collections, configs, flink-standalone, flink-yarn, hbase, kafka, kylli, live_nodes, multiwriter_test, overseer, overseer_elect, zookeeper]
[zk: localhost:2181(CONNECTED) 1] ls /multiwriter_test
[test_table]
```

4.6.4 使用 Delta Streamer 并发写入

基于前面 DeltaStreamer 的例子，使用 Delta Streamer 消费 kafka 的数据写入到 hudi 中，

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

这次加上并发写的参数。

1) 进入配置文件目录，修改配置文件添加对应参数，提交到 Hdfs 上

```
cd /opt/module/hudi-props/
cp kafka-source.properties kafka-multiwriter-source.properties
vim kafka-multiwriter-source.properties

hoodie.write.concurrency.mode=optimistic_concurrency_control
hoodie.cleaner.policy.failed.writes=LAZY
hoodie.write.lock.provider=org.apache.hudi.client.transaction.lock.ZookeeperBasedLockProvider
hoodie.write.lock.zookeeper.url=hadoop1,hadoop2,hadoop3
hoodie.write.lock.zookeeper.port=2181
hoodie.write.lock.zookeeper.lock_key=test_table2
hoodie.write.lock.zookeeper.base_path=/multiwriter_test2

hadoop fs -put
/opt/module/hudi-props/kafka-multiwriter-source.properties
/hudi-props
```

2) 运行 Delta Streamer

```
spark-submit \
--class
org.apache.hudi.utilities.deltastreamer.HoodieDeltaStreamer \
/opt/module/spark-3.2.2/jars/hudi-utilities-bundle_2.12-0.12.0.jar \
--props
hdfs://hadoop1:8020/hudi-props/kafka-multiwriter-source.properties \
--schemaprovider-class
org.apache.hudi.utilities.schema.FilebasedSchemaProvider \
--source-class org.apache.hudi.utilities.sources.JsonKafkaSource \
--source-ordering-field userid \
--target-base-path hdfs://hadoop1:8020/tmp/hudi/hudi_test_multi \
--target-table hudi_test_multi \
--op INSERT \
--table-type MERGE_ON_READ
```

3) 查看 zk 是否产生新的目录

```
/opt/module/apache-zookeeper-3.5.7-bin/bin/zkCli.sh
[zk: localhost:2181(CONNECTED) 0] ls /
[zk: localhost:2181(CONNECTED) 1] ls /multiwriter_test2

[zk: localhost:2181(CONNECTED) 23] ls /
[aliases.json, clickhouse, clusterstate.json, collections, configs, flink-standalone, flink-yarn, hbase, kafka, kylli, live_nodes, multiwriter_test2, overseer, overseer_elect, zookeeper]
[zk: localhost:2181(CONNECTED) 24] ls /multiwriter_test2
test_table2
```

4.7 常规调优

4.7.1 并行度

Hudi 对输入进行分区默认并发度为 1500，以确保每个 Spark 分区都在 2GB 的限制内（在 Spark2.4.0 版本之后去除了该限制），如果有更大的输入，则相应地进行调整。建议设置 shuffle

更多 [Java](#) - 大数据 - 前端 - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

的并发度，配置项为 `hoodie.[insert|upsert|bulkinsert].shuffle.parallelism`，以使其至少达到 `inputdatasize/500MB`。

4.7.2 Off-heap（堆外）内存

Hudi 写入 `parquet` 文件，需要使用一定的堆外内存，如果遇到此类故障，请考虑设置类似 `spark.yarn.executor.memoryOverhead` 或 `spark.yarn.driver.memoryOverhead` 的值。

4.7.3 Spark 内存

通常 Hudi 需要能够将单个文件读入内存以执行合并或压缩操作，因此执行程序的内存应足以容纳此文件。另外，Hudi 会缓存输入数据以便能够智能地放置数据，因此预留一些 `spark.memory.storageFraction` 通常有助于提高性能。

4.7.4 调整文件大小

设置 `limitFileSize` 以平衡接收/写入延迟与文件数量，并平衡与文件数据相关的元数据开销。

4.7.5 时间序列/日志数据

对于单条记录较大的数据库/ `nosql` 变更日志，可调整默认配置。另一类非常流行的数据是时间序列/事件/日志数据，它往往更加庞大，每个分区的记录更多。在这种情况下，请考虑通过 `.bloomFilterFPP()/bloomFilterNumEntries()` 来调整 Bloom 过滤器的精度，以加速目标索引查找时间，另外可考虑一个以事件时间为前缀的键，这将使用范围修剪并显着加快索引查找的速度。

4.7.6 GC 调优

请确保遵循 Spark 调优指南中的垃圾收集调优技巧，以避免 `OutOfMemory` 错误。[必须] 使用 G1 / CMS 收集器，其中添加到 `spark.executor.extraJavaOptions` 的示例如下：

```
-XX:NewSize=1g          -XX:SurvivorRatio=2          -XX:+UseCompressedOops
-XX:+UseConcMarkSweepGC          -XX:+UseParNewGC
-XX:CMSInitiatingOccupancyFraction=70          -XX:+PrintGCDetails
-XX:+PrintGCTimeStamps          -XX:+PrintGCDateStamps
-XX:+PrintGCApplicationStoppedTime
-XX:+PrintGCApplicationConcurrentTime
-XX:+PrintTenuringDistribution          -XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/hoodie-heapdump.hprof
```

4.7.7 OutOfMemory 错误

如果出现 OOM 错误，则可尝试通过如下配置处理：`spark.memory.fraction=0.2`，
更多 [Java](#) - 大数据 - 前端 - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

spark.memory.storageFraction=0.2 允许其溢出而不是 OOM（速度变慢与间歇性崩溃相比）。

4.7.8 完整的生产配置

```
spark.driver.extraClassPath /etc/hive/conf
spark.driver.extraJavaOptions      -XX:+PrintTenuringDistribution
-XX:+PrintGCDetails                -XX:+PrintGCDateStamps
-XX:+PrintGCApplicationStoppedTime
-XX:+PrintGCApplicationConcurrentTime -XX:+PrintGCTimeStamps
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/hoodie-heapdump.hprof
spark.driver.maxResultSize 2g
spark.driver.memory 4g
spark.executor.cores 1
spark.executor.extraJavaOptions      -XX:+PrintFlagsFinal
-XX:+PrintReferenceGC                -verbose:gc                -XX:+PrintGCDetails
-XX:+PrintGCTimeStamps                -XX:+PrintAdaptiveSizePolicy
-XX:+UnlockDiagnosticVMOptions        -XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/hoodie-heapdump.hprof
spark.executor.id driver
spark.executor.instances 300
spark.executor.memory 6g
spark.rdd.compress true

spark.kryoserializer.buffer.max 512m
spark.serializer org.apache.spark.serializer.KryoSerializer
spark.shuffle.service.enabled true
spark.sql.hive.convertMetastoreParquet false
spark.submit.deployMode cluster
spark.task.cpus 1
spark.task.maxFailures 4

spark.yarn.driver.memoryOverhead 1024
spark.yarn.executor.memoryOverhead 3072
spark.yarn.max.executor.failures 100
```

第 5 章 集成 Flink

Hudi	Supported Flink version
0.12.x	1.15.x、 1.14.x、 1.13.x
0.11.x	1.14.x、 1.13.x
0.10.x	1.13.x
0.9.0	1.12.2

0.11.x 不建议使用，如果要用请使用补丁分支：<https://github.com/apache/hudi/pull/6182>

5.1 环境准备

1) 拷贝编译好的 jar 包到 Flink 的 lib 目录下

```
cp
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

```
/opt/software/hudi-0.12.0/packaging/hudi-flink-bundle/target/hudi-flink1.13-bundle_2.12-0.12.0.jar /opt/module/flink-1.13.6/lib/
```

2) 拷贝 guava 包, 解决依赖冲突

```
cp  
/opt/module/hadoop-3.1.3/share/hadoop/common/lib/guava-27.0-jre.jar /opt/module/flink-1.13.6/lib/
```

3) 配置 Hadoop 环境变量

```
sudo vim /etc/profile.d/my_env.sh  
  
export HADOOP_CLASSPATH=`hadoop classpath`  
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop  
  
source /etc/profile.d/my_env.sh
```

4) 启动 Hadoop (略)

5.2 sql-client 方式

5.2.1 启动 sql-client

1) 修改 flink-conf.yaml 配置

```
vim /opt/module/flink-1.13.6/conf/flink-conf.yaml  
  
classloader.check-leaked-classloader: false  
taskmanager.numberOfTaskSlots: 4  
  
state.backend: rocksdb  
execution.checkpointing.interval: 30000  
state.checkpoints.dir: hdfs://hadoop102:8020/ckps  
state.backend.incremental: true
```

2) local 模式

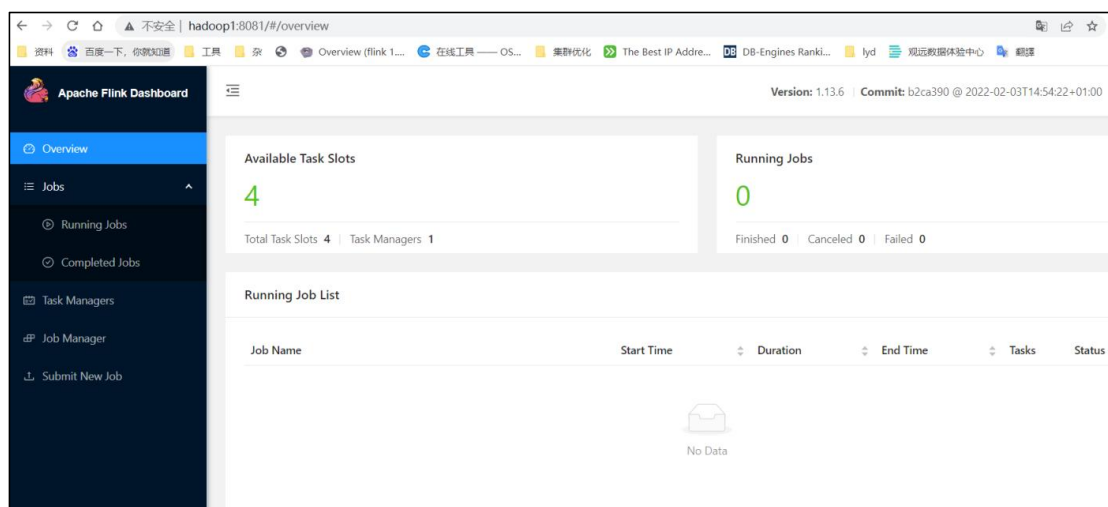
(1) 修改 workers

```
vim /opt/module/flink-1.13.6/conf/workers  
#表示: 会在本地启动 3 个 TaskManager 的 local 集群  
localhost  
localhost  
localhost
```

(2) 启动 Flink

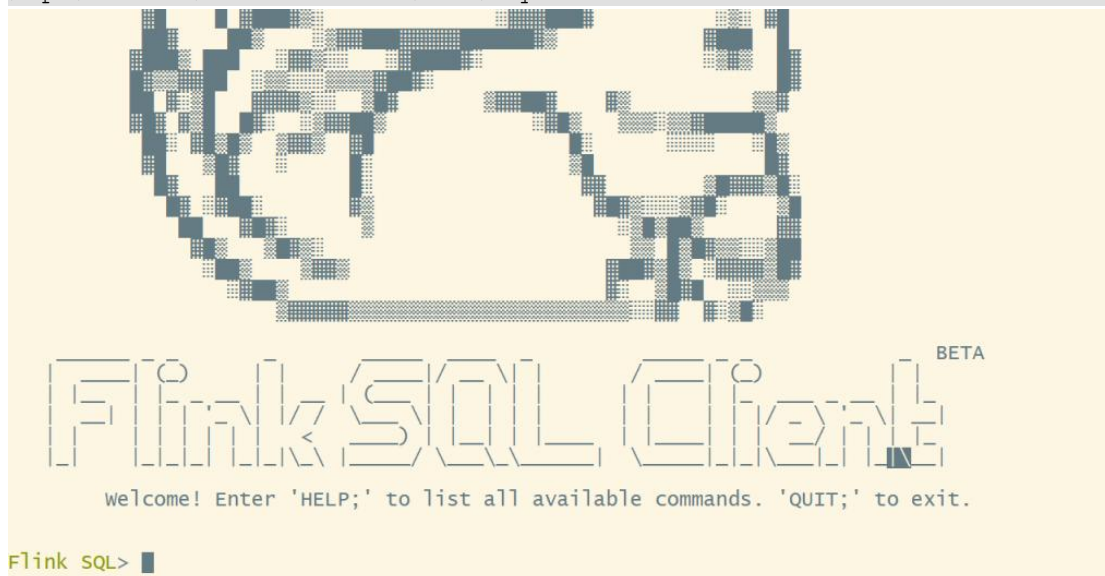
```
/opt/module/flink-1.13.6/bin/start-cluster.sh
```

查看 webui: <http://hadoop102:8081>



(3) 启动 Flink 的 sql-client

```
/opt/module/flink-1.13.6/bin/sql-client.sh embedded
```



3) yarn-session 模式

(1) 解决依赖问题

```
cp
/opt/module/hadoop-3.1.3/share/hadoop/mapreduce/hadoop-mapreduce-
client-core-3.1.3.jar /opt/module/flink-1.13.6/lib/
```

(2) 启动 yarn-session

```
/opt/module/flink-1.13.6/bin/yarn-session.sh -d
```

(3) 启动 sql-client

```
/opt/module/flink-1.13.6/bin/sql-client.sh embedded -s yarn-session
```

5.2.2 插入数据

```
set sql-client.execution.result-mode=tableau;
```

```
-- 创建 hudi 表
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
CREATE TABLE t1(  
  uuid VARCHAR(20) PRIMARY KEY NOT ENFORCED,  
  name VARCHAR(10),  
  age INT,  
  ts TIMESTAMP(3),  
  `partition` VARCHAR(20)  
)  
PARTITIONED BY (`partition`)  
WITH (  
  'connector' = 'hudi',  
  'path' = 'hdfs://hadoop102:8020/tmp/hudi_flink/t1',  
  'table.type' = 'MERGE_ON_READ' -- 默认是 COW  
);
```

或如下写法

```
CREATE TABLE t1(  
  uuid VARCHAR(20),  
  name VARCHAR(10),  
  age INT,  
  ts TIMESTAMP(3),  
  `partition` VARCHAR(20),  
  PRIMARY KEY(uuid) NOT ENFORCED  
)  
PARTITIONED BY (`partition`)  
WITH (  
  'connector' = 'hudi',  
  'path' = 'hdfs://hadoop102:8020/tmp/hudi_flink/t1',  
  'table.type' = 'MERGE_ON_READ'  
);  
  
-- 插入数据  
INSERT INTO t1 VALUES  
  ('id1', 'Danny', 23, TIMESTAMP '1970-01-01 00:00:01', 'par1'),  
  ('id2', 'Stephen', 33, TIMESTAMP '1970-01-01 00:00:02', 'par1'),  
  ('id3', 'Julian', 53, TIMESTAMP '1970-01-01 00:00:03', 'par2'),  
  ('id4', 'Fabian', 31, TIMESTAMP '1970-01-01 00:00:04', 'par2'),  
  ('id5', 'Sophia', 18, TIMESTAMP '1970-01-01 00:00:05', 'par3'),  
  ('id6', 'Emma', 20, TIMESTAMP '1970-01-01 00:00:06', 'par3'),  
  ('id7', 'Bob', 44, TIMESTAMP '1970-01-01 00:00:07', 'par4'),  
  ('id8', 'Han', 56, TIMESTAMP '1970-01-01 00:00:08', 'par4');
```

5.2.3 查询数据

```
select * from t1;
```

5.2.4 更新数据

```
insert into t1 values  
  ('id1', 'Danny', 27, TIMESTAMP '1970-01-01 00:00:01', 'par1');
```

注意，保存模式现在是 **Append**。通常，除非是第一次创建表，否则请始终使用追加模式。现在再次查询数据将显示更新的记录。每个写操作都会生成一个用时间戳表示的新提交。查找前一次提交中相同的 `_hoodie_record_keys` 在 `_hoodie_commit_time`、`age` 字段中的变化。

5.2.5 流式插入

1) 创建测试表

```
CREATE TABLE sourceT (
  uuid varchar(20),
  name varchar(10),
  age int,
  ts timestamp(3),
  `partition` varchar(20)
) WITH (
  'connector' = 'datagen',
  'rows-per-second' = '1'
);

create table t2(
  uuid varchar(20),
  name varchar(10),
  age int,
  ts timestamp(3),
  `partition` varchar(20)
)
with (
  'connector' = 'hudi',
  'path' = '/tmp/hudi_flink/t2',
  'table.type' = 'MERGE_ON_READ'
);
```

2) 执行插入

```
insert into t2 select * from sourceT;
```

3) 查看 job



查看 HDFS 目录:

Hadoop
Overview
Datanodes
Datanode Volume Failures
Snapshot
Startup Progress
Utilities

Browse Directory

Show 25 entries
Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	atguigu	supergroup	33.67 KB		1	512 MB	.1700fe98-805b-45fb-9aa1-71261972b000_20220801195519215.log.1_1-2-0
-rw-r--r--	atguigu	supergroup	40.67 KB		1	512 MB	.d5eede81-81f2-412b-8e75-63a11d8e379e_20220801195519215.log.1_0-2-0
drwxr-xr-x	atguigu	supergroup	0 B		0	0 B	.hoodie
-rw-r--r--	atguigu	supergroup	96 B		1	128 MB	.hoodie_partition_metadata

Showing 1 to 4 of 4 entries

Previous
1
Next

Hadoop, 2019.

4) 查询结果

```
set sql-client.execution.result-mode=tableau;
select * from t2 limit 10;
```

Flink SQL> select * from t2 limit 10;
2022-08-01 11:59:41.392 INFO org.apache.hadoop.hdfs.protocol.datatransfer.sasl.SaslDataTransferClient [] - SASL encryption trust check: localHostTrusted = false, remoteHostTrusted = false

op	uuid	name	age	ts	partition
+I	9eef285101029330551513563af...	25b4676f0a2f825a44aa3a567dd...	-1663433930	2022-11:55:33.434	bdc487068dacc84ae93b4fa8835...
+I	af8f03032237681b209ce34e7bc...	7e7ccc543a661001d8df7ce85a6...	-1070929647	2022-11:56:13.433	8fdd13f6626a0638d87dd322b47...
+I	9402c200ef480988b99cc9331fb...	b016a575e74b94d4bd7b50e70cf...	550354472	2022-11:56:43.434	e95895f05ad4b8a47e913d70b7e...
+I	23549f9a49caecb0e2215b775d5...	dca8acd25112250be87a95f532...	-204207090	2022-11:56:09.433	bca4d40c79782ac2d5a9d03019ae...
+I	3cdfdc49f0075120444c8313f1...	9ebd9616916e51207cae3ba6256...	-1252412486	2022-11:56:39.433	c3e59e94e5a28838f42bd5b0457...
+I	77d54c7a4403522f7ed30631a3e...	258961e23f8f9b7215eb1095f2b...	478605783	2022-11:55:38.433	c7130f0dfce56dd19354891c7a0...
+I	982fae79a1def3c72c6f72ae0e2...	4a779ceb859444a0e78e1997ed7...	-2045079598	2022-11:56:05.433	fbcs114d2b7960e7e02219ef3d4...
+I	d7289c05d70e8429efab1b0dc23...	83ea98c08f9afbe172a1ac6b648...	-1633725472	2022-11:55:27.434	fc90bb807ee6922c1b3a74ea79e...
+I	324e051fac9fca9dec040e6aeed...	0a4818edab08267cd01da57affb...	-489862389	2022-11:57:24.434	4d007d21b273de8f860ec955cbc...
+I	c6df2ff05072aa7a3042613540b...	6196fc5a0419deeee69356e7d47...	-153681230	2022-11:57:26.434	7bdd3c505a2d71ef21d7b1473b7...

Received a total of 10 rows
Flink SQL>

5.3 IDEA 编码方式

除了用 sql-client，还可以自己编写 FlinkSQL 程序，打包提交 Flink 作业。

5.3.1 环境准备

1) 手动 install 依赖

```
mvn install:install-file -DgroupId=org.apache.hudi
-DartifactId=hudi-flink_2.12 -Dversion=0.12.0 -Dpackaging=jar
-Dfile=./hudi-flink1.13-bundle-0.12.0.jar
```

2) 创建 Maven 工程

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.atguigu.hudi</groupId>
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

```
<artifactId>flink-hudi-demo</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
  <maven.compiler.source>8</maven.compiler.source>
  <maven.compiler.target>8</maven.compiler.target>
  <flink.version>1.13.6</flink.version>
  <hudi.version>0.12.0</hudi.version>
  <java.version>1.8</java.version>
  <scala.binary.version>2.12</scala.binary.version>
  <slf4j.version>1.7.30</slf4j.version>
</properties>

<dependencies>

  <dependency>
    <groupId>org.apache.flink</groupId>
    <artifactId>flink-java</artifactId>
    <version>${flink.version}</version>
    <scope>provided</scope>    <!--不会打包到依赖中，只参与编译，不
参与运行 -->
  </dependency>
  <dependency>
    <groupId>org.apache.flink</groupId>

<artifactId>flink-streaming-java_${scala.binary.version}</artifac
tId>
    <version>${flink.version}</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>org.apache.flink</groupId>

<artifactId>flink-clients_${scala.binary.version}</artifactId>
    <version>${flink.version}</version>
    <scope>provided</scope>
  </dependency>

  <dependency>
    <groupId>org.apache.flink</groupId>

<artifactId>flink-table-planner-blink_${scala.binary.version}</ar
tifactId>
    <version>${flink.version}</version>
    <scope>provided</scope>
  </dependency>

  <!--idea 运行时也有 webui-->
  <dependency>
    <groupId>org.apache.flink</groupId>

<artifactId>flink-runtime-web_${scala.binary.version}</artifactId
>
    <version>${flink.version}</version>
    <scope>provided</scope>
  </dependency>
```

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>${slf4j.version}</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-log4j12</artifactId>
  <version>${slf4j.version}</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-to-slf4j</artifactId>
  <version>2.14.0</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>

<artifactId>flink-statebackend-rocksdb_${scala.binary.version}</a
rtifactId>
  <version>${flink.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-client</artifactId>
  <version>3.1.3</version>
  <scope>provided</scope>
</dependency>

<!--手动 install 到本地 maven 仓库-->
<dependency>
  <groupId>org.apache.hudi</groupId>
  <artifactId>hudi-flink_2.12</artifactId>
  <version>${hudi.version}</version>
  <scope>provided</scope>
</dependency>

</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-shade-plugin</artifactId>
      <version>3.2.4</version>
      <executions>
        <execution>
          <phase>package</phase>
          <goals>
```

```

        <goal>shade</goal>
    </goals>
    <configuration>
        <artifactSet>
            <excludes>

<exclude>com.google.code.findbugs:jsr305</exclude>
                <exclude>org.slf4j:*</exclude>
                <exclude>log4j:*</exclude>

<exclude>org.apache.hadoop:*</exclude>
            </excludes>
        </artifactSet>
        <filters>
            <filter>
                <!-- Do not copy the signatures in the
META-INF folder.
                Otherwise, this might cause
SecurityExceptions when using the JAR. -->
                <artifact>*</artifact>
                <excludes>
                    <exclude>META-INF/*.SF</exclude>

<exclude>META-INF/*.DSA</exclude>

<exclude>META-INF/*.RSA</exclude>
                </excludes>
            </filter>
        </filters>
        <transformers combine.children="append">
            <transformer
implementation="org.apache.maven.plugins.shade.resource.ServicesR
esourceTransformer">
                </transformer>
            </transformers>
        </configuration>
    </execution>
</executions>
</plugin>
</plugins>
</build>

</project>

```

5.3.2 编写代码

```

package com.atguigu.hudi.flink;

import
org.apache.flink.contrib.streaming.state.EmbeddedRocksDBStateBack
end;
import org.apache.flink.contrib.streaming.state.PredefinedOptions;
import org.apache.flink.streaming.api.CheckpointingMode;
import
org.apache.flink.streaming.api.environment.CheckpointConfig;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron

```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
ment;
import
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

import java.util.concurrent.TimeUnit;

public class HudiDemo {
    public static void main(String[] args) {
        StreamExecutionEnvironment env =
        StreamExecutionEnvironment.getExecutionEnvironment();

        // 设置状态后端 RocksDB
        EmbeddedRocksDBStateBackend embeddedRocksDBStateBackend =
        new EmbeddedRocksDBStateBackend(true);

        embeddedRocksDBStateBackend.setPredefinedOptions(PredefinedOptions.SPINNING_DISK_OPTIMIZED_HIGH_MEM);
        env.setStateBackend(embeddedRocksDBStateBackend);

        // checkpoint 配置
        env.enableCheckpointing(TimeUnit.SECONDS.toMillis(30),
        CheckpointingMode.EXACTLY_ONCE);
        CheckpointConfig checkpointConfig =
        env.getCheckpointConfig();

        checkpointConfig.setCheckpointStorage("hdfs://hadoop1:8020/ckps")
        ;

        checkpointConfig.setMinPauseBetweenCheckpoints(TimeUnit.SECONDS.toMillis(20));
        checkpointConfig.setTolerableCheckpointFailureNumber(5);

        checkpointConfig.setCheckpointTimeout(TimeUnit.MINUTES.toMillis(1)
        );

        checkpointConfig.enableExternalizedCheckpoints(CheckpointConfig.ExternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION);

        StreamTableEnvironment sTableEnv =
        StreamTableEnvironment.create(env);

        sTableEnv.executeSql("CREATE TABLE sourceT (\n" +
        "    uuid varchar(20),\n" +
        "    name varchar(10),\n" +
        "    age int,\n" +
        "    ts timestamp(3),\n" +
        "    `partition` varchar(20)\n" +
        ") WITH (\n" +
        "    'connector' = 'datagen',\n" +
        "    'rows-per-second' = '1'\n" +
        ")");

        sTableEnv.executeSql("create table t2(\n" +
        "    uuid varchar(20),\n" +
        "    name varchar(10),\n" +
        "    age int,\n" +
        "    ts timestamp(3),\n" +
        "    `partition` varchar(20)\n" +
        ") WITH (\n" +
        "    'connector' = 'datagen',\n" +
        "    'rows-per-second' = '1'\n" +
        ")");
    }
}
```

```

        " age int,\n" +
        " ts timestamp(3),\n" +
        " `partition` varchar(20)\n" +
        ")\n" +
        "with (\n" +
        " 'connector' = 'hudi',\n" +
        " 'path' = '/tmp/hudi_flink/t2',\n" +
        " 'table.type' = 'MERGE_ON_READ'\n" +
        ")\n");

sTableEnv.executeSql("insert into t2 select * from sourceT");
    }
}

```

5.3.3 提交运行

将代码打成 jar 包，上传到目录 myjars，执行提交命令：

```

bin/flink run -t yarn-per-job \
-c com.atguigu.hudi.flink.HudiDemo \
./myjars/flink-hudi-demo-1.0-SNAPSHOT.jar

```

5.4 类型映射

Flink SQL Type	Hudi Type	Avro logical type
CHAR / VARCHAR / STRING	string	
BOOLEAN	boolean	
BINARY / VARBINARY	bytes	
DECIMAL	fixed	decimal
TINYINT	int	
SMALLINT	int	
INT	int	
BIGINT	long	
FLOAT	float	
DOUBLE	double	
DATE	int	date
TIME	int	time-millis
TIMESTAMP	long	timestamp-millis
ARRAY	array	

MAP (key must be string/char/varchar type)	map	
MULTISET (element must be string/char/varchar type)	map	
ROW	record	

5.5 核心参数设置

Flink 可配参数: https://hudi.apache.org/docs/configurations#FLINK_SQL

5.5.1 去重参数

通过如下语法设置主键:

```
-- 设置单个主键
create table hoodie_table (
  f0 int primary key not enforced,
  f1 varchar(20),
  ...
) with (
  'connector' = 'hudi',
  ...
)

-- 设置联合主键
create table hoodie_table (
  f0 int,
  f1 varchar(20),
  ...
  primary key(f0, f1) not enforced
) with (
  'connector' = 'hudi',
  ...
)
```

名称	说明	默认值	备注
hoodie.datasource.write.recordkey.field	主键字段	--	支持主键语法 PRIMARY KEY 设置, 支持逗号分隔的多个字段
precombine.field (0.13.0 之前版本为 write.precombine.field)	去重时间 字段	--	record 合并的时候会按照该字段排序, 选值较大的 record 为合并结果; 不指定则为处理序: 选择后到的

			record
--	--	--	--------

5.5.2 并发参数

1) 参数说明

名称	说明	默认值	备注
write.tasks	writer 的并发，每个 writer 顺序写 1~N 个 buckets	4	增加并发对小文件个数没影响
write.bucket_assign.tasks	bucket assigner 的并发	Flink 的并行度	增加并发同时增加了并发的写的 bucekt 数，也就变相增加了小文件(小 bucket) 数
write.index_bootstrap.tasks	Index bootstrap 算子的并发，增加并发可以加快 bootstrap 阶段的效率，bootstrap 阶段会阻塞 checkpoint，因此需要设置多一些的 checkpoint 失败容忍次数	Flink 的并行度	只在 index.bootstrap.enabled 为 true 时生效
read.tasks	读算子的并发 (batch 和 stream)	4	
compaction.tasks	online compaction 算子的并发	writer 的并发	online compaction 比较耗费资源，建议走 offline compaction

2) 案例演示

可以 flink 建表时在 with 中指定，或 Hints 临时指定参数的方式：在需要调整的表名后面加上 `/*+ OPTIONS() */`

```
insert into t2 /*+
OPTIONS('write.tasks'='2','write.bucket_assign.tasks'='3','compac
tion.tasks'='4') */
select * from sourceT;
```

5.5.3 压缩参数

1) 参数说明

在线压缩的参数，通过设置 `compaction.async.enabled=false` 关闭在线压缩执行，但是调度 `compaction.schedule.enabled` 仍然建议开启，之后通过离线压缩直接执行 在线压缩任务阶段性调度的压缩 plan。

名称	说明	默认值	备注
compaction.schedule.enabled	是否阶段性生成压缩 plan	true	建议开启，即使 compaction.async.enabled 关闭的情况下

compaction.async.enabled	是否开启异步压缩	true	通过关闭此参数关闭在线压缩
compaction.tasks	压缩 task 并发	4	
compaction.trigger.strategy	压缩策略	num_commits	支持四种策略：num_commits、time_elapsed、num_and_time、num_or_time
compaction.delta_commits	默认策略，5 个 commits 压缩一次	5	
compaction.delta_seconds		3600	
compaction.max_memory	压缩去重的 hash map 可用内存	100 (MB)	资源够用的话建议调整到 1GB
compaction.target_io	每个压缩 plan 的 IO 上限,默认 5GB	500 (GB)	

2) 案例演示

```
CREATE TABLE t3(
  uuid VARCHAR(20) PRIMARY KEY NOT ENFORCED,
  name VARCHAR(10),
  age INT,
  ts TIMESTAMP(3),
  `partition` VARCHAR(20)
)
WITH (
  'connector' = 'hudi',
  'path' = 'hdfs://hadoop102:8020/tmp/hudi_flink/t3',
  'compaction.async.enabled' = 'true',
  'compaction.tasks' = '1',
  'compaction.schedule.enabled' = 'true',
  'compaction.trigger.strategy' = 'num_commits',
  'compaction.delta_commits' = '2',
  'table.type' = 'MERGE_ON_READ'
);

set table.dynamic-table-options.enabled=true;
insert into t3
select * from sourceT/*+ OPTIONS('rows-per-second' = '5')*/;
```

注意：如果没有按照 5.2.1 中 yarn-session 模式解决 hadoop 依赖冲突问题，那么无法 compaction 生成 parquet 文件，报错很隐晦，在 Exception 中看不到，要搜索 TaskManager 中关于 compaction 才能看到报错。

```
2022-11-07 07:06:58 ERROR org.apache.hudi.sink.compact.CompactFunction [TaskManager-1] - Executor executes action [Execute compaction for
instant 20221003185658769 from task 0] error
java.lang.NoClassDefFoundError: org/apache/hadoop/mapreduce/lib/output/FileOutputFormat
    at java.lang.ClassLoader.defineClass1(Native Method) ~[?:1.8.0_212]
    at java.lang.ClassLoader.defineClass(ClassLoader.java:763) ~[?:1.8.0_212]
    at java.security.SecureClassLoader.defineClass(SecureClassLoader.java:142) ~[?:1.8.0_212]
    at java.net.URLClassLoader.defineClass(URLClassLoader.java:468) ~[?:1.8.0_212]
    at java.net.URLClassLoader.access$100(URLClassLoader.java:74) ~[?:1.8.0_212]
    at java.net.URLClassLoader$1.run(URLClassLoader.java:369) ~[?:1.8.0_212]
    at java.net.URLClassLoader$1.run(URLClassLoader.java:363) ~[?:1.8.0_212]
    at java.security.AccessController.doPrivileged(Native Method) ~[?:1.8.0_212]
    at java.net.URLClassLoader.findClass(URLClassLoader.java:362) ~[?:1.8.0_212]
```

5.5.4 文件大小

1) 参数说明

Hudi 会自管理文件大小，避免向查询引擎暴露小文件，其中自动处理文件大小起很大作用。在进行 insert/upsert 操作时，Hudi 可以将文件大小维护在一个指定文件大小。

目前只有 log 文件的写入大小可以做到精确控制，parquet 文件大小按照估算值。

名称	说明	默认值	备注
hoodie.parquet.max.file.size	最大可写入的 parquet 文件大小	120 * 1024 * 1024 默认 120MB (单位 byte)	超过该大小切新的 file group
hoodie.logfile.to.parquet.compression.ratio	log 文件大小转 parquet 的比率	0.35	hoodie 统一依据 parquet 大小来评估小文件策略
hoodie.parquet.small.file.limit	在写入时，hudi 会尝试先追加写已存小文件，该参数设置了小文件的大小阈值，小于该参数的文件被认为是小文件	104857600 默认 100MB (单位 byte)	大于 100MB，小于 120MB 的文件会被忽略，避免写过度放大
hoodie.copyonwrite.record.size.estimate	预估的 record 大小，hoodie 会依据历史的 commits 动态估算 record 的大小，但是前提是之前有单次写入超过 hoodie.parquet.small.file.limit 大小，在未达到这个大小时会使用这个参数	1024 默认 1KB (单位 byte)	如果作业流量比较小，可以设置下这个参数
hoodie.logfile.max.size	LogFile 最大大小。这是在将 Log 滚转到下一个版本之前允许的最大大小。	1073741824 默认 1GB (单位 byte)	

2) 案例演示

```
CREATE TABLE t4 (
  uuid VARCHAR(20) PRIMARY KEY NOT ENFORCED,
  name VARCHAR(10),
  age INT,
  ts TIMESTAMP(3),
  `partition` VARCHAR(20)
)
WITH (
  'connector' = 'hudi',
  'path' = 'hdfs://hadoop1:8020/tmp/hudi_flink/t4',
  'compaction.tasks' = '1',
  'hoodie.parquet.max.file.size' = '10000',
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
'hoodie.parquet.small.file.limit'='5000',
'table.type' = 'MERGE_ON_READ'
);

set table.dynamic-table-options.enabled=true;
insert into t4
select * from sourceT /*+ OPTIONS('rows-per-second' = '5')*/;
```

5.5.5 Hadoop 参数

从 0.12.0 开始支持, 如果有跨集群提交执行的需求, 可以通过 sql 的 ddl 指定 per-job 级别的 hadoop 配置

名称	说明	默认值	备注
hadoop.\${you option key}	通过 hadoop. 前缀指定 hadoop 配置项	--	支持同时指定多个 hadoop 配置项

5.6 内存优化

5.6.1 内存参数

名称	说明	默认值	备注
write.task.max.size	一个 write task 的最大可用内存	1024	当前预留给 write buffer 的内存为 write.task.max.size -compaction.max_memory 当 write task 的内存 buffer 达到阈值后会将内存里最大的 buffer flush 出去
write.batch.size	Flink 的写 task 为了提高写数据效率, 会按照写 bucket 提前 buffer 数据, 每个 bucket 的数据在内存达到阈值之前会一直 cache 在内存中, 当阈值达到会把数据 buffer 传递给 hoodie 的 writer 执行写操作	256	一般不用设置, 保持默认值就好
write.log_block.size	hoodie 的 log writer 在收到 write task 的数据后不会马上 flush 数据, writer 是以 LogBlock 为单位往磁盘刷数据的, 在 LogBlock 攒够之前 records 会以序列化字节的形式 buffer 在 writer 内部	128	一般不用设置, 保持默认值就好
write.merge.max_memory	hoodie 在 COW 写操作的时候, 会有增量数据和 base file	100	一般不用设置, 保持默认值就好

	数据 merge 的过程，增量的数据会缓存在内存的 map 结构里，这个 map 是可 spill 的，这个参数控制了 map 可以使用的堆内存大小		
compaction.max_memory	同 write.merge.max_memory: 100MB 类似，只是发生在压缩时。	100	如果是 online compaction，资源充足时可以开大些，比如 1GB

5.6.1 MOR

- (1) state backend 换成 rocksdb (默认的 in-memory state-backend 非常吃内存)
- (2) 内存够的话，compaction.max_memory 调大些 (默认是 100MB 可以调到 1GB)
- (3) 关注 TM 分配给每个 write task 的内存，保证每个 write task 能够分配到 write.task.max.size 所配置的大小，比如 TM 的内存是 4GB 跑了 2 个 StreamWriteFunction 那每个 write function 能分到 2GB，尽量预留一些 buffer，因为网络 buffer，TM 上其他类型 task (比如 BucketAssignFunction 也会吃些内存)
- (4) 需要关注 compaction 的内存变化，compaction.max_memory 控制了每个 compaction task 读 log 时可以利用的内存大小，compaction.tasks 控制了 compaction task 的并发

注意: write.task.max.size - compaction.max_memory 是预留给每个 write task 的内存 buffer

5.6.2 COW

- (1) state backend 换成 rocksdb (默认的 in-memory state-backend 非常吃内存)。
- (2) write.task.max.size 和 write.merge.max_memory 同时调大 (默认是 1GB 和 100MB 可以调到 2GB 和 1GB)。
- (3) 关注 TM 分配给每个 write task 的内存，保证每个 write task 能够分配到 write.task.max.size 所配置的大小，比如 TM 的内存是 4GB 跑了 2 个 StreamWriteFunction 那每个 write function 能分到 2GB，尽量预留一些 buffer，因为网络 buffer，TM 上其他类型 task (比如 BucketAssignFunction 也会吃些内存)。

注意: write.task.max.size - write.merge.max_memory 是预留给每个 write task 的内存 buffer。

5.7 读取方式

5.7.1 流读 (Streaming Query)

当前表默认是快照读取，即读取最新的全量快照数据并一次性返回。通过参数 `read.streaming.enabled` 参数开启流读模式，通过 `read.start-commit` 参数指定起始消费位置，支持指定 `earliest` 从最早消费。

1) WITH 参数

名称	Required	默认值	说明
<code>read.streaming.enabled</code>	false	false	设置 true 开启流读模式
<code>read.start-commit</code>	false	最新 commit	指定 'yyyyMMddHHmmss' 格式的起始 commit (闭区间)
<code>read.streaming.skip_compaction</code>	false	false	流读时是否跳过 compaction 的 commits，跳过 compaction 有两个用途： 1) 避免 upsert 语义下重复消费 (compaction 的 instant 为重复数据，如果不跳过，有小概率会重复消费) 2) changelog 模式下保证语义正确性 0.11 开始，以上两个问题已经通过保留 compaction 的 instant time 修复
<code>clean.retain_commits</code>	false	10	cleaner 最多保留的历史 commits 数，大于此数量的历史 commits 会被清理掉，changelog 模式下，这个参数可以控制 changelog 的保留时间，例如 checkpoint 周期为 5 分钟一次，默认最少保留 50 分钟的时间。

注意：当参数 `read.streaming.skip_compaction` 打开并且 streaming reader 消费落后于 `clean.retain_commits` 数时，流读可能会丢失数据。从 0.11 开始，compaction 不会再变更 record 的 instant time，因此理论上数据不会再重复消费，但是还是会重复读取并丢弃，因此额外的开销还是无法避免，对性能有要求的话还是可以开启此参数。

```
CREATE TABLE t5 (
  uuid VARCHAR(20) PRIMARY KEY NOT ENFORCED,
  name VARCHAR(10),
  age INT,
  ts TIMESTAMP(3),
  `partition` VARCHAR(20)
) WITH (
  'connector' = 'hudi',
  'path' = 'hdfs://hadoop102:8020/tmp/hudi_flink/t5',
  'table.type' = 'MERGE_ON_READ',
  'read.streaming.enabled' = 'true',
  'read.streaming.check-interval' = '4'
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
);

insert into t5 select * from sourceT;

select * from t5;
```

5.7.2 增量读取（Incremental Query）

从 0.10.0 开始支持。

如果有增量读取 batch 数据的需求，增量读取包含三种场景。

- (1) Stream 增量消费，通过参数 `read.start-commit` 指定起始消费位置；
- (2) Batch 增量消费，通过参数 `read.start-commit` 指定起始消费位置，通过参数 `read.end-commit` 指定结束消费位置，区间为闭区间，即包含起始、结束的 commit
- (3) TimeTravel: Batch 消费某个时间点的数据：通过参数 `read.end-commit` 指定结束消费位置即可（由于起始位置默认从最新，所以无需重复声明）

WITH 参数

名称	Required	默认值	说明
<code>read.start-commit</code>	false	默认从最新 commit	支持 earliest 从最早消费
<code>read.end-commit</code>	false	默认到最新 commit	

5.8 限流

如果将全量数据(百亿数量级) 和增量先同步到 kafka，再通过 flink 流式消费的方式将库表数据直接导成 hoodie 表，因为直接消费全量部分数据：量大（吞吐高）、乱序严重（写入的 partition 随机），会导致写入性能退化，出现吞吐毛刺，这时候可以开启限速参数，保证流量平稳写入。

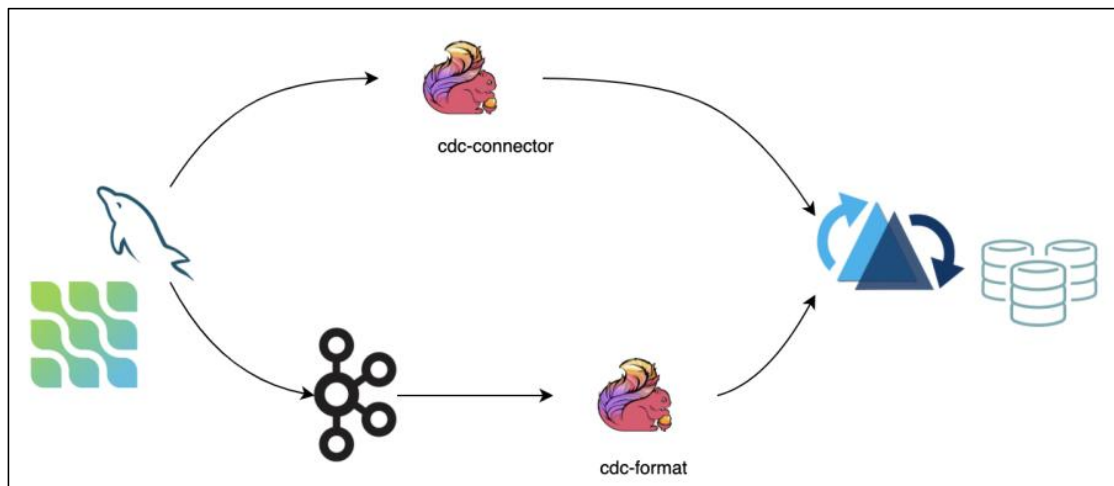
WITH 参数

名称	Required	默认值	说明
<code>write.rate.limit</code>	false	0	默认关闭限速

5.9 写入方式

5.9.1 CDC 数据同步

CDC 数据保存了完整的数据库变更，当前可通过两种途径将数据导入 hudi:



第一种：通过 cdc-connector 直接对接 DB 的 binlog 将数据导入 hudi，优点是不依赖消息队列，缺点是对 db server 造成压力。

第二种：对接 cdc format 消费 kafka 数据导入 hudi，优点是可扩展性强，缺点是依赖 kafka。

注意：如果上游数据无法保证顺序，需要指定 write.precombine.field 字段。

1) 准备 MySQL 表

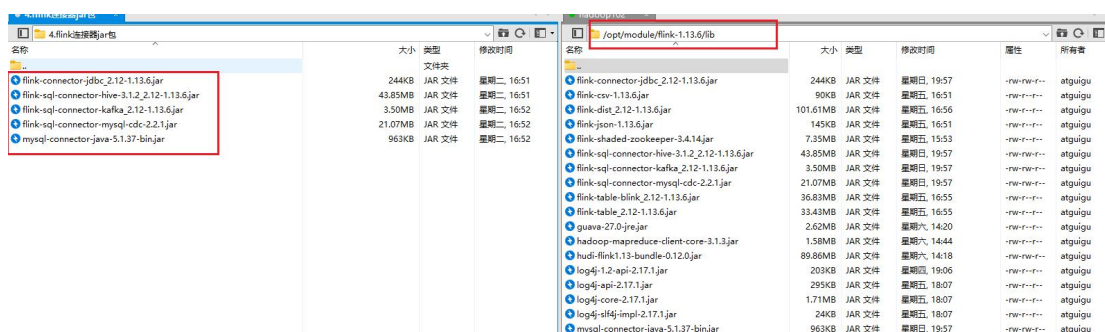
(1) MySQL 开启 binlog

(2) 建表

```
create table stu3 (
  id int unsigned auto_increment primary key COMMENT '自增 id',
  name varchar(20) not null comment '学生名字',
  school varchar(20) not null comment '学校名字',
  nickname varchar(20) not null comment '学生小名',
  age int not null comment '学生年龄',
  class_num int not null comment '班级人数',
  phone bigint not null comment '电话号码',
  email varchar(64) comment '家庭网络邮箱',
  ip varchar(32) comment 'IP 地址'
) engine=InnoDB default charset=utf8;
```

2) 添加 flinkCDC 的依赖

上传 flink 连接器依赖到 flink 的 lib 目录下：



3) flink 读取 mysql binlog 并写入 kafka

(1) 创建 MySQL 表

```
create table stu3_binlog(
  id bigint not null,
  name string,
  school string,
  nickname string,
  age int not null,
  class_num int not null,
  phone bigint not null,
  email string,
  ip string,
  primary key (id) not enforced
) with (
  'connector' = 'mysql-cdc',
  'hostname' = 'hadoop102',
  'port' = '3306',
  'username' = 'root',
  'password' = '000000',
  'database-name' = 'gmall',
  'table-name' = 'stu3'
);
```

(2) 创建 Kafka 表

```
create table stu3_binlog_sink_kafka(
  id bigint not null,
  name string,
  school string,
  nickname string,
  age int not null,
  class_num int not null,
  phone bigint not null,
  email string,
  ip string,
  primary key (id) not enforced
) with (
  'connector' = 'upsert-kafka',
  'topic' = 'cdc_mysql_stu3_sink',
  'properties.zookeeper.connect' = 'hadoop102:2181',
  'properties.bootstrap.servers' = 'hadoop102:9092',
  'key.format' = 'json',
  'value.format' = 'json'
);
```

(3) 将 mysql binlog 日志写入 kafka

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
insert into stu3_binlog_sink_kafka
select * from stu3_binlog;
```

3) flink 读取 kafka 数据并写入 hudi 数据湖

(1) 创建 kafka 源表

```
create table stu3_binlog_source_kafka (
  id bigint not null,
  name string,
  school string,
  nickname string,
  age int not null,
  class_num int not null,
  phone bigint not null,
  email string,
  ip string
) with (
  'connector' = 'kafka',
  'topic' = 'cdc_mysql_stu3_sink',
  'properties.bootstrap.servers' = 'hadoop102:9092',
  'format' = 'json',
  'scan.startup.mode' = 'earliest-offset',
  'properties.group.id' = 'testGroup'
);
```

(2) 创建 hudi 目标表

```
create table stu3_binlog_sink_hudi (
  id bigint not null,
  name string,
  `school` string,
  nickname string,
  age int not null,
  class_num int not null,
  phone bigint not null,
  email string,
  ip string,
  primary key (id) not enforced
)
partitioned by (`school`)
with (
  'connector' = 'hudi',
  'path' =
  'hdfs://hadoop102:8020/tmp/hudi_flink/stu3_binlog_sink_hudi',
  'table.type' = 'MERGE_ON_READ',
  'write.option' = 'insert',
  'write.precombine.field' = 'school'
);
```

(3) 将 kafka 数据写入到 hudi 中

```
insert into stu3_binlog_sink_hudi
select * from stu3_binlog_source_kafka;
```

5.9.2 离线批量导入

如果存量数据来源于其他数据源,可以使用批量导入功能,快速将存量数据导成 Hoodie 表格式。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载,可百度访问: [尚硅谷官网](#)

1) 原理

(1) 批量导入省去了 avro 的序列化以及数据的 merge 过程, 后续不会再有去重操作, 数据的唯一性需要自己来保证。

(2) bulk_insert 需要在 **Batch Execution Mode** 下执行更高效, Batch 模式默认会按照 partition path 排序输入消息再写入 Hoodie, 避免 file handle 频繁切换导致性能下降。

```
SET execution.runtime-mode = batch;
SET execution.checkpointing.interval = 0;
```

(3) bulk_insert write task 的并发通过参数 write.tasks 指定, 并发的数量会影响到小文件的数量, 理论上, bulk_insert write task 的并发数就是划分的 bucket 数, 当然每个 bucket 在写到文件大小上限 (parquet 120 MB) 的时候会 roll over 到新的文件句柄, 所以最后: 写文件数量 $\geq$ bulk_insert write task 数。

2) WITH 参数

名称	Required	默认值	说明
write.operation	true	upsert	配置 bulk_insert 开启该功能
write.tasks	false	4	bulk_insert 写 task 的并发, 最后的文件数 $\geq$ write.tasks
write.bulk_insert.shuffle_by_partition write.bulk_insert.shuffle_input (从 0.11 开始)	false	true	是否将数据按照 partition 字段 shuffle 再通过 write task 写入, 开启该参数将减少小文件的数量 但是可能有数据倾斜风险
write.bulk_insert.sort_by_partition write.bulk_insert.sort_input (从 0.11 开始)	false	true	是否将数据按照 partition 字段排序再写入, 当一个 write task 写多个 partition, 开启可以减少小文件数量
write.sort.memory		128	sort 算子的可用 managed memory (单位 MB)

3) 案例

(1) Flink SQL client 创建 hudi 表

```
create table stu3_bulk_insert_hudi(
  id bigint not null,
  name string,
  `school` string,
  nickname string,
  age int not null,
  class_num int not null,
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```

phone bigint not null,
email string,
ip string,
primary key (id) not enforced
)
partitioned by (`school`)
with (
  'connector' = 'hudi',
  'path'
  'hdfs://hadoop102:8020/tmp/hudi_flink/stu3_bulk_insert_hudi',
  'table.type' = 'MERGE_ON_READ',
  'write.option' = 'bulk_insert',
  'write.precombine.field' = 'school'
);

```

(3) Flink SQL client 执行 mysql 数据插入到 hudi 中

```
insert into stu3_bulk_insert_hudi select * from stu3_binlog;
```

5.9.3 全量接增量

如果已经有全量的离线 Hoodie 表，需要接上实时写入，并且保证数据不重复，可以开启 index bootstrap 功能。

如果觉得流程冗长，可以在写入全量数据的时候资源调大直接走流模式写，全量走完接新数据再将资源调小（或者开启限流功能）。

WITH 参数

名称	Required	默认值	说明
index.bootstrap.enabled	true	false	开启索引加载，会将已存表的最新数据一次性加载到 state 中
index.partition.regex	false	*	设置正则表达式进行分区筛选，默认为加载全部分区

使用流程

- (1) CREATE TABLE 创建和 Hoodie 表对应的语句，注意 table type 要正确
- (2) 设置 index.bootstrap.enabled = true 开启索引加载功能
- (3) 重启任务将 index.bootstrap.enabled 关闭，参数配置到合适的大小，如果 RowDataToHoodieFunction 和 BootstrapFunction 并发不同，可以重启避免 shuffle

说明：

- (1) 索引加载为并发加载，根据数据量大小加载时间不同，可以在 log 中搜索 finish loading the index under partition 和 Load records from file 日志来观察索引加载的进度

5.10 写入模式

5.10.1 Changelog 模式

如果希望 Hoodie 保留消息的所有变更（I-U/U/D），之后接上 Flink 引擎的有状态计算实现全链路近实时数仓生产（增量计算），Hoodie 的 MOR 表通过行存原生支持保留消息的所有变更（format 层面的集成），通过流读 MOR 表可以消费到所有的变更记录。

1) WITH 参数

名称	Required	默认值	说明
changelog.enabled	false	false	默认是关闭状态，即 UPSERT 语义，所有的消息仅保证最后一条合并消息，中间的变更可能会被 merge 掉；改成 true 支持消费所有变更。

批（快照）读仍然会合并所有的中间结果，不管 format 是否已存储中间状态。

开启 changelog.enabled 参数后，中间的变更也只是 Best Effort: 异步的压缩任务会将中间变更合并成 1 条，所以如果流读消费不够及时，被压缩后只能读到最后一条记录。当然，通过调整压缩的 buffer 时间可以预留一定的时间 buffer 给 reader，比如调整压缩的两个参数：

- compaction.delta_commits:5
- compaction.delta_seconds: 3600。

说明：

Changelog 模式开启流读的话，要在 sql-client 里面设置参数：

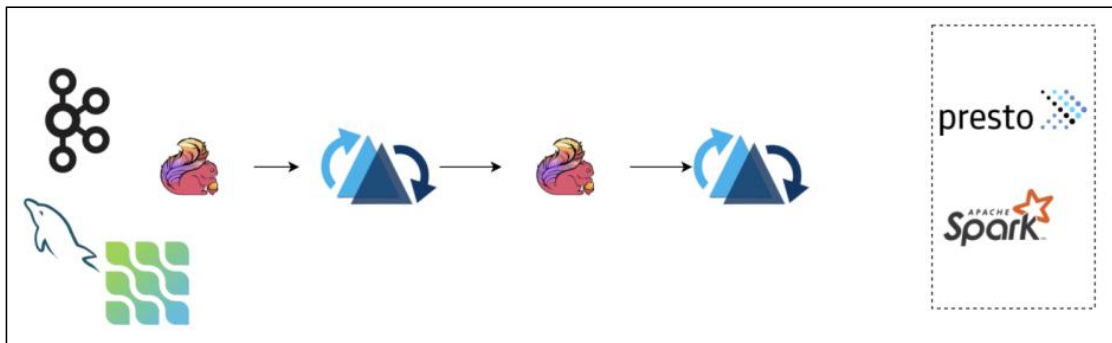
```
set sql-client.execution.result-mode=tableau;
或者
set sql-client.execution.result-mode=changelog;
```

否则中间结果在读的时候会被直接合并。（参考：<https://nightlies.apache.org/flink/flink-docs-release-1.13/docs/dev/table/sqlclient/#running-sql-queries>），

2) 流读 changelog

仅在 0.10.0 支持，本 feature 为实验性。

开启 changelog 模式后，hudi 会保留一段时间的 changelog 供下游 consumer 消费，我们可以通过流读 ODS 层 changelog 接上 ETL 逻辑写入到 DWD 层，如下图的 pipeline：



流读的时候我们要注意 changelog 有可能会被 compaction 合并掉，中间记录会消除，可能会影响计算结果，需要关注 sql-client 的属性（result-mode）同上。

3) 案例演示

(1) 使用 changelog

```
set sql-client.execution.result-mode=tableau;

CREATE TABLE t6(
  id int,
  ts int,
  primary key (id) not enforced
) WITH (
  'connector' = 'hudi',
  'path' = 'hdfs://hadoop102:8020/tmp/hudi_flink/t6',
  'table.type' = 'MERGE_ON_READ',
  'read.streaming.enabled' = 'true',
  'read.streaming.check-interval' = '4',
  'changelog.enabled' = 'true'
);

insert into t6 values (1,1);
insert into t6 values (1,2);
insert into t6 values (1,3);
insert into t6 values (1,4);
insert into t6 values (1,5);
insert into t6 values (1,6);

set table.dynamic-table-options.enabled=true;
select * from t6/*+ OPTIONS('read.start-commit'='earliest')*/;
select      count(*)      from      t6/*+
OPTIONS('read.start-commit'='earliest')*/;
```

(2) 不使用 changelog

```
CREATE TABLE t6_v(
  id int,
  ts int,
  primary key (id) not enforced
) WITH (
  'connector' = 'hudi',
  'path' = 'hdfs://hadoop102:8020/tmp/hudi_flink/t6_v',
  'table.type' = 'MERGE_ON_READ',
```

```
'read.streaming.enabled' = 'true',
'read.streaming.check-interval' = '4'
);

insert into t6_v values (1,1);
insert into t6_v values (1,2);
insert into t6_v values (1,3);
insert into t6_v values (1,4);
insert into t6_v values (1,5);
insert into t6_v values (1,6);

select * from t6_v/*+ OPTIONS('read.start-commit'='earliest')*/;
select          count(*)          from          t6_v/*+
OPTIONS('read.start-commit'='earliest')*/;
```

5.10.2 Append 模式

从 0.10 开始支持

对于 INSERT 模式：

- MOR 默认会 apply 小文件策略：会追加写 avro log 文件
- COW 每次直接写新的 parquet 文件，没有小文件策略

Hudi 支持丰富的 Clustering 策略，优化 INSERT 模式下的小文件问题：

1) Inline Clustering

只有 Copy On Write 表支持该模式

名称	Required	默认值	说明
write.insert.cluster	false	false	是否在写入时合并小文件，COW 表默认 insert 写不合并小文件，开启该参数后，每次写入会优先合并之前的小文件（不会去重），吞吐会受影响

2) Async Clustering

从 0.12 开始支持

(1) WITH 参数

名称	Required	默认值	说明
clustering.schedule.enabled	false	false	是否在写入时定时异步调度 clustering plan，默认关闭
clustering.delta_commits	false	4	调度 clustering plan 的间隔 commits，clustering.schedule.enabled 为 true 时生效
clustering.async.enabled	false	false	是否异步执行 clustering plan，默认关闭

clustering.tasks	false	4	Clustering task 执行并发
clustering.plan.strategy.target.file.max.bytes	false	1024 * 1024 * 1024	Clustering 单文件目标大小，默认 1GB
clustering.plan.strategy.small.file.limit	false	600	小于该大小的文件才会参与 clustering，默认 600MB
clustering.plan.strategy.sort.columns	false	N/A	支持指定特殊的排序字段
clustering.plan.partition.filter.mode	false	NONE	支持 NONE：不做限制 RECENT_DAYS：按时间（天）回溯 SELECTED_PARTITION S：指定固定的 partition
clustering.plan.strategy.daybased.lookback.partitions	false	2	RECENT_DAYS 生效，默认 2 天

（2）Clustering Plan Strategy

支持定制化的 clustering 策略。

名称	Required	默认值	说明
clustering.plan.partition.filter.mode	false	NONE	支持 - NONE：不做限制 - RECENT_DAYS：按时间（天）回溯 - SELECTED_PARTITIONS：指定固定的 partition
clustering.plan.strategy.daybased.lookback.partitions	false	2	RECENT_DAYS 生效，默认 2 天
clustering.plan.strategy.cluster.begin.partition	false	N/A	SELECTED_PARTITIONS 生效，指定开始 partition(inclusive)
clustering.plan.strategy.cluster.end.partition	false	N/A	SELECTED_PARTITIONS 生效，指定结束 partition(include)
clustering.plan.strategy.partition.regex.pattern	false	N/A	正则表达式过滤 partitions
clustering.plan.strategy.partition.selected	false	N/A	显示指定目标 partitions，支持逗号，

		分割多个 partition
--	--	----------------

5.11 Bucket 索引

从 0.11 开始支持

默认的 flink 流式写入使用 state 存储索引信息：primary key 到 fileId 的映射关系。当数据量比较大的时候，state 的存储开销可能成为瓶颈，bucket 索引通过固定的 hash 策略，将相同 key 的数据分配到同一个 fileGroup 中，避免了索引的存储和查询开销。

1) WITH 参数

名称	Required	默认值	说明
index.type	false	FLINK_STATE	设置 BUCKET 开启 Bucket 索引功能
hoodie.bucket.index.hash.field	false	主键	可以设置成主键的子集
hoodie.bucket.index.num.buckets	false	4	默认每个 partition 的 bucket 数，当前设置后则不可再变更。

2) 和 state 索引的对比：

- (1) bucket index 没有 state 的存储计算开销，性能较好
- (2) bucket index 无法扩 buckets，state index 则可以依据文件的大小动态扩容
- (3) bucket index 不支持跨 partition 的变更(如果输入是 cdc 流则没有这个限制)，state index 没有限制

5.12 Hudi Catalog

从 0.12.0 开始支持，通过 catalog 可以管理 flink 创建的表，避免重复建表操作，另外 hms 模式的 catalog 支持自动补全 hive 同步参数。

DFS 模式 Catalog SQL 样例：

```
CREATE CATALOG hoodie_catalog
WITH (
  'type'='hudi',
  'catalog.path' = '${catalog 的默认路径}',
  'mode'='dfs'
);
```

Hms 模式 Catalog SQL 样例：

```
CREATE CATALOG hoodie_catalog
WITH (
  'type'='hudi',
  'catalog.path' = '${catalog 的默认路径}',
  'hive.conf.dir' = '${hive-site.xml 所在的目录}',
  'mode'='hms' -- 支持 'dfs' 模式通过文件系统管理表属性
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

```
);
```

1) WITH 参数

名称	Required	默认值	说明
catalog.path	true	--	默认的 catalog 根路径，用作表路径的自动推导，默认的表路径: \${catalog.path}/\${db_name}/\${table_name}
default-database	false	default	默认的 database 名
hive.conf.dir	false	--	hive-site.xml 所在的目录，只在 hms 模式下生效
mode	false	dfs	支持 hms 模式通过 hive 管理元数据
table.external	false	false	是否创建外部表，只在 hms 模式下生效

2) 使用 dfs 方式

(1) 创建 sql-client 初始化 sql 文件

```
vim /opt/module/flink-1.13.6/conf/sql-client-init.sql
```

```
CREATE CATALOG hoodie_catalog
WITH (
  'type'='hudi',
  'catalog.path' = '/tmp/hudi_catalog',
  'mode'='dfs'
);
```

```
USE CATALOG hoodie_catalog;
```

(2) 指定 sql-client 启动时加载 sql 文件

```
hadoop fs -mkdir /tmp/hudi_catalog
```

```
bin/sql-client.sh    embedded    -i    conf/sql-client-init.sql    -s
yarn-session
```

(3) 建库建表插入

```
create database test;
use test;

create table t2(
  uuid varchar(20),
  name varchar(10),
  age int,
  ts timestamp(3),
  `partition` varchar(20),
  primary key (uuid) not enforced
)
with (
  'connector' = 'hudi',
  'path' = '/tmp/hudi_catalog/test/t2',
  'table.type' = 'MERGE_ON_READ'
);

insert into t2 values('1','zs',18,TIMESTAMP '1970-01-01
00:00:01','a');
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

(4) 退出 sql-client, 重新进入, 表信息还在

```
use test;
show tables;
select * from t2;
```

5.13 离线 Compaction

MOR 表的 compaction 默认是自动打开的, 策略是 5 个 commits 执行一次压缩。因为压缩操作比较耗费内存, 和写流程放在同一个 pipeline, 在数据量比较大的时候 (10w+/s qps), 容易干扰写流程, 此时采用离线定时任务的方式执行 compaction 任务更稳定。

5.13.1 设置参数

- compaction.async.enabled 为 false, 关闭在线 compaction。
- compaction.schedule.enabled 仍然保持开启, 由写任务阶段性触发压缩 plan。

Compaction 过程

5.13.2 原理

一个 compaction 的任务的执行包括两部分:

- schedule 压缩 plan
该过程推荐由写任务定时触发, 写参数 compaction.schedule.enabled 默认开启
- 执行对应的压缩 plan

5.13.3 使用方式

1) 执行命令

离线 compaction 需要手动执行 Java 程序, 程序入口:

- hudi-flink1.13-bundle-0.12.0.jar
- org.apache.hudi.sink.compact.HoodieFlinkCompactor

```
// 命令行的方式
./bin/flink run -c
org.apache.hudi.sink.compact.HoodieFlinkCompactor
lib/hudi-flink1.13-bundle-0.12.0.jar --path hdfs://xxx:8020/table
```

2) 参数配置

参数名	required	默认值	备注
--path	true	--	目标表的路径
--compaction-tasks	false	-1	压缩 task 的并发, 默认是待压缩 file group 的数量

--compaction-max-memory	false	100 （单位 MB）	压缩时 log 数据的索引 map，默认 100MB，内存足够可以开大些
--schedule	false	false	是否要执行 schedule compaction 的操作，当写流程还在持续写入表数据的时候，开启这个参数有丢失查询数据的风险，所以开启该参数一定要保证当前没有任务往表里写数据，写任务的 compaction plan 默认是一直 schedule 的，除非手动关闭（默认 5 个 commits 一次压缩）
--seq	false	LIFO	执行压缩任务的顺序，默认是从最新的压缩 plan 开始执行，可选值： LIFO: 从最新的 plan 开始执行； FIFO: 从最老的 plan 开始执行
--service	false	false	是否开启 service 模式，service 模式会打开常驻进程，一直监听压缩任务并提交到集群执行（从 0.11 开始执行）
--min-compaction-interval-seconds	false	600 （单位 秒）	service 模式下的执行间隔，默认 10 分钟

3) 案例演示

(1) 创建表，关闭在线压缩

```
create table t7(
  id int,
  ts int,
  primary key (id) not enforced
)
with (
  'connector' = 'hudi',
  'path' = '/tmp/hudi_catalog/test/t7',
  'compaction.async.enabled' = 'false',
  'compaction.schedule.enabled' = 'true',
  'table.type' = 'MERGE_ON_READ'
);

insert into t7 values(1,1);
insert into t7 values(2,2);
insert into t7 values(3,3);
insert into t7 values(4,4);
insert into t7 values(5,5);

// 命令行的方式
./bin/flink run -c
org.apache.hudi.sink.compact.HoodieFlinkCompactor
lib/hudi-flink1.13-bundle-0.12.0.jar --path
hdfs://hadoop102:8020/tmp/hudi_catalog/test/t7
```

5.14 离线 Clustering

异步的 clustering 相对于 online 的 async clustering 资源隔离，从而更加稳定。

5.14.1 设置参数

- clustering.async.enabled 为 false，关闭在线 clustering。
- clustering.schedule.enabled 仍然保持开启，由写任务阶段性触发 clustering plan。

5.14.2 原理

一个 clustering 的任务的执行包括两部分：

- schedule plan
推荐由写任务定时触发，写参数 clustering.schedule.enabled 默认开启。
- 执行对应的 plan

5.14.3 使用方式

1) 执行命令

离线 clustering 需要手动执行 Java 程序，程序入口：

- hudi-flink1.13-bundle-0.12.0.jar
- org.apache.hudi.sink.clustering.HoodieFlinkClusteringJob

注意：必须是分区表，否则报错空指针异常。

```
// 命令行的方式
./bin/flink run -c
org.apache.hudi.sink.clustering.HoodieFlinkClusteringJob
lib/hudi-flink1.13-bundle-0.12.0.jar --path hdfs://xxx:8020/table
```

2) 参数配置

参数名	required	默认值	备注
--path	true	--	目标表的路径。
--clustering-tasks	false	-1	Clustering task 的并发，默认是待压缩 file group 的数量。
--schedule	false	false	是否要执行 schedule clustering plan 的操作，当写流程还在持续写入表数据的时候，开启这个参数有丢失查询数据的风险，所以开启该参数一定要保证当前没有任务往表里写数据，写任务的 clustering plan 默认是一直 schedule 的，除非手动关闭（默认 4 个 commits 一次 clustering）。

--seq	false	FIFO	执行压缩任务的顺序，默认是从最老的 clustering plan 开始执行，可选值： LIFO: 从最新的 plan 开始执行； FIFO: 从最老的 plan 开始执行
--target-file-max-bytes	false	1024 * 1024 * 1024	最大目标文件，默认 1GB。
--small-file-limit	false	600	小于该大小的文件会参与 clustering，默认 600MB。
--sort-columns	false	N/A	Clustering 可选排序列。
--service	false	false	是否开启 service 模式，service 模式会打开常驻进程，一直监听压缩任务并提交到集群执行（从 0.11 开始执行）。
--min-compaction-interval-seconds	false	600 (单位秒)	service 模式下的执行间隔，默认 10 分钟。

3) 案例演示

(1) 创建表，关闭在线压缩

```
create table t8(
  id int,
  age int,
  ts int,
  primary key (id) not enforced
) partitioned by (age)
with (
  'connector' = 'hudi',
  'path' = '/tmp/hudi_catalog/test/t8',
  'clustering.async.enabled' = 'false',
  'clustering.schedule.enabled' = 'true',
  'table.type' = 'COPY_ON_WRITE'
);

insert into t8 values(1,18,1);
insert into t8 values(2,18,2);
insert into t8 values(3,18,3);
insert into t8 values(4,18,4);
insert into t8 values(5,18,5);

// 命令行的方式
./bin/flink run -c
org.apache.hudi.sink.clustering.HoodieFlinkClusteringJob
lib/hudi-flink1.13-bundle-0.12.0.jar --path
hdfs://hadoop102:8020/tmp/hudi_catalog/test/t8
```

5.15 常见基础问题

5.15.1 存储一直看不到数据

如果是 streaming 写，请确保开启 checkpoint，Flink 的 writer 有 3 种刷数据到磁盘的策略：

- 当某个 bucket 在内存积攒到一定大小（可配，默认 64MB）
- 当总的 buffer 大小积攒到一定大小（可配，默认 1GB）
- 当 checkpoint 触发，将内存里的数据全部 flush 出去

5.15.2 数据有重复

如果是 COW 写，需要开启参数 `write.insert.drop.duplicates`，COW 写每个 bucket 的第一个文件默认是不去重的，只有增量的数据会去重，全局去重需要开启该参数；MOR 写不需要开启任何参数，定义好 primary key 后默认全局去重。（注意：从 0.10 版本开始，该属性改名 `write.precombine` 并且默认为 true。）

如果需要多 partition 去重，需要开启参数：`index.global.enabled` 为 true。（注意：从 0.10 版本开始，该属性默认为 true。）

索引 index 是判断数据重复的核心数据结构，`index.state.ttl` 设置了索引保存的时间，默认为 1.5 天，对于长时间周期的更新，比如更新一个月前的数据，需要将 `index.state.ttl` 调大（单位天），设置小于 0 代表永久保存。（注意：从 0.10 版本开始，该属性默认为 0。）

5.15.3 Merge On Read 写只有 log 文件

Merge On Read 默认开启了异步的 compaction，策略是 5 个 commits 压缩一次，当条件满足参会触发压缩任务，另外，压缩本身因为耗费资源，所以不一定能跟上写入效率，可能会有滞后。

可以先观察 log，搜索 compaction 关键词，看是否有 compact 任务调度：

After filtering, Nothing to compact for 关键词说明本次 compaction strategy 是不做压缩。

第 6 章 集成 Hive

Hudi 源表对应一份 HDFS 数据，通过 Spark, Flink 组件或者 Hudi CLI，可以将 Hudi 表的数据映射为 **Hive 外部表**，基于该外部表，Hive 可以方便的进行实时视图，读优化视图以及增量视图的查询。

更多 [Java - 大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

6.1 集成步骤

以 hive3.1.2、hudi 0.12.0 为例，其他版本类似。

1) 拷贝编译好的 jar 包

将 hudi-hadoop-mr-bundle-0.12.0.jar , hudi-hive-sync-bundle-0.12.0.jar 放到 hive 节点的 lib 目录下;

```
cp
/opt/software/hudi-0.12.0/packaging/hudi-hadoop-mr-bundle/target/
hudi-hadoop-mr-bundle-0.12.0.jar /opt/module/hive/lib/

cp
/opt/software/hudi-0.12.0/packaging/hudi-hive-sync-bundle/target/
hudi-hive-sync-bundle-0.12.0.jar /opt/module/hive/lib/
```

2) hive-site.xml 文件中添加如下配置信息

```
<!-- 指定存储元数据要连接的地址 -->
<property>
  <name>hive.metastore.uris</name>
  <value>thrift://hadoop102:9083</value>
</property>
```

3) 配置完后重启 hive

```
// 按照需求选择合适的方式重启
nohup hive --service metastore &
nohup hive --service hiveserver2 &
```

4) 也可以使用 hive 服务的启动脚本

```
[atguigu@hadoop102 hive]$ vim $HIVE_HOME/bin/hiveservices.sh

#!/bin/bash
HIVE_LOG_DIR=$HIVE_HOME/logs
if [ ! -d $HIVE_LOG_DIR ]
then
    mkdir -p $HIVE_LOG_DIR
fi
#检查进程是否运行正常，参数 1 为进程名，参数 2 为进程端口
function check_process()
{
    pid=$(ps -ef 2>/dev/null | grep -v grep | grep -i $1 | awk '{print $2}')
    ppid=$(netstat -nlt 2>/dev/null | grep $2 | awk '{print $7}' | cut -d '/' -f 1)
    echo $pid
    [[ "$pid" =~ "$ppid" ]] && [ "$ppid" ] && return 0 || return 1
}

function hive_start()
{
    metapid=$(check_process HiveMetastore 9083)
    cmd="nohup hive --service metastore >$HIVE_LOG_DIR/metastore.log 2>&1 &"
    [ -z "$metapid" ] && eval $cmd || echo "Metastore 服务已启动"
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
server2pid=$(check_process HiveServer2 10000)
cmd="nohup hiveserver2 >$HIVE_LOG_DIR/hiveServer2.log 2>&1 &"
[ -z "$server2pid" ] && eval $cmd || echo "HiveServer2 服务已启动"
"
}

function hive_stop()
{
metapid=$(check_process HiveMetastore 9083)
[ "$metapid" ] && kill $metapid || echo "Metastore 服务未启动"
server2pid=$(check_process HiveServer2 10000)
[ "$server2pid" ] && kill $server2pid || echo "HiveServer2 服务未启动"
}

case $1 in
"start")
hive_start
;;
"stop")
hive_stop
;;
"restart")
hive_stop
sleep 2
hive_start
;;
"status")
check_process HiveMetastore 9083 >/dev/null && echo "Metastore 服务运行正常" || echo "Metastore 服务运行异常"
check_process HiveServer2 10000 >/dev/null && echo "HiveServer2 服务运行正常" || echo "HiveServer2 服务运行异常"
;;
*)
echo Invalid Args!
echo 'Usage: '$(basename $0)' start|stop|restart|status'
;;
esac
```

6.2 Hive 同步

6.2.1 Flink 同步 Hive

1) 使用方式

Flink hive sync 现在支持两种 hive sync mode, 分别是 hms 和 jdbc 模式。其中 hms 只需要配置 metastore uris; 而 jdbc 模式需要同时配置 jdbc 属性和 metastore uris, 具体配置模版如下:

```
## hms mode 配置

CREATE TABLE t1(
  uuid VARCHAR(20),
  name VARCHAR(10),
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
age INT,
ts TIMESTAMP(3),
`partition` VARCHAR(20)
)
PARTITIONED BY (`partition`)
with(
  'connector'='hudi',
  'path' = 'hdfs://xxx.xxx.xxx:9000/t1',
  'table.type'='COPY_ON_WRITE',          -- MERGE_ON_READ 方式在没生成
parquet 文件前, hive 不会有输出
  'hive_sync.enable'='true',             -- required, 开启 hive 同步功能
  'hive_sync.table'='${hive_table}',     -- required, hive 新
建的表名
  'hive_sync.db'='${hive_db}',           -- required, hive 新建的数据
库名
  'hive_sync.mode' = 'hms',              -- required, 将hive sync mode
设置为hms, 默认jdbc
  'hive_sync.metastore.uris' = 'thrift://ip:9083' -- required,
metastore 的端口
);
```

2) 案例实操

```
CREATE TABLE t10(
  id int,
  num int,
  ts int,
  primary key (id) not enforced
)
PARTITIONED BY (num)
with(
  'connector'='hudi',
  'path' = 'hdfs://hadoop102:8020/tmp/hudi_flink/t10',
  'table.type'='COPY_ON_WRITE',
  'hive_sync.enable'='true',
  'hive_sync.table'='h10',
  'hive_sync.db'='default',
  'hive_sync.mode' = 'hms',
  'hive_sync.metastore.uris' = 'thrift://hadoop102:9083'
);

insert into t10 values(1,1,1);
```

6.2.2 Spark 同步 Hive

参数: https://hudi.apache.org/docs/basic_configurations#Write-Options

1) 使用方式

以 Spark shell 为例:

```
option("hoodie.datasource.hive_sync.enable", "true").
//设置数据集注册并同步到hive
option("hoodie.datasource.hive_sync.mode", "hms").
//使用hms
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
option("hoodie.datasource.hive_sync.metastore.uris",
"thrift://ip:9083"). //hivemetastore 地址
option("hoodie.datasource.hive_sync.username", "").
//登入 hiveserver2 的用户
option("hoodie.datasource.hive_sync.password", "").
//登入 hiveserver2 的密码
option("hoodie.datasource.hive_sync.database",
"").
//设置 hudi 与 hive 同步的数据库
option("hoodie.datasource.hive_sync.table",
"").
//设置 hudi 与 hive 同步的表名
option("hoodie.datasource.hive_sync.partition_fields",
"").
//hive 表同步的分区列
option("hoodie.datasource.hive_sync.partition_extractor_class",
"org.apache.hudi.hive.MultiPartKeyValueExtractor"). // 分区提取器
按/ 提取分区
```

2) 案例实操

```
import org.apache.hudi.QuickstartUtils._
import scala.collection.JavaConversions._
import org.apache.spark.sql.SaveMode._
import org.apache.hudi.DataSourceReadOptions._
import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.config.HoodieWriteConfig._

val tableName = "hudi_trips_cow"
val basePath = "file:///tmp/hudi_trips_cow"
val dataGen = new DataGenerator

val inserts = convertToStringList(dataGen.generateInserts(10))
val df = spark.read.json(spark.sparkContext.parallelize(inserts, 2))
    .withColumn("a", split(col("partitionpath"), "\\")(0))
    .withColumn("b", split(col("partitionpath"), "\\")(1))
    .withColumn("c", split(col("partitionpath"), "\\")(2))
df.write.format("hudi").
    options(getQuickstartWriteConfigs).
    option(PRECOMBINE_FIELD_OPT_KEY, "ts").
    option(RECORDKEY_FIELD_OPT_KEY, "uuid").
    option("hoodie.table.name", tableName).
    option("hoodie.datasource.hive_sync.enable", "true").
    option("hoodie.datasource.hive_sync.mode", "hms").
    option("hoodie.datasource.hive_sync.metastore.uris",
"thrift://hadoop102:9083").
    option("hoodie.datasource.hive_sync.database", "default").
    option("hoodie.datasource.hive_sync.table", "spark_hudi").
    option("hoodie.datasource.hive_sync.partition_fields", "a,b,c").
    option("hoodie.datasource.hive_sync.partition_extractor_class",
"org.apache.hudi.hive.MultiPartKeyValueExtractor").
    mode(Overwrite).
    save(basePath)
```

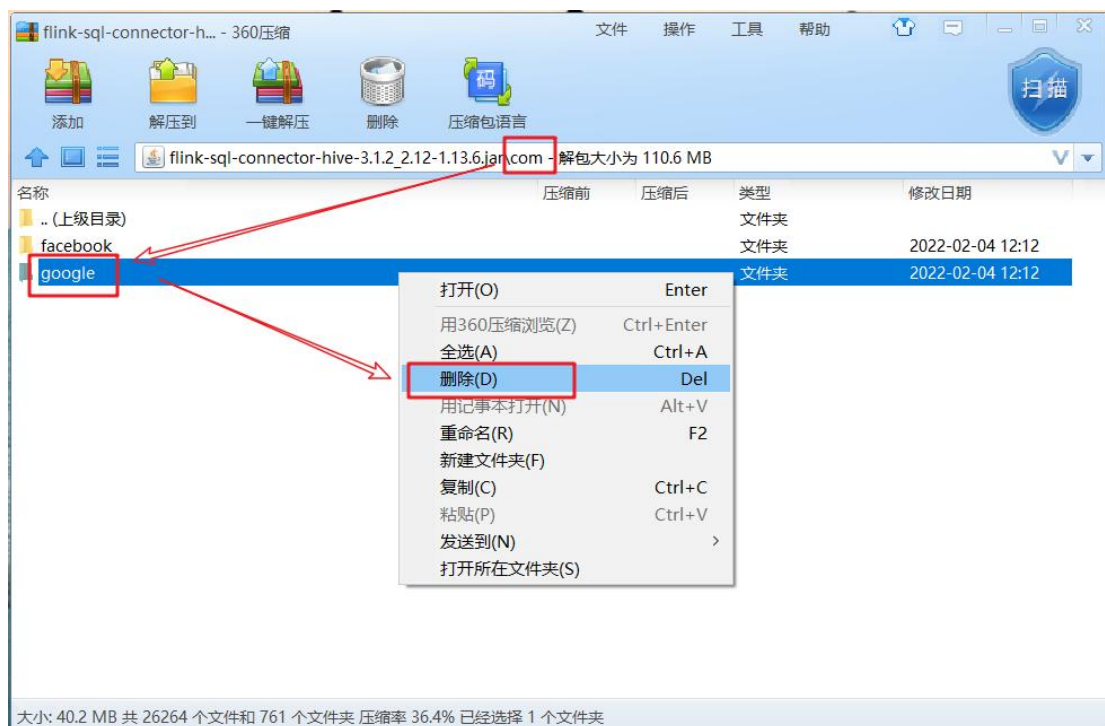
6.3 Flink 使用 HiveCatalog

6.3.1 直接使用 Hive Catalog

1) 上传 hive connector 到 flink 的 lib 中

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

hive3.1.3 的 connector 存在 guava 版本冲突，需要解决：官网下载 connector 后，用压缩软件打开 jar 包，删除/com/google 文件夹。处理完后上传 flink 的 lib 中。



2) 解决与 hadoop 的冲突

避免与 hadoop 的冲突,拷贝 hadoop-mapreduce-client-core-3.1.3.jar 到 flink 的 lib 中(5.2.1 已经做过)

3) 创建 catalog

```
CREATE CATALOG hive_catalog
WITH (
  'type' = 'hive',
  'default-database' = 'default',
  'hive-conf-dir' = '/opt/module/hive/conf',
  'hadoop-conf-dir' = '/opt/module/hadoop-3.1.3/etc/hadoop'
);

use catalog hive_catalog;

-- hive-connector 内置了 hive module, 提供了 hive 自带的系统函数
load module hive with ('hive-version'='3.1.2');
show modules;
show functions;

-- 可以调用 hive 的 split 函数
select split('a,b', ',');
```

6.4 创建 Hive 外表

一般来说 Hudi 表在用 Spark 或者 Flink 写入数据时会自动同步到 Hive 外部表 (同更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网)

6.2)，此时可以直接通过 `beeline` 查询同步的外部表，若写入引擎没有开启自动同步，则需要手动利用 `hudi` 客户端工具 `run_hive_sync_tool.sh` 进行同步，具体后面介绍。

6.5 查询 Hive 外表

6.5.1 设置参数

使用 `Hive` 查询 `Hudi` 表前，需要通过 `set` 命令设置 `hive.input.format`，否则会出现数据重复，查询异常等错误，如下面这个报错就是典型的没有设置 `hive.input.format` 导致的：

```
java.lang.IllegalArgumentException: HoodieRealtimeReader can only work on RealTimeSplit and not with xxxxxxxxxxxx
```

除此之外对于增量查询，还需要 `set` 命令额外设置 3 个参数。

```
set hoodie.mytableName.consume.mode=INCREMENTAL;
set hoodie.mytableName.consume.max.commits=3;
set hoodie.mytableName.consume.start.timestamp=commitTime;
```

注意这 3 个参数是表级别参数。

参数名	描述
hoodie.mytableName.consume.mode	Hudi 表的查询模式。 增量查询：INCREMENTAL。 非增量查询：不设置或者设为 SNAPSHOT
hoodie.mytableName.consume.start.timestamp	Hudi 表增量查询起始时间。
hoodie.mytableName.consume.max.commits	Hudi 表基于 hoodie.mytableName.consume.start.timestamp 之后要查询的增量 commit 次数。 例如： 设置为 3 时，增量查询从指定的起始时间之后 commit 3 次的数据 设为 -1 时，增量查询从指定的起始时间之后提交的所有数据

6.5.2 COW 表查询

这里假设同步的 `Hive` 外表名为 `hudi_cow`。

1) 实时视图

设置 `hive.input.format` 为以下两个之一：

➤ `org.apache.hadoop.hive.ql.io.HiveInputFormat`

➤ `org.apache.hudi.hadoop.hive.HoodieCombineHiveInputFormat`

像普通的 hive 表一样查询即可：

```
set                                     hive.input.format=
org.apache.hadoop.hive.ql.io.HiveInputFormat;
select count(*) from hudi_cow;
```

2) 增量视图

除了要设置 `hive.input.format`，还需要设置上述的 3 个增量查询参数，且增量查询语句中的必须添加 `where` 关键字并将 ``_hoodie_commit_time` > 'startCommitTime'` 作为过滤条件（这地方主要是 hudi 的小文件合并会把新旧 commit 的数据合并成新数据，hive 是没法直接从 parquet 文件知道哪些是新数据哪些是老数据）

```
set                                     hive.input.format=
org.apache.hadoop.hive.ql.io.HiveInputFormat;
set hoodie.hudicow.consume.mode= INCREMENTAL;
set hoodie.hudicow.consume.max.commits=3;
set hoodie.hudicow.consume.start.timestamp= xxxx;
select count(*) from hudicow where `_hoodie_commit_time` > 'xxxx'
-- （这里注意`_hoodie_commit_time` 的引号是反引号（tab 键上面那个）不是单引号， 'xxxx'是单引号）
```

6.5.3 MOR 表查询

这里假设 MOR 类型 Hudi 源表的表名为 `hudi_mor`，映射为两张 Hive 外部表 `hudi_mor_ro`（ro 表）和 `hudi_mor_rt`（rt 表）。

1) 实时视图

设置了 `hive.input.format` 之后，即可查询到 Hudi 源表的最新数据

```
set                                     hive.input.format=
org.apache.hadoop.hive.ql.io.HiveInputFormat;
select * from hudicow_rt;
```

2) 读优化视图

ro 表全称 `read optimized table`，对于 MOR 表同步的 `xxx_ro` 表，只暴露压缩后的 parquet。其查询方式和 COW 表类似。设置完 `hiveInputFormat` 之后 和普通的 Hive 表一样查询即可。

3) 增量视图

这个增量查询针对的 rt 表，不是 ro 表。同 COW 表的增量查询类似：

```
set
hive.input.format=org.apache.hudi.hadoop.hive.HoodieCombineHiveInputFormat; // 这地方指定为 HoodieCombineHiveInputFormat
set hoodie.hudimor.consume.mode=INCREMENTAL;
set hoodie.hudimor.consume.max.commits=-1;
set hoodie.hudimor.consume.start.timestamp=xxxx;
```

```
select * from hudimor_rt where `_hoodie_commit_time`>'xxxx';// 这个表名要是 rt 表索引
```

说明：

- set hive.input.format=org.apache.hudi.hadoop.hive.HoodieCombineHiveInputFormat; 最好只用于 rt 表的增量查询 当然其他种类的查询也可以设置为这个，这个参数会影响到普通的 hive 表查询，因此在 rt 表增量查询完成后，应该设置 set hive.input.format=org.apache.hadoop.hive.ql.io.HiveInputFormat; 或者改为默认值 set hive.input.format=org.apache.hadoop.hive.ql.io.CombineHiveInputFormat; 用于其他表的查询。
- set hoodie.mytableName.consume.mode=INCREMENTAL; 仅用于该表的增量查询模式，若要对表切换为其他查询模式，应设置 set hoodie.hudisourcetableName.consume.mode=SNAPSHOT;

6.6 hive sync tool

若写入引擎没有开启自动同步，则需要手动利用 Hudi 客户端工具进行同步，Hudi 提供 Hive sync tool 用于同步 Hudi 最新的元数据（包含自动建表、增加字段、同步分区信息）到 hive metastore。

Hive sync tool 提供三种同步模式，JDBC，HMS，HIVEQL。这些模式只是针对 Hive 执行 DDL 的三种不同方式。在这些模式中，JDBC 或 HMS 优于 HIVEQL，HIVEQL 主要用于运行 DML 而不是 DDL。

6.6.1 使用语法及参数

脚本位置在 hudi 源码路径下的 hudi-sync/hudi-hive-sync/run_sync_tool.sh

1) 语法

```
#查看语法帮助
./run_sync_tool.sh --help

#语法:
./run_sync_tool.sh \
--jdbc-url jdbc:hive2://\hiveserver:10000 \
--user hive \
--pass hive \
--partitioned-by partition \
--base-path <basePath> \
--database default \
--table <tableName>
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

从 Hudi 0.5.1 版本开始，读时合并优化版本的表默认带有'_ro'后缀。为了向后兼容旧的 Hudi 版本，提供了一个可选的配置 `--skip-ro-suffix`，如果需要，可以关闭'_ro'后缀。

2) 参数说明

HiveSyncConfig	DataSourceWriteOption	描述
<code>--database</code>	<code>hoodie.datasource.hive_sync.database</code>	同步到 hive 的目标库名
<code>--table</code>	<code>hoodie.datasource.hive_sync.table</code>	同步到 hive 的目标表名
<code>--user</code>	<code>hoodie.datasource.hive_sync.username</code>	hive metastore 用户名
<code>--pass</code>	<code>hoodie.datasource.hive_sync.password</code>	hive metastore 密码
<code>--use-jdbc</code>	<code>hoodie.datasource.hive_sync.use_jdbc</code>	使用 JDBC 连接到 hive metastore
<code>--jdbc-url</code>	<code>hoodie.datasource.hive_sync.jdbcurl</code>	Hive metastore url
<code>--sync-mode</code>	<code>hoodie.datasource.hive_sync.mode</code>	同步 hive 元数据的方式. 有效值为 <code>hms</code> , <code>jdbc</code> 和 <code>hiveql</code> .
<code>--partitioned-by</code>	<code>hoodie.datasource.hive_sync.partition_fields</code>	hive 分区字段名，多个字段使用逗号连接。
<code>--partition-value-extractor</code>	<code>hoodie.datasource.hive_sync.partition_extractor_class</code>	解析分区值的类名，默认 <code>SlashEncodedDayPartitionValueExtractor</code>

6.6.2 解决依赖问题

`run_sync_tool.sh` 这个脚本就是查找 hadoop、hive 和 bundle 包的依赖，实际上使用的时候会报错各种 `ClassNotFoundException`、`NoSuchMethod`，所以要动手修改依赖的加载逻辑：

```
vim
/opt/software/hudi-0.12.0/hudi-sync/hudi-hive-sync/run_sync_tool.sh
```

1) 修改 hadoop、hive、hudi-hive-sync-bundle-0.12.0.jar 的依赖加载

(1) 将 34 行 `HUDI_HIVE_UBER_JAR=xxxx` 注释掉

```
23 #
24 if [ -z "${HADOOP_HOME}" ]; then
25     error_exit "Please make sure the environment variable HADOOP_HOME is setup"
26 fi
27
28 if [ -z "${HIVE_HOME}" ]; then
29     error_exit "Please make sure the environment variable HIVE_HOME is setup"
30 fi
31
32 DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && pwd )"
33 #Ensure we pick the right jar even for hive11 builds
34 #HUDI_HIVE_UBER_JAR="ls -c $DIR/../../packaging/hudi-hive-sync-bundle/target/hudi-hive-sync-*.jar | grep -v source | head -1"
35
36 if [ -z "${HADOOP_CONF_DIR}" ]; then
37     echo "setting hadoop conf dir"
38     HADOOP_CONF_DIR="${HADOOP_HOME}/etc/hadoop"
39 fi
40
41 ## Include only specific packages from HIVE_HOME/lib to avoid version mismatches
42 HIVE_EXEC="ls ${HIVE_HOME}/lib/hive-exec-*.jar | tr '\n' ':'"
43 HIVE_SERVICE="ls ${HIVE_HOME}/lib/hive-service-*.jar | grep -v rpc | tr '\n' ':'"
44 HIVE_METASTORE="ls ${HIVE_HOME}/lib/hive-metastore-*.jar | tr '\n' ':'"
45 HIVE_JDBC="ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | tr '\n' ':'"
46 if [ -z "${HIVE_JDBC}" ]; then
47     HIVE_JDBC="ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | grep -v handler | tr '\n' ':'"
48 fi
49 HIVE_JACKSON="ls ${HIVE_HOME}/lib/jackson-*.jar | tr '\n' ':'"
```

(2) 将 52 行 `HADOOP_HIVE_JARS=xxx` 注释掉

```
#在 54 行 添加如下：
HADOOP_HIVE_JARS=`hadoop classpath`:${HIVE_HOME}/lib/*
HUDI_HIVE_UBER_JAR=/opt/software/hudi-0.12.0/packaging/hudi-hive-
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

sync-bundle/target/hudi-hive-sync-bundle-0.12.0.jar

```

33 #Ensure we pick the right jar even for hive11 builds
34 #HUDI_HIVE_UBER_JAR='ls -c $DIR/../../packaging/hudi-hive-sync-bundle/target/hudi-hive-sync-*.jar | grep -v source | head -1'
35
36 if [ -z "$HADOOP_CONF_DIR" ]; then
37     echo "setting hadoop conf dir"
38     HADOOP_CONF_DIR="$HADOOP_HOME/etc/hadoop"
39 fi
40
41 ## Include only specific packages from HIVE_HOME/lib to avoid version mismatches
42 HIVE_EXEC='ls ${HIVE_HOME}/lib/hive-exec-*.jar | tr '\n' ':'
43 HIVE_SERVICE='ls ${HIVE_HOME}/lib/hive-service-*.jar | grep -v rpc | tr '\n' ':'
44 HIVE_METASTORE='ls ${HIVE_HOME}/lib/hive-metastore-*.jar | tr '\n' ':'
45 HIVE_JDBC='ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | tr '\n' ':'
46 if [ -z "${HIVE_JDBC}" ]; then
47     HIVE_JDBC='ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | grep -v handler | tr '\n' ':'
48 fi
49 HIVE_JACKSON='ls ${HIVE_HOME}/lib/jackson-*.jar | tr '\n' ':'
50 HIVE_JARS=$HIVE_METASTORE:$HIVE_SERVICE:$HIVE_EXEC:$HIVE_JDBC:$HIVE_JACKSON
51
52 #HADOOP_HIVE_JARS=${HIVE_JARS}:${HADOOP_HOME}/share/hadoop/common/*:${HADOOP_HOME}/share/hadoop/mapreduce/*:${HADOOP_HOME}/share/
53 #hadoop/hdfs/*:${HADOOP_HOME}/share/hadoop/common/lib/*:${HADOOP_HOME}/share/hadoop/hdfs/lib/*
54
55 HADOOP_HIVE_JARS='hadoop classpath':$HIVE_HOME/lib/*
56 HUDI_HIVE_UBER_JAR=/opt/software/hudi-0.12.0/packaging/hudi-hive-sync-bundle/target/hudi-hive-sync-bundle-0.12.0.jar 手动添加行
57
58 echo "Running Command : java -cp ${HADOOP_HIVE_JARS}:${HADOOP_CONF_DIR}:${HUDI_HIVE_UBER_JAR} org.apache.hudi.hive.HiveSyncTool $@"
59 java -cp ${HUDI_HIVE_UBER_JAR}:${HADOOP_HIVE_JARS}:${HADOOP_CONF_DIR} org.apache.hudi.hive.HiveSyncTool "$@"

```

2) 解决 parquet-column 的版本冲突

(1) 上传 parquet-column-1.12.2.jar 到/opt/software/, 脚本中添加如下:

```

PARQUET_JAR=/opt/software/parquet-column-1.12.2.jar
34 #HUDI_HIVE_UBER_JAR='ls -c $DIR/../../packaging/hudi-hive-sync-bundle/target/hudi-hive-sync-*.jar | grep -v source | head -1'
35
36 if [ -z "$HADOOP_CONF_DIR" ]; then
37     echo "setting hadoop conf dir"
38     HADOOP_CONF_DIR="$HADOOP_HOME/etc/hadoop"
39 fi
40
41 ## Include only specific packages from HIVE_HOME/lib to avoid version mismatches
42 HIVE_EXEC='ls ${HIVE_HOME}/lib/hive-exec-*.jar | tr '\n' ':'
43 HIVE_SERVICE='ls ${HIVE_HOME}/lib/hive-service-*.jar | grep -v rpc | tr '\n' ':'
44 HIVE_METASTORE='ls ${HIVE_HOME}/lib/hive-metastore-*.jar | tr '\n' ':'
45 HIVE_JDBC='ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | tr '\n' ':'
46 if [ -z "${HIVE_JDBC}" ]; then
47     HIVE_JDBC='ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | grep -v handler | tr '\n' ':'
48 fi
49 HIVE_JACKSON='ls ${HIVE_HOME}/lib/jackson-*.jar | tr '\n' ':'
50 HIVE_JARS=$HIVE_METASTORE:$HIVE_SERVICE:$HIVE_EXEC:$HIVE_JDBC:$HIVE_JACKSON
51
52 #HADOOP_HIVE_JARS=${HIVE_JARS}:${HADOOP_HOME}/share/hadoop/common/*:${HADOOP_HOME}/share/hadoop/mapreduce/*:${HADOOP_HOME}/share/
53 #hadoop/hdfs/*:${HADOOP_HOME}/share/hadoop/common/lib/*:${HADOOP_HOME}/share/hadoop/hdfs/lib/*
54
55 HADOOP_HIVE_JARS='hadoop classpath':$HIVE_HOME/lib/*
56 HUDI_HIVE_UBER_JAR=/opt/software/hudi-0.12.0/packaging/hudi-hive-sync-bundle/target/hudi-hive-sync-bundle-0.12.0.jar
57 PARQUET_JAR=/opt/software/parquet-column-1.12.2.jar
58
59 echo "Running Command : java -cp ${HADOOP_HIVE_JARS}:${HADOOP_CONF_DIR}:${HUDI_HIVE_UBER_JAR} org.apache.hudi.hive.HiveSyncTool $@"
60 java -cp ${HUDI_HIVE_UBER_JAR}:${HADOOP_HIVE_JARS}:${HADOOP_CONF_DIR} org.apache.hudi.hive.HiveSyncTool "$@"

```

(2) 拼接路径到命令最前面(只能最前面!)

```

${PARQUET_JAR}:
34 #HUDI_HIVE_UBER_JAR='ls -c $DIR/../../packaging/hudi-hive-sync-bundle/target/hudi-hive-sync-*.jar | grep -v source | head -1'
35
36 if [ -z "$HADOOP_CONF_DIR" ]; then
37     echo "setting hadoop conf dir"
38     HADOOP_CONF_DIR="$HADOOP_HOME/etc/hadoop"
39 fi
40
41 ## Include only specific packages from HIVE_HOME/lib to avoid version mismatches
42 HIVE_EXEC='ls ${HIVE_HOME}/lib/hive-exec-*.jar | tr '\n' ':'
43 HIVE_SERVICE='ls ${HIVE_HOME}/lib/hive-service-*.jar | grep -v rpc | tr '\n' ':'
44 HIVE_METASTORE='ls ${HIVE_HOME}/lib/hive-metastore-*.jar | tr '\n' ':'
45 HIVE_JDBC='ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | tr '\n' ':'
46 if [ -z "${HIVE_JDBC}" ]; then
47     HIVE_JDBC='ls ${HIVE_HOME}/lib/hive-jdbc-*.jar | grep -v handler | tr '\n' ':'
48 fi
49 HIVE_JACKSON='ls ${HIVE_HOME}/lib/jackson-*.jar | tr '\n' ':'
50 HIVE_JARS=$HIVE_METASTORE:$HIVE_SERVICE:$HIVE_EXEC:$HIVE_JDBC:$HIVE_JACKSON
51
52 #HADOOP_HIVE_JARS=${HIVE_JARS}:${HADOOP_HOME}/share/hadoop/common/*:${HADOOP_HOME}/share/hadoop/mapreduce/*:${HADOOP_HOME}/share/
53 #hadoop/hdfs/*:${HADOOP_HOME}/share/hadoop/common/lib/*:${HADOOP_HOME}/share/hadoop/hdfs/lib/*
54
55 HADOOP_HIVE_JARS='hadoop classpath':$HIVE_HOME/lib/*
56 HUDI_HIVE_UBER_JAR=/opt/software/hudi-0.12.0/packaging/hudi-hive-sync-bundle/target/hudi-hive-sync-bundle-0.12.0.jar
57 PARQUET_JAR=/opt/software/parquet-column-1.12.2.jar
58
59 echo "Running Command : java -cp ${HADOOP_HIVE_JARS}:${HADOOP_CONF_DIR}:${HUDI_HIVE_UBER_JAR} org.apache.hudi.hive.HiveSyncTool $@"
60 java -cp ${PARQUET_JAR}:${HUDI_HIVE_UBER_JAR}:${HADOOP_HIVE_JARS}:${HADOOP_CONF_DIR} org.apache.hudi.hive.HiveSyncTool "$@"
-- 插入 --

```

(3) 保存退出

6.6.2 JDBC 模式同步

通过 hive2 jdbc 协议同步, 提供的是 hive server2 的地址, 如 jdbc:hive2://hive-server:10000。

默认为 jdbc。

```
cd /opt/software/hudi-0.12.0/hudi-sync/hudi-hive-sync
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网

```
./run_sync_tool.sh \  
--base-path hdfs://hadoop102:8020/tmp/hudi_flink/t12/ \  
--database default \  
--table t12_flink \  
--jdbc-url jdbc:hive2://hadoop102:10000 \  
--user atguigu \  
--pass 123456 \  
--partitioned-by num
```

6.6.2 HMS 模式同步

提供 hive metastore 的地址，如 thrift://hms:9083，通过 hive metastore 的接口完成同步。

使用时需要设置 --sync-mode=hms。

如果使用的是远程 metastore，那么确保 hive-site.xml 配置文件中设置 hive.metastore.uris。

```
./run_sync_tool.sh \  
--base-path hdfs://hadoop102:8020/tmp/hudi_flink/t3 \  
--database default \  
--table t3_flink \  
--user atguigu \  
--pass atguigu \  
--partitioned-by age \  
--sync-mode hms \  
--jdbc-url thrift://hadoop102:9083
```